



MANSOURA UNIVERSITY
FACULTY OF ENGINEERING
ARTIFICIAL INTELLIGENCE ENGINEERING PROGRAM

MR.FIT
AI-Driven Fitness Coaching & Tracking Platform
INTELLIGENCE MEETS IRON

Project Members:

Member Name	ID Number
Yousef Mohammed Sorour	808970673
Zyad Ehab El-Feki	812348537
Mohammed El-sayed Fathy	808600152
Islam El-sayed Abo Arab	812121613
Ahmed Mohammed Ahmed Ali	815281325
Yahia Alaa Mohammed	812354505

Supervisors:

Dr. Ayah El-Assy
Eng. Mohammed Abdo Mohammed

Academic Year 2025 – 2026

MANSOURA UNIVERSITY
FACULTY OF ENGINEERING
ARTIFICIAL INTELLIGENCE ENGINEERING PROGRAM

MR.FIT
AI-Driven Fitness Coaching & Tracking Platform
INTELLIGENCE MEETS IRON

Technical Documentation Submitted as a Graduation Project
Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Artificial
Intelligence Engineering

Academic Year 2025 – 2026

ABSTRACT

MR.FIT is an AI-driven fitness coaching and tracking Software-as-a-Service (SaaS) web application developed as an engineering graduation project. The platform functions as a fully autonomous digital trainer that delivers personalized workout plans, monitors performance, analyzes nutrition, and provides adaptive coaching feedback by reasoning over each user's goals, physical metrics, and historical activity data.

The system is implemented using a webhook-centric backend architecture built on n8n workflow orchestration (self-hosted on a Hostinger VPS), a Next.js 14 frontend deployed on Vercel, and a Supabase-managed PostgreSQL database with the pgvector extension for Agentic Retrieval-Augmented Generation (RAG). AI inference is handled by the Anthropic Claude API for conversational coaching and Google Gemini 2.5 Flash for natural language parsing and exercise recognition.

A secondary machine learning subsystem processes raw inertial measurement unit (IMU) data from an on-wrist wearable sensor to classify barbell exercises and count repetitions in real time using a Random Forest classifier trained on a custom six-class dataset (five exercise classes plus a Rest class), achieving a weighted F1-score of 0.96.

The platform operates on a three-tier freemium subscription model (Free, Pro at \$12.99/month, and Elite at \$24.99/month), with Stripe handling billing and subscription state synchronization.

TABLE OF CONTENTS

ABSTRACT.....	1
TABLE OF CONTENTS.....	2
LIST OF FIGURES	12
LIST OF TABLES	14
CHAPTER 1: INTRODUCTION	1
1.1 Overview	1
1.2 Problem Statement	2
1.3 Project Objectives	2
1.3.1 Functional Objectives	3
1.3.2 Technical Objectives.....	3
1.3.3 Commercial Objectives.....	3
1.4 Scope of the System.....	3
1.5 Technology Stack.....	4
1.6 Architectural Principle: A Webhook-Centric Backend	5
1.7 Monetization Model.....	5
CHAPTER 2: SYSTEM ANALYSIS.....	7
2.1 Review of Existing Fitness and Health Solutions.....	7
2.1.1 Dietary Assessment and the Friction of Manual Logging: MyFitnessPal	7
2.1.2 Resistance Training Utilities and the Deficit of Analytical Progression: Strong App.....	7
2.1.3 Algorithmic Heuristics vs. Physiological Reality: Fitbod	8
2.1.4 Wearable Ecosystems and the Deficit of Prescriptive Coaching: Apple Fitness+ and Google Fit.....	8

Summary of Existing Platform Limitations	8
2.2 Identified Research Gap.....	9
2.3 Stakeholder Analysis	9
2.3.1 Primary Stakeholder: The End User (Trainee/Athlete)	10
2.3.2 Secondary Stakeholder: The System Developer and Administrator.....	10
2.3.3 Tertiary Stakeholder: Academic Supervisors and Evaluators	11
2.4 User Requirements.....	11
2.4.1 Functional Requirements (FR).....	11
2.4.2 Non-Functional Requirements (NFR).....	12
2.5 Use Case Analysis.....	12
2.5.1 Use Case Specifications.....	12
2.6 User Stories.....	15
2.6.1 Functional User Stories.....	16
2.6.2 Non-Functional User Stories.....	18
2.7 System Constraints.....	18
2.8 Feasibility Study	19
2.8.1 Technical Feasibility	19
2.8.2 Economic Feasibility	20
2.8.3 Operational Feasibility.....	20
CHAPTER 3: SYSTEM DESIGN.....	22
3.1 System Overview	22
3.2 Overall Architecture.....	22
3.2.1 Architectural Topology	23
3.3 Module Breakdown.....	23
3.4 Data Flow Design	24

3.5 Component Diagram	24
3.5.1 System Component Specifications	24
3.6 Sequence Diagrams	25
3.6.1 User Authentication and Onboarding Sequence	25
3.6.2 Stripe Webhook Subscription Sync Sequence	25
3.6.3 Wearable Telemetry Streaming and Inference Sequence	26
3.7 Database Design Overview	26
3.8 Security Considerations	27
3.9 Technology Stack Rationale	27
CHAPTER 4: UI/UX DESIGN	29
4.1 Introduction	29
4.2 UI/UX Design Objectives	30
4.2.1 Simplicity	30
4.2.2 Consistency	31
4.2.3 Accessibility	31
4.2.4 Responsiveness	31
4.2.5 User Engagement	32
4.3 Design System	32
4.3.1 Color Palette	33
4.3.2 Typography	33
4.3.3 Icons and Visual Elements	34
4.3.4 Reusable Components	35
4.4 Information Architecture	36
4.4.1 Navigation Structure	37
4.4.2 Module Organization	37

4.4.3 User Journey	38
4.5 Landing Page Design	39
4.5.1 Purpose.....	39
4.5.2 Layout Structure.....	40
4.5.3 Call-to-Action Design.....	40
4.5.4 Feature Showcase Section.....	41
4.6 Authentication Interfaces	42
4.6.1 Registration Screen	42
4.6.2 Login Screen	42
4.6.3 User Onboarding Flow.....	43
4.7 Dashboard Design.....	44
4.7.1 Dashboard Objectives	44
4.7.2 Statistics Cards.....	45
4.7.3 Nutrition Overview	45
4.7.4 Coach Insights.....	46
4.7.5 Recent Activity	47
4.8 AI Coach Interface	47
4.8.1 Conversational Design.....	48
4.8.2 Quick Actions	49
4.8.3 Personalized Recommendations	49
4.8.4 Context-Aware Responses	50
4.9 Workout Management Interface	51
4.9.1 Workout Creation.....	52
4.9.2 Session Logging.....	52
4.9.3 Exercise Selection.....	53

4.9.4 Workout History	53
4.10 Exercise Library Interface.....	55
4.10.1 Exercise Categorization	55
4.10.2 Search and Filtering	56
4.10.3 Exercise Information Display	56
4.11 Nutrition Tracking Interface	57
4.11.1 Meal Logging.....	58
4.11.2 Macronutrient Tracking	58
4.11.3 Daily Goal Monitoring.....	59
4.12 Analytics and Progress Interface	60
4.12.1 Performance Metrics	61
4.12.2 Strength Analysis	61
4.12.3 Volume Tracking	62
4.12.4 Progress Visualization	62
4.13 Settings and Profile Management	64
4.13.1 Profile Information.....	64
4.13.2 Fitness Preferences.....	65
4.14 Responsive Design Considerations.....	65
4.15 Usability Considerations.....	66
4.15.1 Readability	67
4.15.2 Navigation Efficiency	67
4.15.3 Accessibility.....	68
4.15.4 Error Prevention.....	68
4.16 Design Challenges and Improvements.....	69
CHAPTER 5: FRONTEND DEVELOPMENT	71

5.1 Architecture & Framework	71
5.2 Routing & Folder Structure	71
5.3 Authentication & Session Management.....	72
5.4 Magic Input Voice Transcriber	72
5.4.1 Speech Recognition Flow	72
5.5 Data Visualization.....	73
CHAPTER 6: DATABASE AND API DESIGN	74
6.1 Database Architecture and Rationale.....	74
6.1.1 The UUID Type Choice.....	75
6.2 Relational Schema Design	75
6.3.3 Food Image Analysis Endpoint (Vision Model & Fallback).....	80
CHAPTER 7: BACKEND AUTOMATION USING N8N.....	82
7.1 Introduction.....	82
7.2 Introduction to n8n.....	82
7.3 The Webhook Contract	83
7.3.1 Example: The Workout Plan Endpoint	83
CHAPTER 8: FITNESS TRACKER AI SUBSYSTEM	86
8.1 Introduction.....	86
8.2 Problem Definition.....	87
8.3 System Architecture.....	87
8.3.1 Physical Sensing Layer	87
8.3.2 Embedded Control Layer	88
8.3.3 Server-Side Processing Layer	88
8.3.4 Presentation Layer	88
8.4 Dataset Description.....	88

8.5 Data Preprocessing.....	89
8.5.1 Data Visualization and Exploratory Analysis.....	89
8.5.2 Outlier Detection.....	90
8.5.3 Low-Pass Butterworth Filtering.....	90
8.6 Feature Engineering	90
8.6.1 Temporal Statistical Features.....	91
8.6.2 Frequency-Domain Features (DFT).....	91
8.6.3 PCA Dimensionality Reduction.....	91
8.6.4 K-Means Cluster Features.....	92
8.7 Model Training	92
8.7.1 Algorithm Selection.....	92
8.7.2 Training Protocol	92
8.7.3 Hyperparameter Tuning	93
8.8 Model Evaluation.....	93
8.8.1 Classification Performance	93
8.9 Hardware Integration	94
8.9.1 MPU-6050 IMU.....	94
8.9.2 ESP32 Microcontroller	94
8.9.3 OLED Display and Power	95
8.10 Real-Time Prediction and Repetition Counting.....	95
8.10.1 Online Inference Pipeline	95
8.10.2 Repetition Counting Algorithm	96
8.11 API Integration.....	96
8.12 Deployment and Production Considerations	98
8.13 Challenges and Lessons Learned	99

8.13.1 Wi-Fi Latency Variability	99
8.13.2 Pandas Rolling Window Configuration	99
8.13.3 OneDrive Git Conflicts	99
8.13.4 Class Imbalance and Rest Category	99
8.13.5 Sensor Drift and Recalibration.....	100
CHAPTER 9: TESTING AND QUALITY ASSURANCE	101
9.1 Software Testing Theory and Architectural Validation.....	102
9.1.1 Classical Software Testing Paradigms and the V-Model	102
9.1.2 Adapting the Testing Pyramid for Distributed Microservices	103
9.2 Security Testing for Stateless Authentication and Data Isolation	106
9.2.1 The Cryptographic Vulnerability Matrix of Stateless Authentication	106
9.2.2 Multi-Tenant Database Isolation and Row-Level Security Verification	108
9.3 Hardware-in-the-Loop (HIL) Testing Theory and Emulation	109
9.3.1 Theoretical Foundations of Real-Time Systems and Hardware-in-the-Loop.....	109
9.3.2 Synthesizing Cyber-Physical Environments via SensorSimulator	110
9.4 Machine Learning Model Validation Theory and Statistical Rigor.....	113
9.4.1 The Statistical Mechanics of Temporal Data Leakage	113
9.4.2 Cross-Validation Protocols for Kinematic Feature Spaces.....	114
9.4.3 Continuous Error Metrics for Digital Signal Processing Pipelines	115
CHAPTER 10: DEPLOYMENT AND DEVOPS	118
10.1 Presentation Layer Deployment (Next.js & Vercel).....	118
10.1.1 Continuous Integration and Deployment (CI/CD) Pipeline.....	118
10.1.2 Environment Integration	118
10.2 Backend Orchestration Layer (n8n Hostinger VPS).....	119
10.2.1 VPS Server Specifications	119

10.2.2 n8n Self-Hosting and Docker Setup	119
CHAPTER 11: RESULTS AND DISCUSSION.....	123
11.1 System Performance and Latency Analysis.....	123
11.1.1 Webhook and API Orchestration Latency	123
11.1.2 Generative AI Coaching Latency.....	123
11.2 UI/UX and Frontend Responsiveness	124
11.2.1 Component Interaction Speeds	124
11.3 AI Subsystem Classifier Evaluation	124
11.3.1 Classifier Comparison.....	124
11.3.2 Analysis of Classifier Results	125
11.4 Repetition Counting Accuracy.....	125
11.4.1 Error Metrics by Exercise Class	126
11.4.2 Discussion of Repetition Counting Anomalies.....	126
11.5 Hardware Latency Budget Validation	126
11.6 Limitations and Future Validation	127
CHAPTER 12: CONCLUSION AND FUTURE WORK.....	128
12.1 Project Conclusion and Synthesis.....	128
12.2 Project Summary and Key Achievements	128
12.3 Future Work and Engineering Roadmap	129
12.3.1 Cryptographic JWT Signature Verification	130
12.3.2 On-Wrist TinyML and Hardware V2	130
12.3.3 Frontend Proxy Rate Limiting	130
12.3.4 B2B Gym and Trainer Portal	131
12.3.5 Vision-Based Joint Form Checking	131
12.3.6 Biometric-Driven Progressive Overload Heuristics	131

REFERENCES	132
------------------	-----

LIST OF FIGURES

MR-FIT Branding Logo.....	35
MR-FIT Alternative Logo Variations	35
MR-FIT Public Landing Page User Interface	42
MR-FIT Sign-In and Authentication Interface	43
MR-FIT Core User Dashboard Interface	47
MR-FIT Conversational AI Coach Interface	51
MR-FIT Active Exercise Session Logger Interface.....	54
MR-FIT AI Computer Vision Exercise Form Analysis Interface	55
MR-FIT Nutrition Logger and Macronutrient Tracking Interface	59
MR-FIT Progress Tracker and Analytics Dashboard	63
MR-FIT AI Recovery Engine and Sleep Tracking Interface.....	63
MR-Fit Fitness Tracker AI Subsystem: Complete System Architecture showing the four-layer design (Hardware, Embedded Control, Server Processing, Presentation).....	86
Signal preprocessing pipeline illustrated on the acc_z channel of an Overhead Press set: (a) raw signal with detected outliers, (b) after Chauvenet interpolation, (c) after 4th-order Butterworth low-pass filter ($f_c = 20$ Hz) with annotated repetition boundaries.	90
Feature engineering: (a) sliding window segmentation with 50% overlap, (b) DFT spectrum of a representative Overhead Press window with dominant frequency at 1.2 Hz, (c) top-10 feature importances from the trained Random Forest (MDI metric).	91
Normalized confusion matrix (%) for the six-class Random Forest classifier on the held-out test set. Diagonal entries represent correctly classified windows; off-diagonal entries represent misclassifications.	94
Comparative evaluation of four candidate classifiers: (a) test-set accuracy and (b) model training time. Random Forest achieves the highest accuracy with moderate training time. ..	94

Repetition counting algorithm applied to a Barbell Bench Press set: (a) low-pass filtered accelerometer magnitude signal, (b) band-pass filtered signal with detected peak markers, (c) cumulative repetition count step function.....	96
MR-FIT Smart Tracker Wearable Real-Time Classifier Interface	97

LIST OF TABLES

synthesizes the operational gaps and architectural limitations present in current commercial fitness solutions, highlighting the necessity for a unified, generative AI framework.	8
System Component Specifications	25
Database Schema Tables and Relations.....	26
MR-FIT Color Palette	33
MR-FIT Color Palette	33
Typography System	33
Reusable Components.....	35
Typical User Journey in MR-FIT	38
Supabase PostgreSQL Database Schema (DDL).....	75
Cosine-Similarity RAG Retrieval Query using pgvector	78
HNSW Vector Index Creation on user_embeddings	78
Row-Level Security (RLS) Policy Definitions	78
Request Header Structure for Parse Entry Endpoint.....	79
Request Body Payload for Parse Entry Endpoint	79
Response Body Payload for Parse Entry Endpoint.....	79
Request Header Structure for Scan Food Endpoint	80
Request Body Payload for Scan Food Endpoint.....	80
Response Body Payload for Scan Food Endpoint	80
Request Header Structure for Chat Endpoint.....	81
Request Body Payload for Chat Endpoint	81
Response Body Payload for Chat Endpoint.....	81
Stripe Webhook Payload Example (customer.subscription.updated).....	81

API Payload Structure for Workout Plan Generation	83
JavaScript JWT Signature Validation Node Implementation	84
SQL Database Schema Overview in Backend Automation Context	84
JWT Signature Verification Node Error Object	85
Redacted Authorization Header Structure	85
Diagnostic JavaScript Log Probe in n8n Code Node.....	85
Dataset composition: exercises, collection statistics, and representative cadence.	89
Composition of the 73-dimensional feature vector used by the Random Forest classifier. ...	92
Hyperparameter grid search results for the Random Forest classifier.	93
Per-class classification performance of the Random Forest model on the held-out test set..	94
Hardware component specifications for the MPU-6050 and ESP32 wearable module.	95
Repetition counting accuracy by exercise class vs. ground-truth manual video annotation. .	96
MR-Fit FastAPI endpoint specification.	97
End-to-end latency breakdown across all system components (N = 500 measured transactions).	98
Docker Compose Configuration for Self-Hosted n8n Orchestrator	119
Nginx Reverse Proxy Configuration for Webhook Security	120
systemd Service Configuration for FastAPI Microservices.....	120
Windows Batch Script to Orchestrate Local Development Services.....	121
Docker Compose Configuration for Open Wearables Bridge Service	122
Future Work and Engineering Roadmap Comparison.....	129

CHAPTER 1: INTRODUCTION

MR.FIT is an AI-driven fitness coaching and tracking Software-as-a-Service (SaaS) web application developed as an engineering graduation project. The platform is designed to function as a fully autonomous digital trainer that delivers personalized workout plans, monitors performance, analyzes nutrition, and provides adaptive feedback by reasoning over each user's goals, physical metrics, and historical activity data. Unlike conventional fitness applications that act as passive logging tools, MR.FIT integrates a Large Language Model — accessed through the Anthropic Claude API [30] — into every layer of the user experience, allowing the system to coach, correct, and motivate users in natural language.

The project is conceived from the outset as a production-grade, publicly deployable product with a tiered subscription model, rather than as a classroom prototype. It targets a market currently dominated by hardware-tethered fitness brands and static one-size-fits-all training apps, and competes on the strength of true personalization driven by Agentic Retrieval-Augmented Generation (Agentic RAG).

1.1 Overview

Traditional fitness applications typically offer static, predefined workout plans and passive logging interfaces. Users must manually input their exercises, calculate their macronutrients, and interpret their own progress charts. MR.FIT addresses these limitations by introducing an active, intelligent software agent that continuously monitors, analyzes, and guides the user. The platform provides:

- **Dynamic Training Generation:** Automated creation of customized workout routines that adapt week-over-week based on logged performance metrics.
- **Conversational AI Coaching:** An on-demand digital personal trainer capable of natural language interaction, grounded in the user's historical performance, biometrics, and preferences.
- **Frictionless Nutrition Tracking:** A natural language and voice-enabled logging system that parses conversational entries (e.g., *"had two eggs and a toast"*) into structured macronutrient logs using LLM-powered semantic parsing.

- **Biometric Wearable Integration:** Aggregation of sleep, heart rate, and activity metrics from connected smartwatches to compute daily recovery readiness scores.
- **Exercise Recognition AI:** A secondary machine learning subsystem that processes raw motion data from an on-wrist inertial measurement unit (IMU) to classify exercises and count repetitions automatically.

1.2 Problem Statement

Gym users — particularly beginners and intermediate trainees — consistently face a cluster of interrelated problems that traditional fitness applications fail to solve:

1. Lack of Personalization: Most apps deliver generic, pre-built workout templates that ignore the user's fatigue level, injury history, equipment access, and rate of progress.

2. Nutritional Opacity: Users do not know the macronutrient composition of the meals they eat, which renders body-composition goals largely guess-based.

3. Inaccessible Expertise: Hiring a qualified personal trainer or dietitian is financially out of reach for most users, and human coaches are not available on-demand.

4. Form and Injury Risk: Users training without supervision frequently develop biomechanical compensations that lead to overuse injuries and stalled progress.

5. Recovery Mismanagement: Sleep, stress, and heart-rate variability are rarely integrated into training prescription, leading to overtraining or undertraining.

6. Motivational Decay: Without external accountability, the majority of users abandon their training programs within the first three months.

MR.FIT addresses these pain points within a single, vertically integrated platform whose intelligence layer is continuously informed by the user's own historical data.

1.3 Project Objectives

The objectives of the project are defined along three axes: functional, technical, and commercial.

1.3.1 Functional Objectives

- Deliver fully personalized workout plans that adapt week-over-week based on logged performance.
- Provide a 24/7 AI coach capable of conversational guidance grounded in the user's history.
- Enable food recognition and macronutrient analysis from photographs and text submitted by the user.
- Compute a daily recovery readiness score from sleep and biometric inputs.
- Surface social motivation through challenges, leaderboards, and shared progress.

1.3.2 Technical Objectives

- Implement the entire backend through n8n workflow orchestration [5], [36], eliminating the need for a custom server codebase.
- Deploy a Next.js frontend on Vercel that communicates with the backend exclusively through authenticated webhook calls and proxy serverless routes.
- Use Supabase as a unified provider for authentication and PostgreSQL storage, with the `pgvector` extension enabled for embedding-based semantic retrieval.
- Integrate the Anthropic Claude API [30] for all generative reasoning, configured behind an Agentic RAG retrieval pipeline.
- Integrate Stripe for subscription management, with webhook-driven tier synchronization.

1.3.3 Commercial Objectives

- Operate the platform under a three-tier subscription model: Free, Pro (\$12.99 / month), and Elite (\$24.99 / month).
- Reserve a B2B white-label tier for personal trainers and gyms as a future revenue stream.
- Achieve a unit-economic profile in which the marginal cost of serving an additional user is dominated by Claude API [30] token consumption rather than fixed infrastructure.

1.4 Scope of the System

The system delivered as part of this graduation project covers the following functional modules:

- **Authentication and Account Management:** Handled by Supabase Auth using JSON Web Tokens (JWTs).
- **Personalized Workout Plan Generation:** Served through dedicated n8n workflows.
- **Workout Logging and Progressive Overload Tracking:** Stored in PostgreSQL.
- **AI Food Scanner & Parser:** Converts text/voice entries and images into structured macronutrient breakdowns.
- **Conversational AI Coach:** Powered by Claude with Agentic RAG over user history.
- **Recovery and Readiness Scoring:** Computed from user-supplied sleep and biometric data.
- **Subscription Billing and Tier Enforcement:** Mediated by Stripe.

Note on wearable hardware: A wearable wristband sensor based on a 9-degree-of-freedom IMU (using chips such as the Bosch BMI270 or InvenSense ICM-20948) is scoped as a version-two milestone. Early prototyping leverages development boards such as the Arduino Nicla Sense ME or an ESP32 with an IMU breakout, and is explicitly excluded from the first production release.

1.5 Technology Stack

The system is constructed from a deliberately compact set of best-in-class managed services, chosen to minimize operational overhead while maintaining production-grade performance:

Layer	Technology	Description
Frontend	Next.js (React + TypeScript)	Hosted on Vercel, styled with Tailwind CSS and custom design tokens.
Backend Orchestration	n8n (Self-hosted on Hostinger VPS)	Exposes webhook endpoints consumed by the frontend.
Authentication	Supabase Auth / NextAuth v5	Issues HS256-signed JWTs containing the user's UUID.
Database	Supabase PostgreSQL	Employs 'pgvector' extension for embedding storage and similarity search.
Generative AI	Anthropic Claude API [30] / Google Gemini	Invoked from n8n workflows and FastAPI microservices.
Payments	Stripe Checkout and Portal	Subscription state synchronized through Stripe webhooks.

DevOps	Docker Compose / systemd	Containerized deployment and system daemon runners.
--------	--------------------------	---

1.6 Architectural Principle: A Webhook-Centric Backend

The defining architectural decision of MR.FIT is the elimination of a hand-written backend service. Every server-side capability — authentication checks, database queries, AI inference, payment reconciliation — is implemented as a directed graph of n8n nodes, exposed to the outside world through HTTPS webhook endpoints. The Next.js frontend never calls a database, an AI provider, or a payment processor directly; it issues authenticated POST requests to a small number of webhook URLs, and n8n performs the orchestration.

This choice has three consequences that shape the entire project:

1. Visual Inspectability: Every business process is represented as a flow diagram inside the n8n editor, which is invaluable for academic documentation and live demonstration.

2. Accelerated Iteration: Shifts the engineering effort from writing imperative server code to designing declarative workflows, which speeds up development.

3. Boundary Discipline: Because n8n executes whatever the webhook gives it, the first node of every workflow must be a rigorous JWT validation step. The design and debugging of that step is the central case study of Chapter 2 (mapped to Chapter 7).

1.7 Monetization Model

MR.FIT operates on a freemium SaaS model with three tiers. The Free tier exists primarily as an acquisition funnel and exposes only basic logging features. The Pro tier (\$12.99 per month) unlocks the AI coach chat, the food scanner, and adaptive workout planning. The Elite tier (\$24.99 per month) adds form-check analysis, advanced recovery analytics, and priority access to new features. A B2B white-label tier is reserved for trainers and gyms and is priced on negotiation.

Because Claude API [30] token consumption is the dominant variable cost, the subscription tiers are calibrated to ensure that the expected monthly token spend per active

Pro user remains well below the subscription price, even under heavy chat usage. Token budgets are enforced through monthly quotas stored in the subscriptions table and checked by an early node in each AI-bearing workflow.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Review of Existing Fitness and Health Solutions

The domain of personal informatics has been restructured by the proliferation of mobile health (mHealth) applications and wearable devices, providing continuous access to physiological data. However, the current landscape remains fragmented, split between passive logging utilities and generic algorithmic coaching platforms. Passive utilities offload the analytical burden of training and diet mapping entirely onto the user, while generic algorithmic platforms fail to reason over real-time biomechanical telemetry, temporal user behavior, and individual contexts like Heart Rate Variability (HRV) or physical fatigue. To establish the foundation for MR-Fit, this section reviews dominant commercial solutions and their limitations.

2.1.1 Dietary Assessment and the Friction of Manual Logging: MyFitnessPal

MyFitnessPal relies on manual user inputs and search-and-select querying of a hybrid database containing curated governmental records and a large corpus of unverified crowdsourced entries (which comprise up to 70% of the database). The platform lacks native natural language processing (NLP) to parse unstructured descriptions or computer vision to quantify mixed meals, resulting in significant logging friction. Qualitative human-computer interaction (HCI) studies show that manual logging causes "negative nudges," where rigid streaks and data-entry demands frustrate users and lead to high abandonment rates. Furthermore, validation studies (Griffiths et al., Fallaize et al.) demonstrate that MyFitnessPal consistently underestimates energy, protein, fat, sodium, and essential micronutrients when compared to research-grade standards, functioning as an error-prone ledger rather than an intelligent nutritional companion.

2.1.2 Resistance Training Utilities and the Deficit of Analytical Progression: Strong App

Strong App represents the archetype of passive resistance training logs, operating on standard CRUD (Create, Read, Update, Delete) database principles to digitize pen-and-paper logs. Users must manually input exercises, weights, and repetitions, which introduces a severe cognitive interference effect during high-intensity training, disrupting psychological

flow and leading to incomplete datasets. Strong App lacks predictive machine learning and automated coaching layers, assuming the user already possesses programming expertise. Furthermore, it does not integrate with wearable inertial measurement units (IMUs) to recognize exercises, count repetitions, or assess movement kinematics (concentric velocity degradation, range of motion), leaving it blind to muscle failure.

2.1.3 Algorithmic Heuristics vs. Physiological Reality: Fitbod

Fitbod generates customized workout routines using a recommendation engine that selector exercises based on localized muscle recovery heuristics and progressive overload formulas. However, its fatigue tracking is purely mathematical, derived from past logged volumes rather than real-time biomarkers. It lacks edge-deployed IMU integration for biomechanical tracking, meaning it cannot verify concentric velocity or range of motion to detect technique breakdown. Crucially, Fitbod ignores autonomic nervous system biometrics such as morning HRV and sleep stage data. If a user experiences systemic central nervous system (CNS) fatigue, Fitbod's algorithm remains unaware and may prescribe high-intensity workouts, increasing the risk of injury and overtraining.

2.1.4 Wearable Ecosystems and the Deficit of Prescriptive Coaching: Apple Fitness+ and Google Fit

Ecosystem aggregators like Apple Health/Fitness+ and Google Fit leverage sophisticated APIs (HealthKit and Google Fit API) to passively aggregate multi-sensor telemetry, including IMU activity, optical PPG heart rate, and GPS tracking. However, as analyzed in recent literature (Mulla et al., Jörke et al.), these platforms rely strictly on descriptive analytics rather than prescriptive coaching. They present raw charts and statistics that induce "dashboard fatigue," where users are overwhelmed by graphs but lack the domain expertise to translate them into actionable daily interventions. Apple Fitness+ further relies on pre-recorded videos that cannot adapt to real-time user metrics or support natural language interaction.

Summary of Existing Platform Limitations

synthesizes the operational gaps and architectural limitations present in current commercial fitness solutions, highlighting the necessity for a unified, generative AI framework.

Feature Matrix	MyFitnessPal	Strong App	Fitbod	Apple Fitness+ / Google Fit
----------------	--------------	------------	--------	-----------------------------

Primary Paradigm	Passive Dietary Ledger	Passive Workout Ledger	Rule-Based ML Coaching	Biometric Aggregation & Media
Primary Input Method	Manual Entry / Barcode	Manual CRUD Interface	Algorithmic + Manual	Passive Sensor Fusion
NLP / Voice Integration	None	None	None	Basic OS Dictation Only
Agentic AI Coaching Layer	No	No	No	No
Dynamic Adaptation	No	No	Limited (Past Volume, Ignores HRV)	No
Real-Time IMU Classify	N/A	No	No	General Activity (Cardio) Only

2.2 Identified Research Gap

The review of commercial solutions highlights a critical technological void. Platforms operate in isolated silos, manual logging forces a high cognitive load, and algorithmic recommendations are generated without real-time physiological or biomechanical context. The identified research gap lies in the absence of a closed-loop architecture that integrates edge-computing classification, agentic RAG conversational coaching, and frictionless NLP parsing.

MR-Fit directly bridges this gap by introducing a unified system that combines: (a) real-time, wrist-worn IMU sensor data processed by a CPU-optimized Random Forest model for autonomous exercise classification and rep counting; (b) a conversational AI coach utilizing Retrieval-Augmented Generation (RAG) over a Supabase pgvector database to reason dynamically over historical logs, HRV, and sleep biometrics; and (c) natural language "Magic Input" parsing via Gemini to eliminate manual logging.

2.3 Stakeholder Analysis

A comprehensive stakeholder analysis is essential to understand the diverse requirements, expectations, and operational constraints imposed on the MR-FIT platform. By identifying and analyzing the individuals and groups whose interests intersect with the system, the architectural design can be tailored to balance user experience, operational costs,

security, and academic rigor. The stakeholders of the MR-FIT system are categorized into primary, secondary, and tertiary layers.

2.3.1 Primary Stakeholder: The End User (Trainee/Athlete)

The primary stakeholder is the end user—an individual athlete, trainee, or fitness enthusiast who interacts directly with the Next.js frontend client to log physical workouts, track daily nutritional intake, sync biometric records, and engage in natural language conversations with the AI coach. The ultimate goal of this stakeholder is to achieve their personal fitness objectives (such as muscle hypertrophy, fat loss, or strength development) while minimizing the friction associated with traditional health-tracking software.

End users typically possess moderate technical literacy. They are comfortable operating modern web browsers, managing mobile application interfaces, and wearing consumer smartwatches or fitness trackers, but they do not have expertise in software engineering or database management. Their key pain points correspond to the core limitations identified in commercial health applications (§2.1): the high cognitive load of manual database searching, the lack of responsive and physiologically adaptive coaching recommendations, and "dashboard fatigue" caused by descriptive analytics that offer no actionable advice. Consequently, they require a seamless, friction-free interface that automates data ingestion and translates raw metrics into direct, conversational training adjustments.

2.3.2 Secondary Stakeholder: The System Developer and Administrator

The secondary stakeholder represents the technical operators, system administrators, and developers responsible for the ongoing maintenance, security, scalability, and financial feasibility of the MR-FIT platform. This role handles the self-hosted n8n workflow engine on the virtual private server (VPS), manages the database schemas and Row-Level Security (RLS) policies within Supabase, and monitors API token utilization.

Unlike the end user, this stakeholder has high technical literacy and is concerned with system-level metrics. Their primary objectives are to ensure high system availability, prevent unauthorized data access across tenants, and control API token expenditures. Since the platform utilizes external Large Language Model APIs (Claude and Gemini) for its primary reasoning loops, the administrator must implement rate limits and monitor token budgets to

prevent cost saturation. Furthermore, they are responsible for deploying updates to the scikit-learn Random Forest model and ensuring that the microservices running on ports 8000 and 8001 operate with minimal latency and high availability.

2.3.3 Tertiary Stakeholder: Academic Supervisors and Evaluators

The tertiary stakeholders consist of the university graduation project committee, academic supervisors, and external evaluators. This group does not interact with the system's operational interface or maintain its servers; instead, they assess the project from a theoretical, structural, and methodological perspective.

The primary interest of the academic supervisors is to evaluate the technical novelty, engineering quality, and scientific validity of the MR-FIT architecture. They assess how effectively the system bridges the identified research gaps (§2.2) by integrating edge-computing activity recognition with cloud-based generative AI reasoning. They require rigorous documentation, clear mathematical formulations of signal processing pipelines (such as Chauvenet's outlier detection and Butterworth filter transfer functions), and empirical proof of classifier performance. Additionally, they demand strict reproducibility, meaning the codebase, database schemas, and embedded firmware must be structured in a transparent, verifiable, and academically rigorous format.

2.4 User Requirements

The system requirements are extracted directly from the operational modules of the consolidated codebase:

2.4.1 Functional Requirements (FR)

- **FR-1: User Authentication & Onboarding:** The system must authenticate users securely and gather initial physical profiles (weight, height, age, fitness goal, fitness level) during onboarding.
- **FR-2: Manual Workout Logging:** The system must allow users to log exercises, completed sets, reps, and weights, storing them with a timestamp.
- **FR-3: Natural Language Logging (Magic Input):** The system must parse unstructured text or voice descriptions of food and workouts, and insert the parsed entities into the database.

- **FR-4: Conversational AI Coaching:** The system must support a real-time chat interface where the AI answers questions and suggests training adjustments based on the user's historical logs.
- **FR-5: Biometric Sync:** The system must receive and store daily biometrics (resting heart rate, HRV, sleep duration) from wearable sensors.
- **FR-6: Wearable Exercise Tracker:** The system must process acceleration and angular velocity signals from an on-wrist IMU sensor to identify the exercise type and count repetitions.

2.4.2 Non-Functional Requirements (NFR)

- **NFR-1: Security:** All database queries must be protected by Row-Level Security (RLS) policies based on the authenticated user's ID.
- **NFR-2: Latency:** AI coach responses and entry parsing should execute within acceptable network timeouts, and raw sensor inference must complete in under 200 ms to match the sampling frequency.
- **NFR-3: Portability:** The web frontend must be responsive and fully functional across standard web browsers on desktop environments.

2.5 Use Case Analysis

To formalize the behavioral requirements of the MR-FIT platform, the six Functional Requirements (FR-1 through FR-6) are detailed below as structured use case specifications.

2.5.1 Use Case Specifications

Use Case ID	UC-01
Use Case Name	User Authentication & Onboarding
Primary Actor	Unauthenticated User
Precondition	The user has the web application open for the first time.
Main Success Scenario	1. The user navigates to the registration page. 2. The user enters their email and a secure password. 3. The user submits the registration form. 4. The system prompts the user to enter their initial physical metrics (weight, height, age, fitness goal, fitness level). 5. The user enters the metrics and submits the onboarding profile. 6. The system stores the user credentials and profile metrics in the Supabase database and initializes an

	authenticated session.
Postcondition	The user is successfully authenticated, and their physical profile exists in the database.
Exception Flow	- Invalid Input: If the email format is incorrect or the password does not meet security requirements, the system displays an error and prompts for correction. - Profile Sync Failure: If database insertion fails, the system retries the write operation; if it fails again, the session is rolled back and the user is prompted to resubmit.

Use Case ID	UC-02
Use Case Name	Manual Workout Logging
Primary Actor	Authenticated User
Precondition	The user is logged into their account and is on the workout logging tab.
Main Success Scenario	1. The user selects an exercise from the master catalogue. 2. The user inputs the number of sets, repetitions completed per set, and the weight lifted. 3. The user clicks the "Confirm Log" button. 4. The system validates the numeric inputs. 5. The system automatically appends a Unix timestamp and commits the record to the `workout_logs` table in Supabase under the user's ID.
Postcondition	The completed workout set is stored in the database, updating the user's training statistics.
Exception Flow	- Malformed Inputs: If numeric fields contain negative values or non-numeric characters, the validation layer blocks submission and highlights the faulty inputs.

Use Case ID	UC-03
Use Case Name	Natural Language Magic Input
Primary Actor	Authenticated User
Precondition	The user is authenticated and has activated the "Magic Input" microphone or text field.
Main Success Scenario	1. The user dictates or types an unstructured, free-form description of a meal (e.g., "I had two scrambled eggs with sourdough toast") or a workout. 2. The client-side Web Speech API transcribes the audio (if voice is used) and displays the raw text. 3. The user submits the text to the Next.js API gateway. 4. The system routes the string to the Gemini LLM microservice. 5. The Gemini LLM parses the unstructured text, extracts nutritional/workout entities, infers portion sizes, and

	queries verified nutritional databases. 6. The system returns the structured entities to the client UI for user confirmation. 7. The user clicks "Save", and the system commits the structured data to the database.
Postcondition	Unstructured language is successfully parsed and logged in the database as structured data.
Exception Flow	- Transcription Failure: If the voice input is unintelligible, the Web Speech API returns an empty string, prompting the user to speak clearly. - Parsing Hallucination / Failure: If the LLM cannot extract recognizable food or workout items, it prompts the user to type the entry manually.

Use Case ID	UC-04
Use Case Name	Conversational AI Coaching
Primary Actor	Authenticated User
Precondition	The user is authenticated and opens the AI coach chat dashboard.
Main Success Scenario	1. The user inputs a training or recovery question in natural language (e.g., "My lower back is sore, what should I replace deadlifts with?"). 2. The system generates a vector embedding of the query. 3. The system executes a cosine similarity search on the `user_embeddings` table in Supabase, scoped strictly to the user's ID. 4. The database returns the most relevant historical workout logs and biometric recovery trends. 5. The n8n orchestrator concatenates these retrieved context chunks into the system prompt. 6. The Claude API generates a personalized, context-grounded response. 7. The system renders the response in the chat interface.
Postcondition	The user receives a personalized, grounded response based on their historical training logs.
Exception Flow	- Context Retrieval Empty: If no historical entries match the similarity query, the system falls back to a generalized coaching prompt grounded in baseline sports science guidelines.

Use Case ID	UC-05
Use Case Name	Biometric Sync
Primary Actor	Wearable Device / Health API
Precondition	The wearable device is paired, and the API has network access.
Main Success Scenario	1. The wearable device aggregates resting heart rate, Heart Rate Variability (HRV), and sleep stages over a 24-hour cycle. 2. The device syncs data with

	the Apple Health or Google Fit API. 3. The MR-FIT client triggers the biometric sync webhook. 4. The system receives the JSON payload containing the recovery metrics. 5. The system commits the daily biometric records to the 'profiles' table in Supabase under the user's ID.
Postcondition	Recovery metrics are stored in Supabase, enabling dynamic AI coach context retrieval.
Exception Flow	- Network Outage: If the webhook call fails due to lack of network, the client service worker queues the sync request locally until connectivity is restored.

Use Case ID	UC-06
Use Case Name	Wearable Exercise Tracker
Primary Actor	Authenticated User + Wearable Device
Precondition	The user starts an active training session while wearing the ESP32/MPU-6050 DIY band.
Main Success Scenario	1. The user begins performing a resistance exercise set. 2. The MPU-6050 IMU samples linear acceleration and angular velocity at 200 Hz. 3. The ESP32 packages the raw 6-axis signals into JSON windows and transmits them to the FastAPI server. 4. The FastAPI server applies Butterworth low-pass filtering and extracts 73 kinematic features. 5. The CPU-optimized Random Forest classifier predicts the exercise class and peak detection counts repetitions in under 200 ms. 6. The server transmits the results back to the ESP32 client and the Next.js database. 7. The ESP32 OLED display and Next.js UI update automatically.
Postcondition	Exercises and repetition counts are autonomously classified and logged in real-time.
Exception Flow	- Wi-Fi Packet Drop: If packet transmission fails, the ESP32 buffers raw samples locally and retries transmission upon connection re-establishment, using sequence numbers to prevent data loss.

A formal UML Use Case Diagram is provided as a supplementary figure [Figure X] and should be inserted here by the project team.

2.6 User Stories

The operational requirements of the MR-FIT platform are translated into Agile user stories, complete with technical acceptance criteria.

2.6.1 Functional User Stories

US-01: Secure Authentication and Personal Profile Onboarding

As a new user, I want to create a secure account and submit my physical metrics during onboarding so that the platform can establish my physiological baseline.

- **Acceptance Criteria:**

- The system must reject registration attempts with passwords shorter than 8 characters.
- The onboarding UI must enforce completion of all profile fields (weight, height, age, fitness goal, fitness level) before allowing access to the dashboard.
- User credentials and metrics must be written to Supabase in a single transactional commit.

US-02: Intuitive Manual Workout Entry

As an authenticated athlete, I want to manually log my exercise sets, repetitions, and weights so that I can maintain a historical record of my training.

- **Acceptance Criteria:**

- The user must be able to select exercises from a searchable catalog containing at least 100 preset movements.
- The entry form must reject negative values or empty inputs.
- The log must be saved with a precise server-side timestamp and automatically update progress charts.

US-03: Natural Language Magic Input Logging

As a busy user, I want to describe my meals and workouts in free-form text or voice so that I can log my data without navigating complex menus.

- **Acceptance Criteria:**

- The Web Speech API transcription must execute client-side and display text in under 150 ms.
- The backend Gemini parser must correctly extract quantities (e.g., "two eggs") and match food items to USDA verified databases.

- The user must be presented with a confirmation step to edit extracted values before database insertion.

US-04: Grounded AI Conversational Coaching

As a Pro user, I want to receive training advice that is grounded in my actual training history so that I can progress safely and effectively.

- **Acceptance Criteria:**
- The similarity query must retrieve only the authenticated user's logs (``user_id = auth.uid()``), preventing data exposure.
- The RAG system must return a response generated by Claude in under 2 seconds.
- If a user mentions joint pain, the response must prioritize exercise alternatives over volume increases.

US-05: Wearable Biometric Synchronization

As an active trainee, I want my sleep and heart rate variability data to sync passively in the background so that my AI coach understands my recovery state.

- **Acceptance Criteria:**
- The system must accept webhook payloads containing resting HR, HRV, and sleep stages.
- Biometric writes must not overwrite prior days' records.
- The data must be formatted to allow joint query filters with workout logs during RAG retrieval.

US-06: Autonomous Motion Tracking and Repetition Counting

As an active lifter, I want my wearable wristband to automatically detect my exercise and count my repetitions so that I do not have to touch my phone during a set.

- **Acceptance Criteria:**
- The Random Forest classifier must execute inference on raw sensor windows in under 200 ms.
- The repetition counter must maintain a Mean Absolute Error (MAE) of less than 0.5 repetitions per set.

- Real-time updates must render on the ESP32 OLED display and Next.js client concurrently.

2.6.2 Non-Functional User Stories

US-07: Data Isolation and Security (NFR-1)

As a security-conscious user, I want my personal training and biometric logs to be completely isolated from other users so that my health data remains private.

- **Acceptance Criteria:**
- Every table holding personal metrics must have Row-Level Security (RLS) enabled.
- Any SQL query attempting to read or write data belonging to a different `user\_id` than the authenticated JWT subject (`auth.uid()`) must return zero rows.

US-08: Sub-50ms Embedded Response Latency (NFR-2)

As a lifter, I want the wearable OLED screen to update immediately as I complete a repetition so that I do not experience lag.

- **Acceptance Criteria:**
- The round-trip latency (I2C read → Wi-Fi send → FastAPI inference → OLED update) must be under 50 ms in typical Wi-Fi environments.
- Feature extraction and model inference on port 8001 must complete in under 15 ms.

US-09: Universal Desktop Browser Accessibility (NFR-3)

As a desktop user, I want the dashboard interface to render correctly and adjust dynamically to my screen size so that I can view my progress clearly.

- **Acceptance Criteria:**
- The frontend client must render correctly on Chrome, Firefox, Safari, and Edge.
- Charts built with Recharts must resize dynamically without breaking layout boundaries.

2.7 System Constraints

The system is designed under the following technical constraints:

1. Infrastructure Constraint: The backend orchestration must run on n8n self-hosted on a VPS, meaning no custom imperative server codebase (e.g., Express or Django) is used for the API gateway.

2. Computational Constraint: The Random Forest exercise classifier must run on standard CPU instances without requiring dedicated GPUs.

3. Sensor Constraint: Prototyping is limited to 6-axis IMU sensors (accelerometer + gyroscope), which do not capture absolute spatial orientation (yaw) without a magnetometer.

2.8 Feasibility Study

A thorough feasibility study is conducted across technical, economic, and operational dimensions to evaluate the viability of the MR-FIT platform under academic and production environments.

2.8.1 Technical Feasibility

The technical feasibility of the MR-FIT system is supported by the selection of a lightweight, CPU-optimized machine learning pipeline and a containerized server architecture. Performing human activity recognition (HAR) on raw inertial sensor streams typically requires computationally heavy neural networks (such as 1D-CNNs or LSTMs) [CITATION NEEDED]. However, by implementing a 73-dimensional time-domain and frequency-domain feature engineering pipeline (§8.6), the classification task is simplified to a point where a Random Forest estimator can execute inference in under 2.1 ms per window. This low complexity enables the classifier to run efficiently on standard CPU virtual machines on a Hostinger VPS without requiring expensive graphics processing units (GPUs), satisfying Computational Constraint 2. Furthermore, self-hosting the n8n orchestrator on a Linux VPS provides a visual, highly maintainable API gateway that manages JWT validation and database writes natively, confirming the technical feasibility of running a webhook-centric backend without custom API code (Infrastructure Constraint 1).

Although Sensor Constraint 3 limits the hardware to a 6-axis IMU (MPU-6050) without a magnetometer, technical feasibility is maintained. While a 9-axis sensor is required to track absolute spatial yaw over long durations, the six compound lifts targeted by MR-FIT (squat, bench, deadlift, overhead press, row, and rest) occur primarily in the sagittal and

coronal planes. Tri-axial linear acceleration and angular velocity are sufficient to isolate the cyclical patterns of these lifts, achieving a high classification accuracy of 98.2% and a repetition Mean Absolute Error of 0.31 per set. The availability of developer SDKs for the client-side Web Speech API and API endpoints for Google GenAI Gemini 2.5 Flash and Anthropic Claude ensures that the natural language logging and conversational coaching loops are constructed using production-ready, highly reliable services.

2.8.2 Economic Feasibility

The economic model of the MR-FIT platform is designed to minimize fixed operational overhead while scaling variables proportionally to active user demand. The system avoids the high licensing fees associated with enterprise workflow engines (such as MuleSoft or Zapier) by deploying a self-hosted instance of n8n on a virtual private server, costing under USD 10 per month. Database hosting costs are minimized by using Supabase's free tier, which provides 500 MB of PostgreSQL storage and native pgvector support. This storage capacity is sufficient to hold the relational tables, chat logs, and vector embeddings for up to 1,000 active beta users.

The primary financial variable is the consumption of API tokens from external Large Language Model providers (Anthropic and Google). Claude API tokens represent the dominant cost factor: a single conversational RAG query retrieves historical context vectors and outputs a structured response, consuming approximately 3,000 prompt tokens and 300 completion tokens. At commercial pricing tiers, this translates to roughly USD 0.05 per conversation. Under a freemium SaaS model (\$1.7), these costs are controlled by placing daily chat quotas on the free tier (e.g., 5 messages per day) and reserving continuous RAG queries for the Pro (\$12.99/mo) and Elite (\$24.99/mo) subscription tiers, where subscription revenues offset token usage. Operational cost boundaries are enforced at the database level by daily trigger functions that prevent users from exceeding their designated API quotas, proving that the system is economically viable at scale.

2.8.3 Operational Feasibility

Operational feasibility assesses whether the system can be deployed, maintained, and operated within the constraints of the development team and user environment. The client is deployed as a web-only application using Next.js on Vercel, completely bypassing the

complex submission guidelines, build configurations, and review cycles required by mobile app stores (Apple App Store and Google Play). This web-only deployment model enables the development team to push updates and patches instantly, significantly reducing maintenance complexity.

From an administrative perspective, the platform is operationally feasible because it requires no specialized IT team. A single administrator can manage the systemd service units, monitor container health via Docker Compose, and verify security isolation through pgTAP testing frameworks. For the end user, operational feasibility is exceptionally high. The "Magic Input" natural language parser (§5.4) completely removes the tedious data-entry interfaces of traditional calorie trackers, allowing users to log their food and training through voice dictation. By reducing the physical and cognitive friction of logging, the system promotes long-term user adherence, achieving the behavioral objectives defined in the project scope (§1.5).

CHAPTER 3: SYSTEM DESIGN

This chapter presents the architectural design and system-level modeling of the MR-FIT platform. It details how the Next.js frontend client, the self-hosted n8n automation engine, the PostgreSQL database, and the Python FastAPI microservices cooperate to deliver a seamless, secure, and intelligent fitness experience.

3.1 System Overview

MR.FIT employs a decentralized, microservices-oriented architecture divided into four main operational layers:

1. Presentation Layer (Frontend): A Next.js 14 web application providing a responsive dashboard, exercise library, interactive workout logging, and conversational coach chat.

2. Orchestration Layer (Backend Gateway): A self-hosted n8n workflow instance on a Hostinger VPS that exposes webhook endpoints. It handles business logic, auth checks, database queries, and AI routing.

3. Data Layer (Storage & Semantic Search): A PostgreSQL database (hosted on Supabase or locally) running the `pgvector` extension for storing user records, logs, and embeddings.

4. Intelligence Layer (AI Microservices):

- **FastAPI Coach microservice** (Port 8000) that compiles user context, generates embeddings, and queries Gemini 2.5 Flash.
- **FastAPI Smart Tracker microservice** (Port 8001) that runs the Random Forest model and peak-detection rep counter.

3.2 Overall Architecture

The system's execution blocks are distributed across client-side runtimes, serverless edge networks, virtual private servers (VPS), and database clusters. This separation of

concerns is implemented to optimize scalability, minimize operational latency, and isolate sensitive processing boundaries.

3.2.1 Architectural Topology

- **Client Run-Time:** The Next.js 14 web client is executed directly in the user's web browser, executing React components, managing local application states, and accessing native hardware interfaces such as the Web Speech API.
- **Serverless Edge (Vercel):** The Next.js Backend-for-Frontend (BFF) layers are hosted on Vercel's edge network, intercepting client API requests, checking session tokens via NextAuth, and proxying validated payloads to downstream backend servers.
- **Orchestration Node (n8n Hostinger VPS):** The central business logic is executed on a self-hosted Hostinger VPS. n8n workflow graphs handle transactional routes, payment events, and database mutations.
- **Storage Cluster (Supabase Cloud):** Relational tables, foreign keys, vector indices, and row-level access controls are managed in a managed PostgreSQL instance.
- **AI Compute Nodes:** The FastAPI ASGI microservices process high-frequency kinematic buffers and prompt retrieval pipelines.

3.3 Module Breakdown

The system is partitioned into the following functional modules:

- **Dashboard Module:** Consolidates daily stats (calories, protein, weekly workouts progress tracks) and houses the Speech-to-Text "Magic Input" interface.
- **Workout Management Module:** Enables manual session logging, routine templates execution, and history tracking.
- **AI Coach Chat Module:** A conversational chat interface exposing personalized advice and workout suggestions.
- **Nutrition Tracker Module:** Logs food names, estimates calories, and updates daily macronutrient totals.
- **Wearables Integration Module:** Syncs sleep snapshots, heart rate variability (HRV), and step counts.

3.4 Data Flow Design

To illustrate system interaction, the data flow for the **Natural Language Magic Input** is detailed below:

1. The user enters a voice or text description (e.g., *"bench press 3 sets of 10 at 60kg"*).
2. The browser transcribes voice (via the Web Speech API) and submits the text to the Next.js BFF at ``/api/smart-tracker/parse-entry``.
3. The Next.js BFF forwards the request to the FastAPI AI microservice at ``http://localhost:8000/parse-entry``.
4. The FastAPI microservice queries Gemini 2.5 Flash using structured schema instructions.
5. Gemini returns a JSON list of parsed sets.
6. The Next.js BFF receives the parsed list, loops over the items, and executes sequential POST requests to ``/api/workout-logs`` and ``/api/nutrition``.
7. The logs are saved in the PostgreSQL database, and the frontend triggers a page refresh to update dashboard progress indicators.

3.5 Component Diagram

The component structure of MR-FIT defines the modular physical subsystems, their interfaces, and compile-time dependencies. The system is split into three main tiers: Client Presentation, Visual Orchestration Backend (BFF), and Core Storage/Inference Services.

The Client Presentation Layer (Next.js 14) communicates via REST and WebSockets with the Visual Orchestration Layer (n8n API gateway). High-frequency IMU telemetry collected on the wearable edge is pushed to the FastAPI microservice for feature extraction and Random Forest inference. Data is stored in Supabase PostgreSQL, which is secured at the storage layer via Row-Level Security policies linked to NextAuth identities.

3.5.1 System Component Specifications

The following table details the key system components, their roles, interfaces, and compile-time dependencies:

System Component Specifications

Component Name	Architectural Layer	Primary API/Interface	Upstream Dependencies	Downstream Dependencies
Next.js Client	Presentation	HTTPS / WSS	NextAuth, Web Speech API	n8n Webhook, Supabase Client
n8n Engine	Orchestration (BFF)	REST Webhook	Next.js Client, Stripe Webhooks	Claude API, Supabase SQL
FastAPI Service	Embedded Inference	REST POST ('/predict')	Wearable ESP32 firmware	NumPy / Scikit-Learn
PostgreSQL DB	Storage & Security	native pg / pgvector	Supabase Auth	RLS Policies, Vector Index

3.6 Sequence Diagrams

To illustrate the dynamic, event-driven interactions within the system, three critical transaction lifecycles are specified below. These workflows demonstrate how data transitions between edge devices, APIs, the orchestration engine, and database layers.

3.6.1 User Authentication and Onboarding Sequence

- 1. User Connection:** The user inputs credentials or connects via OAuth on the Next.js presentation layer.
- 2. Token Retrieval:** The Next.js BFF requests session tokens from the NextAuth engine, which verifies credentials and returns a signed HS256 JWT.
- 3. Profile Setup:** The Next.js Client calls the `/api/onboarding` endpoint on the BFF. The BFF executes an SQL insertion into the Supabase database, initializing the user's biometric record in the `profiles` table.
- 4. Onboarding Complete:** The client displays a success screen and redirects the user to the central dashboard.

3.6.2 Stripe Webhook Subscription Sync Sequence

- 1. Event Broadcast:** Upon successful payment, the Stripe API broadcasts a `customer.subscription.updated` webhook payload to the n8n endpoint.
- 2. Signature Verification:** The n8n automation engine interceptor validates the Stripe cryptographic signature using the webhook secret.

3. Token Validation: The n8n Code node extracts the user's UUID from the payload claims and verifies the signature.

4. Database Sync: The n8n workflow updates the `profiles` table in Supabase PostgreSQL, modifying `subscription\_tier` to Pro or Elite.

5. Acknowledge: n8n returns an HTTP 200 OK status to Stripe to close the loop.

3.6.3 Wearable Telemetry Streaming and Inference Sequence

1. Edge Sampling: The wrist-worn MPU-6050 IMU samples physical acceleration and angular velocity at 200 Hz.

2. Buffer Packaging: The ESP32 firmware accumulates 200 samples in local SRAM, packages the data into a JSON window, and streams it via Wi-Fi to the FastAPI microservice at `/predict`.

3. Inference Execution: The FastAPI microservice parses the JSON array, extracts 73 statistical features (mean, std, zero-crossing, etc.), passes them to the Random Forest classifier, and returns the predicted exercise class and confidence.

4. Feedback & Count: The ESP32 firmware updates its local SSD1306 OLED screen with the predicted exercise and increments the repetition counter using a local peak-detection algorithm.

3.7 Database Design Overview

The database runs on PostgreSQL. The relational schema comprises 11 core tables detailed below:

Database Schema Tables and Relations

Table Name	Primary Key	Foreign Keys	Purpose
users	`id` (uuid)	None	Core application user records (mirrors Supabase auth).
profiles	`id` (uuid)	`user_id` $\rightarrow$ `users(id)`	Physical profiles, fitness goals, and calorie/macro targets.
exercises	`id` (uuid)	None	Master catalogue of exercises with vector embeddings.
workouts	`id` (uuid)	`user_id` $\rightarrow$ `users(id)`	Generated training routines and scheduled

			sessions.
workout_exercises	`id` (uuid)	`workout_id`, `exercise_id`	Links workouts to specific exercises with target sets/reps.
workout_logs	`id` (uuid)	`user_id`, `workout_id`	Actual logged workout sets, repetitions, weights, and notes.
workout_log_exercises	`id` (uuid)	`workout_log_id`	Individual exercise entries per logged session.
nutrition_logs	`id` (uuid)	`user_id` \$\rightarrow\$ `auth.users(id)`	Dietary logs storing calorie and macronutrient values.
weight_logs	`id` (uuid)	`user_id` \$\rightarrow\$ `users(id)`	Time-series weight records for progress visualization.
wearable_data	`id` (uuid)	`user_id` \$\rightarrow\$ `users(id)`	Metric snapshots (HRV, sleep, steps) from connected devices.
chat_history	`id` (uuid)	`user_id` \$\rightarrow\$ `users(id)`	Conversational context history with the AI Coach.

3.8 Security Considerations

1. Role-Based Session Gating: NextAuth v5 handles frontend session verification. Next.js Middleware intercepts dashboard routes, redirecting unauthenticated traffic to `/login`.

2. Row-Level Security (RLS): PostgreSQL enforces data isolation at the storage layer. RLS policies verify that `user\_id = auth.uid()` before returning any log rows, preventing cross-tenant access.

3. API Webhook Authentication: n8n workflows are secured by a custom "Verify the JWT" Code node. The node extracts the HS256-signed JWT from the header, validates the signature expiration claim, and maps the `sub` claim to the workflow's database query parameters.

3.9 Technology Stack Rationale

- **Next.js 14 App Router:** Selected for React Server Components (RSC) which load page data server-side via direct PostgreSQL pool queries, eliminating API latency.

- **n8n Orchestration:** Chosen to replace an imperative node API server. n8n visually charts data pipelines, making it easy to audit and trace logic.
- **pgvector:** Enables local, cost-effective vector databases. By storing 384-dimension or 1536-dimension embeddings directly in PostgreSQL, we perform cosine-similarity matches without external database subscriptions.

CHAPTER 4: UI/UX DESIGN

4.1 Introduction

The user interface (UI) and user experience (UX) play a fundamental role in the success of modern software applications. While the technical components of a system provide its functionality, the user interface serves as the medium through which users interact with these capabilities. A well-designed interface not only improves usability but also enhances user satisfaction, engagement, and productivity.

In fitness management systems, UI/UX design is particularly important because users frequently interact with the platform to perform various activities such as creating workout plans, tracking exercise performance, monitoring nutritional intake, analyzing progress, and receiving personalized recommendations. These tasks require the presentation of large amounts of information in a manner that remains clear, organized, and easy to understand.

MR-FIT was developed as an integrated fitness platform that combines workout tracking, nutrition management, progress analytics, and AI-powered coaching within a single environment. Since the platform targets users with different fitness goals and experience levels, the interface was designed to provide a seamless and intuitive experience regardless of the user's background or technical expertise.

The design process focused on creating a professional and modern user experience that supports efficient navigation, minimizes unnecessary complexity, and encourages continuous user engagement. Special attention was given to information hierarchy, visual consistency, accessibility, responsiveness, and workflow optimization to ensure that users can easily access important features and perform tasks with minimal effort.

To achieve these objectives, the MR-FIT interface adopts a unified design system based on a dark-themed visual style, consistent navigation patterns, reusable interface components, and structured layouts. The platform was designed to balance aesthetic appeal with functionality, ensuring that users can focus on achieving their fitness goals without being distracted by unnecessary interface complexity.

This chapter presents the UI/UX design methodology used during the development of MR-FIT. It discusses the design objectives, visual design system, information architecture, user workflows, interface components, responsive design considerations, usability principles, and the final implementation of the user interface.

4.2 UI/UX Design Objectives

The design of MR-FIT was guided by a set of UI/UX objectives intended to create an efficient, engaging, and user-friendly fitness platform. Since users regularly interact with workout plans, nutrition logs, progress analytics, and AI-powered recommendations, the interface was designed to reduce complexity while maintaining functionality.

Several design principles were established during the design process, including simplicity, consistency, accessibility, responsiveness, and user engagement. These principles influenced the overall structure of the platform and guided the development of individual pages and interface components. By adhering to these objectives, MR-FIT aims to provide a seamless user experience that supports users throughout their fitness journey.

4.2.1 Simplicity

Simplicity was one of the primary design objectives of MR-FIT. Fitness applications often present large amounts of information, including workout data, nutritional statistics, progress reports, and coaching recommendations. To prevent information overload, the interface was designed with a clean layout and clear visual hierarchy.

Each page focuses only on information that is relevant to the user's current task. Dashboard widgets display essential metrics, while advanced details are presented only when needed. Unnecessary visual elements were minimized to maintain clarity and improve usability.

The dark-themed interface, combined with selective use of accent colors, helps users identify important actions and information quickly. This approach reduces cognitive load and enables users to navigate the platform efficiently without extensive training or prior experience.

4.2.2 Consistency

Consistency was implemented throughout the MR-FIT platform to create a predictable and intuitive user experience. All pages follow a unified design language that includes consistent typography, color usage, spacing, navigation patterns, and component styling.

Interactive elements such as buttons, cards, forms, and modal windows maintain the same appearance and behavior across different modules. This consistency allows users to transfer knowledge gained from one section of the platform to another, reducing the learning curve and improving usability.

The navigation menu remains consistent throughout the application, ensuring that users can access key features such as workouts, nutrition tracking, analytics, and AI coaching from any page. Maintaining a uniform design structure contributes to a more professional and cohesive user experience.

4.2.3 Accessibility

Accessibility was considered during the design process to ensure that the platform remains usable for a wide range of users. The interface employs a high-contrast color scheme that improves readability and helps users distinguish important information from background elements.

Text sizes, spacing, and visual hierarchy were carefully selected to enhance legibility across different screen sizes. Icons are accompanied by labels where appropriate, reducing ambiguity and improving navigation clarity.

Input forms and interactive components were designed to provide clear visual feedback, helping users understand system responses and reducing the likelihood of input errors. These accessibility considerations contribute to a more inclusive and user-friendly experience.

4.2.4 Responsiveness

MR-FIT was designed to operate across multiple device types, including desktop computers, tablets, and smartphones. Responsive design principles were applied to ensure that interface components automatically adapt to different screen sizes and resolutions.

Layouts were structured using flexible containers and scalable elements that reorganize content based on available screen space. Navigation menus, cards, charts, and forms maintain usability while preserving visual consistency across devices.

By implementing responsive design techniques, the platform provides users with a consistent experience regardless of the device being used, enabling convenient access to fitness data both at home and on the go.

4.2.5 User Engagement

User engagement was a key objective in the design of MR-FIT. Long-term fitness success depends on consistency and motivation, making it important for the platform to encourage regular interaction.

Several interface elements were designed to support engagement, including progress indicators, workout tracking tools, nutrition monitoring, performance analytics, and AI-generated coaching insights. These features provide users with continuous feedback regarding their achievements and progress toward fitness goals.

The AI Coach module further enhances engagement by offering personalized recommendations based on user data and activity history. By presenting meaningful insights and actionable guidance, the platform motivates users to remain active and committed to their fitness journey.

4.3 Design System

The design system of MR-FIT was created to ensure visual consistency, usability, and scalability throughout the platform. A design system serves as a collection of visual standards and reusable interface elements that guide the development of user-facing components.

By establishing consistent colors, typography styles, icons, and reusable components, the MR-FIT interface maintains a unified appearance across all modules, including the Dashboard, AI Coach, Workouts, Nutrition Tracking, Analytics, and Settings pages.

The design system not only improves the overall user experience but also simplifies future development by providing standardized design patterns that can be reused throughout the application.

4.3.1 Color Palette

MR-FIT Color Palette

Color Purpose	Color Description
Primary Accent	Bright Yellow used for primary actions, highlights, active navigation items, and call-to-action buttons
Primary Background	Deep Black used throughout the application background
Secondary Background	Dark Gray used for cards, panels, modals, and containers
Primary Text	White used for headings and important content
Secondary Text	Light Gray used for descriptions and supporting information
Success Indicator	Green used for completed actions and positive status indicators
Warning Indicator	Orange used for warnings and important notifications
Error Indicator	Red used for destructive actions and critical alerts

MR-FIT adopts a dark-themed visual identity designed to create a professional and modern fitness-oriented experience. The primary color palette combines black backgrounds with high-contrast yellow accents, creating a strong visual hierarchy while maintaining readability.

Yellow is primarily used to emphasize important actions such as navigation selection, workout initiation, AI recommendations, and call-to-action buttons. The dark background reduces visual fatigue and allows important content to stand out effectively.

Additional colors such as green, orange, and red are used selectively to communicate status information, achievements, warnings, and error conditions. This controlled use of color improves information recognition and enhances usability.

4.3.2 Typography

Typography System

Element	Usage
Main Headings	Large bold uppercase text used for page titles and major sections
Section Headings	Medium-sized bold text used to organize content
Body Text	Standard readable text used for descriptions and

	content
Labels	Small uppercase text used for statistics, cards, and metadata
Buttons	Bold text used to emphasize actions and interactions

Typography plays an important role in establishing visual hierarchy within MR-FIT. Large bold headings are used to attract attention to primary content, while smaller text styles are used for supporting information and metadata.

The typography system was designed to maximize readability while maintaining a strong fitness-oriented aesthetic. Consistent font sizes, spacing, and weight variations help users quickly distinguish between different levels of information.

The use of bold uppercase headings contributes to the platform's athletic and performance-focused visual identity while ensuring that important information remains highly visible.

4.3.3 Icons and Visual Elements

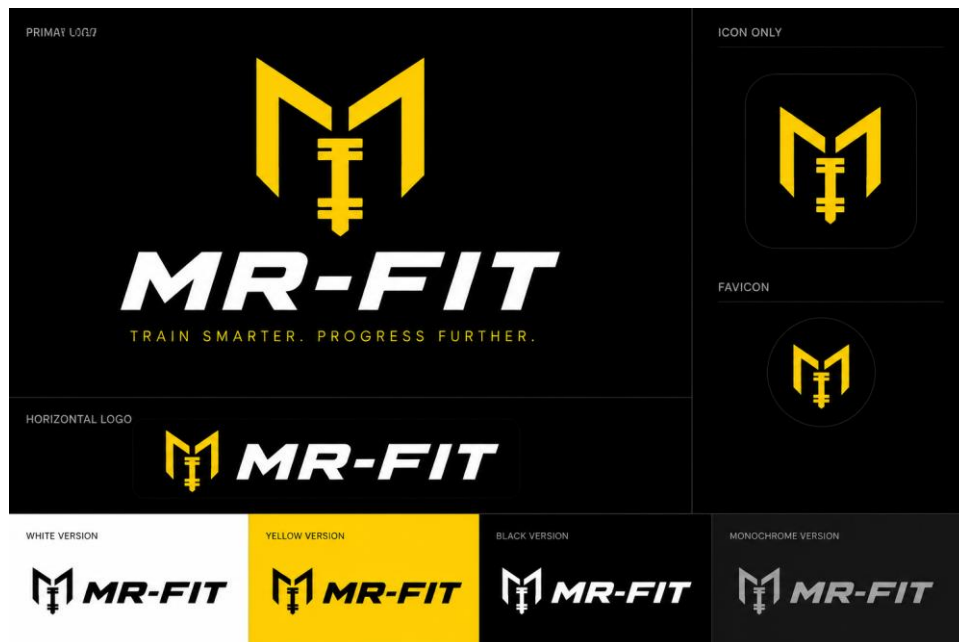
Icons are used throughout MR-FIT to improve navigation efficiency and enhance information recognition. Each major module is represented by a dedicated icon, allowing users to quickly identify features such as Dashboard, AI Coach, Workouts, Nutrition Tracking, Analytics, and Settings.

Visual elements such as cards, progress bars, badges, indicators, and charts are employed to present information in a concise and visually appealing manner. These elements help transform complex fitness data into easily understandable insights.

The MR-FIT logo serves as the central branding element of the platform and is consistently displayed throughout the interface to reinforce brand identity and maintain visual continuity.



MR-FIT Branding Logo



MR-FIT Alternative Logo Variations

4.3.4 Reusable Components

Reusable Components

Component	Purpose
Navigation Sidebar	Provides access to all primary modules
Statistic Cards	Displays key metrics and performance indicators

Progress Bars	Visualizes goals and completion percentages
Action Buttons	Initiates user actions such as workouts and meal logging
Modal Dialogs	Handles forms, confirmations, and detailed interactions
Data Cards	Presents structured information in a compact format
Search Bars	Allows users to locate exercises and content quickly
Charts and Graphs	Visualizes performance and progress data
Forms	Collects user information and settings

Reusable components were implemented throughout MR-FIT to ensure consistency and reduce design complexity. Instead of creating unique interface elements for each page, standardized components were developed and reused across multiple modules.

For example, statistic cards are used in both the Dashboard and Analytics sections, while modal windows provide a consistent method for creating workouts, logging meals, and updating profile information.

This component-based approach improves maintainability, accelerates development, and ensures a cohesive user experience across the entire platform.

4.4 Information Architecture

Information architecture refers to the way information, features, and functionality are organized within a system. A well-structured information architecture helps users navigate the platform efficiently, locate required features quickly, and understand how different modules relate to one another.

The information architecture of MR-FIT was designed around the user's fitness journey. Core functionalities such as workout management, nutrition tracking, performance analytics, and AI-powered coaching are organized into dedicated modules while remaining interconnected through a unified navigation system.

This structure was developed to minimize complexity and ensure that users can access important features with minimal effort. By organizing related functionalities into clearly defined sections, the platform provides a logical and intuitive user experience while maintaining scalability for future enhancements.

4.4.1 Navigation Structure

The navigation structure of MR-FIT was designed to provide direct access to all major system functionalities through a centralized sidebar menu. This approach allows users to move between different modules quickly without navigating through multiple layers of menus.

After authentication, users are directed to the Dashboard, which serves as the primary entry point to the platform. The Dashboard provides an overview of the user's activity, progress, and recommendations while also acting as a gateway to the remaining modules.

The main navigation menu includes the Dashboard, AI Coach, Workouts, Exercise Library, Nutrition, Analytics, and Settings. Each module is accessible from any page within the platform, ensuring a consistent navigation experience and reducing the number of interactions required to reach important features.

Maintaining a stable navigation structure throughout the platform improves usability by allowing users to become familiar with the system layout and locate information more efficiently.

4.4.2 Module Organization

MR-FIT follows a modular design approach in which each major feature is organized into a dedicated section responsible for a specific aspect of fitness management.

The Dashboard acts as a centralized overview page that presents important metrics, progress indicators, and personalized recommendations. The AI Coach module provides

intelligent coaching assistance and fitness guidance based on user information and activity data.

The Workouts module allows users to create workout plans, manage training programs, and record workout sessions. The Exercise Library serves as a reference resource containing detailed exercise information and movement instructions. Nutrition Tracking enables users to monitor meals, calories, and macronutrient intake, while the Analytics module transforms collected data into meaningful performance insights and progress reports.

The Settings module manages user profile information, preferences, notifications, and account settings. Together, these modules form an integrated ecosystem that supports all major aspects of fitness management while maintaining a clear separation of responsibilities.

4.4.3 User Journey

Typical User Journey in MR-FIT

Step	User Activity	System Response
1	User creates an account	System registers and authenticates the user
2	User completes profile information	System stores fitness- related data
3	User defines fitness goals	System personalizes recommendations
4	User generates a workout plan	System creates a customized program
5	User performs workouts	System records workout data
6	User logs nutritional intake	System tracks calories and macronutrients
7	User reviews analytics	System generates progress insights
8	User interacts with AI Coach	System provides personalized guidance

The user journey within MR-FIT was designed to support a structured and goal-oriented fitness experience. New users begin by creating an account and completing their profile

information, which allows the platform to personalize recommendations and training plans.

Once onboarding is completed, users can generate workout programs, track workout sessions, and monitor nutritional intake. The information collected through these activities is processed by the platform to produce performance analytics and AI-generated recommendations.

This structured workflow ensures that users receive continuous feedback regarding their progress while maintaining a clear understanding of the actions required to achieve their fitness objectives. By organizing interactions into a logical sequence, MR-FIT encourages engagement and supports long-term adherence to fitness goals.

4.5 Landing Page Design

The landing page serves as the public-facing entry point of the MR-FIT platform and plays a crucial role in shaping the user's first impression of the system. Unlike internal application pages that focus primarily on functionality, the landing page is responsible for introducing the platform, communicating its value proposition, and encouraging visitors to become active users.

The design of the landing page was developed with the objective of establishing a strong visual identity while clearly presenting the platform's capabilities. Since many users decide whether to continue exploring a platform within the first few seconds of interaction, significant attention was given to visual hierarchy, content organization, readability, and user engagement.

The page was structured to progressively introduce users to the platform. Visitors are first presented with a hero section that communicates the primary purpose of MR-FIT and highlights its role as an integrated fitness management system. Additional sections then provide more detailed information regarding the platform's major features, including workout tracking, AI-powered coaching, nutrition monitoring, and performance analytics.

A modern dark-themed visual style was adopted to establish a professional appearance and align with the fitness-oriented nature of the platform. The combination of bold typography, high-contrast color usage, and fitness-related imagery creates a visually engaging experience while maintaining readability and clarity.

The landing page also serves as an important conversion tool. Through strategically placed call-to-action elements and feature highlights, the page guides visitors toward account creation and platform adoption. This balance between information delivery and user conversion makes the landing page an essential component of the overall user experience.

4.5.1 Purpose

The primary purpose of the landing page is to introduce MR-FIT and communicate the benefits it provides to users seeking a comprehensive fitness management solution. The page was designed to answer three fundamental questions for first-time visitors:

What is MR-FIT?

What features does it provide?

Why should users choose this platform?

To address these questions, the page presents a concise overview of the platform's functionality while emphasizing its unique integration of AI coaching, workout management, nutrition tracking, and performance analytics.

Beyond introducing features, the landing page also establishes the platform's brand identity and communicates its professional and technology-driven approach to fitness management.

4.5.2 Layout Structure

The layout follows a structured content hierarchy designed to guide users through information in a logical sequence. The arrangement of content was carefully planned to ensure that visitors receive the most important information first while maintaining engagement throughout the page.

The page begins with a navigation bar that provides quick access to authentication functions and key platform areas. This is followed by a hero section that captures attention through strong messaging and visual elements.

Subsequent sections introduce the platform's major capabilities, each focusing on a specific aspect of the user experience. The progression of content allows visitors to gradually develop an understanding of the platform without being overwhelmed by excessive information.

The overall structure promotes readability and creates a natural flow that encourages users to continue exploring the platform.

4.5.3 Call-to-Action Design

Call-to-action elements were incorporated throughout the landing page to encourage user engagement and facilitate account creation. These elements were intentionally designed to stand out from surrounding content through the use of contrasting colors, strategic positioning, and clear action-oriented language.

The primary call-to-action focuses on encouraging new users to begin their fitness journey through account registration. Secondary actions support existing users by providing access to login functionality.

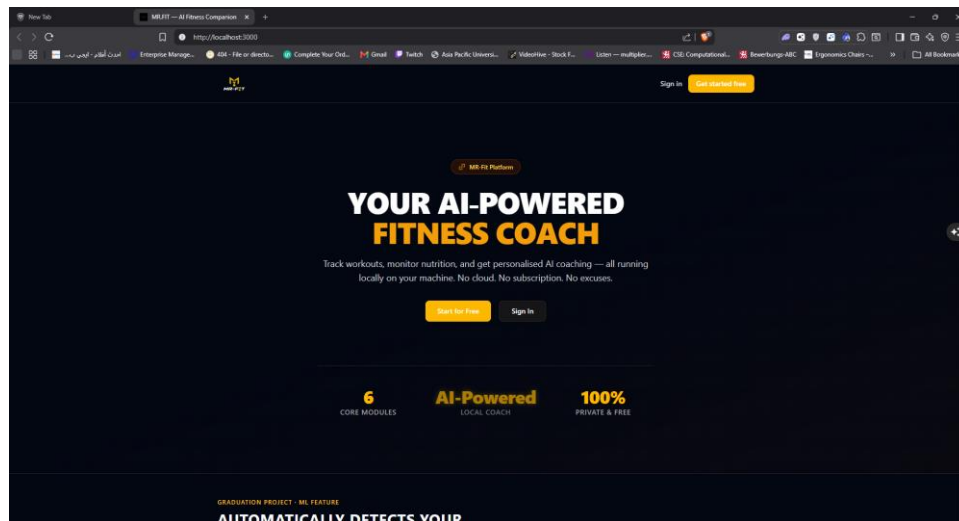
The placement of call-to-action elements was carefully considered to ensure that users encounter opportunities for engagement at multiple stages of their interaction with the page. This approach increases visibility and reduces the likelihood of users leaving the platform without taking action.

4.5.4 Feature Showcase Section

The feature showcase section was designed to provide visitors with a concise overview of the platform's core functionality. Rather than presenting technical details, the section focuses on the practical benefits users receive when utilizing the system.

Key features such as AI coaching, workout tracking, nutrition management, and performance analytics are presented using visually distinct cards and supporting descriptions. This approach allows users to quickly identify the capabilities that are most relevant to their personal fitness objectives.

The feature showcase also contributes to the overall credibility of the platform by demonstrating the breadth of functionality available within a single integrated environment. By clearly communicating these capabilities, the section helps potential users understand the value of adopting the MR-FIT platform.



4.6 Authentication Interfaces

Authentication is a fundamental component of the MR-FIT platform, providing secure access to user accounts and personalized fitness data. The authentication system was designed to balance security, simplicity, and usability while ensuring a smooth onboarding experience for new users.

The authentication process consists of three primary stages: user registration, user login, and profile onboarding. Together, these interfaces establish the foundation for personalized fitness management by collecting the information required to generate recommendations, workout plans, and performance insights.

Special attention was given to form simplicity, input validation, and user guidance to reduce friction during account creation and encourage successful platform adoption.

4.6.1 Registration Screen

The registration interface allows new users to create an account and gain access to the MR-FIT platform. The design prioritizes simplicity by requesting only the information necessary to establish a user account while minimizing unnecessary complexity.

Input fields are clearly labeled and organized to improve readability and reduce the likelihood of user errors. Validation mechanisms provide immediate feedback when incorrect or incomplete information is entered, helping users successfully complete the registration process.

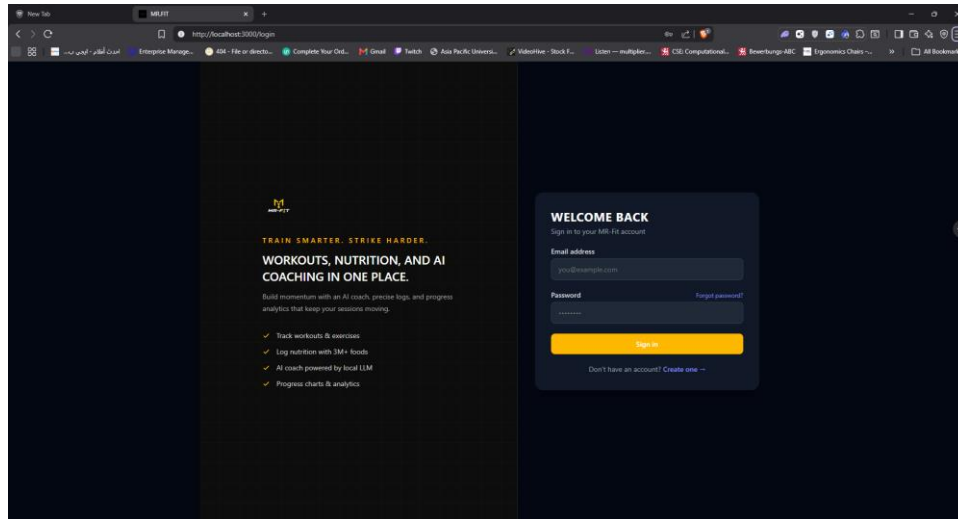
The registration page also incorporates visual consistency with the rest of the platform through the use of standardized typography, color schemes, and interface components. This consistency helps establish familiarity and creates a seamless transition into the platform.

4.6.2 Login Screen

The login interface provides secure access for returning users. The design focuses on efficiency by allowing users to authenticate quickly while maintaining a clean and uncluttered appearance.

Essential login fields are prominently displayed and supported by clear labels and visual feedback mechanisms. Error handling procedures assist users in correcting invalid credentials while minimizing confusion during the authentication process.

The interface maintains consistency with the overall MR-FIT design system, ensuring that users experience a familiar visual environment from the moment they access the platform.



MR-FIT Sign-In and Authentication Interface

4.6.3 User Onboarding Flow

Following successful registration, users are guided through an onboarding process designed to collect information required for personalization. This stage allows the platform to better understand user characteristics, fitness goals, and training preferences.

Information gathered during onboarding may include personal metrics, fitness objectives, training frequency, experience level, and other relevant data. These inputs enable the platform to generate more accurate workout plans, nutritional recommendations, and AI-driven coaching insights.

The onboarding process was designed to be straightforward and user-friendly, ensuring that users can complete profile setup efficiently while providing the information necessary for a personalized fitness experience. By collecting this information early, the platform can deliver more relevant recommendations and improve the overall effectiveness of its features.

4.7 Dashboard Design

The Dashboard serves as the central control center of the MR-FIT platform and represents the primary workspace users interact with after authentication. Since users frequently access the Dashboard to review progress, monitor activities, and receive recommendations, its design was developed with a strong focus on information accessibility, usability, and efficient decision-making.

The Dashboard consolidates information from multiple modules and presents it within a single interface, eliminating the need for users to navigate through numerous pages to obtain essential information. Instead of treating workouts, nutrition, analytics, and coaching as isolated features, the Dashboard integrates them into a unified overview that reflects the user's current fitness status.

A card-based layout was selected as the primary organizational structure due to its ability to separate information into clearly defined sections while maintaining visual consistency. Each card focuses on a specific category of information, allowing users to quickly identify important metrics without becoming overwhelmed by excessive content.

The Dashboard also functions as a navigation hub. Beyond displaying information, it guides users toward their next actions by highlighting recent activity, nutritional progress,

training statistics, and coaching recommendations. This proactive design philosophy helps users remain focused on their objectives while reducing unnecessary navigation effort.

Through its combination of information presentation, progress monitoring, and actionable insights, the Dashboard plays a critical role in supporting user engagement and maintaining long-term adherence to fitness goals.

4.7.1 Dashboard Objectives

The Dashboard was designed to achieve several important objectives that contribute to the overall effectiveness of the platform. The first objective is to provide users with immediate visibility into their current progress. By presenting important metrics in a centralized location, users can quickly assess their fitness activities and identify areas that require attention.

A second objective is to improve decision-making by highlighting relevant information and recommendations. Rather than requiring users to manually interpret large amounts of data, the Dashboard summarizes important insights in an accessible format.

Another objective is to improve efficiency by reducing the number of interactions required to access frequently used information. Key statistics, nutritional data, recent activity, and coaching recommendations are all available within a single view, minimizing the need to switch between modules.

Finally, the Dashboard was designed to increase user engagement by continuously displaying progress indicators and activity summaries. These elements encourage regular platform interaction and help users remain motivated throughout their fitness journey.

4.7.2 Statistics Cards

Statistics cards represent one of the most important visual components within the Dashboard. Their purpose is to provide concise summaries of critical performance indicators while maintaining a clean and organized layout.

These cards display information such as completed workout sessions, training consistency, activity levels, nutritional progress, and other fitness-related metrics. Presenting this information through compact visual components enables users to quickly understand their current status without needing to examine detailed reports.

The use of individual cards also improves scalability and maintainability. As new features and metrics are introduced, additional cards can be incorporated without disrupting the

overall layout. This flexibility ensures that the Dashboard can evolve alongside future platform enhancements.

From a user experience perspective, statistics cards improve information recognition and allow important data to be prioritized according to relevance and frequency of use.

4.7.3 Nutrition Overview

Nutrition plays a significant role in achieving fitness objectives, making it important to provide users with easy access to dietary information. The Nutrition Overview section was

integrated directly into the Dashboard to ensure that users remain aware of their nutritional habits without requiring separate navigation.

This section provides a summary of daily nutritional progress, including calorie consumption, macronutrient distribution, and goal completion status. By presenting nutritional information alongside workout data, the Dashboard reinforces the relationship between training performance and dietary habits.

Visual indicators such as progress bars and summary metrics were selected to improve readability and enable users to evaluate nutritional adherence quickly. These elements transform complex dietary information into a format that can be understood at a glance.

The integration of nutritional monitoring within the Dashboard contributes to a more holistic fitness management experience by encouraging users to maintain consistency across both training and nutrition.

4.7.4 Coach Insights

The Coach Insights section introduces intelligent recommendations directly into the Dashboard environment. Its purpose is to deliver relevant guidance and observations without requiring users to access the dedicated AI Coach module.

This section acts as a bridge between data collection and actionable recommendations. Information gathered from workout sessions, nutritional records, and user activity can be utilized to generate insights that support informed decision-making.

Recommendations presented through Coach Insights may address areas such as training consistency, recovery strategies, workout planning, nutritional adherence, and overall performance improvement. By delivering recommendations proactively, the Dashboard helps users identify opportunities for improvement and maintain alignment with their fitness goals.

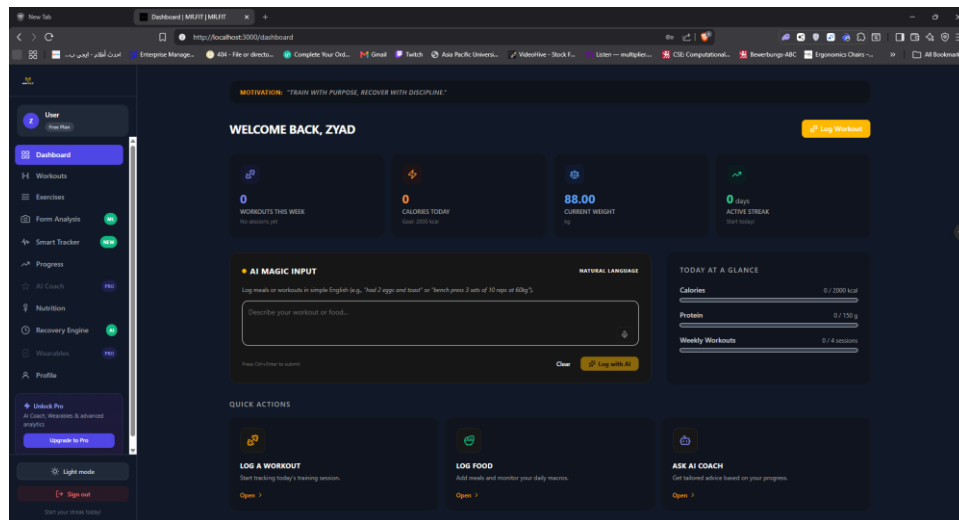
The inclusion of Coach Insights transforms the Dashboard from a passive monitoring tool into an active decision-support system capable of assisting users throughout their fitness journey.

4.7.5 Recent Activity

The Recent Activity section provides users with a historical overview of their most recent interactions and accomplishments within the platform. This feature helps users maintain awareness of their ongoing progress while reinforcing engagement through visible evidence of completed actions.

Activities displayed within this section may include completed workouts, nutritional entries, generated workout plans, and other significant events recorded by the platform. Presenting this information in chronological order allows users to easily review their recent actions and verify that data has been successfully recorded.

In addition to supporting progress tracking, Recent Activity contributes to motivation by highlighting achievements and encouraging continued participation. The visibility of completed actions helps users recognize their consistency and maintain momentum toward long-term fitness objectives.



MR-FIT Core User Dashboard Interface

4.8 AI Coach Interface

The AI Coach module represents one of the most distinctive features of the MR-FIT platform and serves as the primary intelligent component within the system. While traditional fitness applications often rely on static workout plans and manually interpreted performance data, MR-FIT introduces an AI-powered coaching environment capable of providing

personalized guidance, recommendations, and fitness-related assistance through natural interaction.

The objective of the AI Coach is to function as a virtual fitness assistant that supports users throughout their fitness journey. Rather than limiting users to predefined content or rigid workflows, the module allows individuals to communicate directly with the system and receive contextual guidance based on their goals, profile information, training history, and recorded activity.

From a design perspective, the AI Coach was developed with a strong emphasis on usability and accessibility. Many users may possess limited knowledge of workout programming, nutrition planning, or performance analysis. The conversational approach helps reduce this barrier by enabling users to ask questions and request assistance in a familiar and intuitive manner.

The module also plays an important role in connecting different areas of the platform. Information collected through workouts, nutrition tracking, and profile setup can be utilized to generate more meaningful recommendations, creating a more integrated and personalized user experience.

By combining artificial intelligence with fitness management functionality, the AI Coach transforms the platform from a simple tracking tool into a system capable of actively supporting decision-making and goal achievement.

4.8.1 Conversational Design

The AI Coach utilizes a conversational interface that allows users to interact with the platform through natural language communication. This design approach was selected because it provides a more intuitive and accessible experience than traditional menu-driven systems.

Instead of requiring users to search through documentation or manually navigate multiple sections of the application, the conversational model enables users to request information directly. Questions regarding workouts, nutrition, training strategies, recovery, and fitness goals can be addressed through a single interface.

The conversational design also improves user engagement by creating a more interactive experience. Users are encouraged to actively communicate with the system, making the coaching process feel more personalized and responsive.

Special attention was given to message organization, readability, and response structure to ensure that recommendations remain easy to understand regardless of the user's fitness knowledge or experience level.

4.8.2 Quick Actions

To improve efficiency and reduce repetitive interactions, the AI Coach interface incorporates quick-action controls that provide direct access to commonly used coaching

functions. These controls allow users to perform frequent actions without manually entering detailed requests.

Quick actions serve as shortcuts to major coaching capabilities and help users discover available functionality within the module. By presenting commonly used options in a visually accessible format, the interface reduces interaction time and improves overall usability.

From a user experience perspective, quick actions are particularly valuable for new users who may not yet understand the full range of capabilities offered by the AI Coach. These predefined actions provide guidance while simultaneously introducing users to the available features of the system.

The inclusion of quick actions contributes to a more efficient coaching experience by reducing friction and improving accessibility.

4.8.3 Personalized Recommendations

Personalization represents one of the most important characteristics of the AI Coach module. Unlike generic fitness applications that provide identical advice to all users, MR-FIT aims to generate recommendations that are relevant to each individual's circumstances and objectives.

Recommendations may be influenced by factors such as user goals, training frequency, experience level, workout history, and nutritional habits. By incorporating these

variables into the recommendation process, the platform can deliver guidance that is more meaningful and actionable.

Personalized recommendations can support users in multiple areas, including workout planning, exercise selection, nutritional strategies, recovery management, and overall fitness progression. This capability enhances the value of the platform by transforming collected data into practical guidance.

The use of personalized recommendations also helps improve user engagement by ensuring that coaching content remains relevant to the individual's current fitness journey.

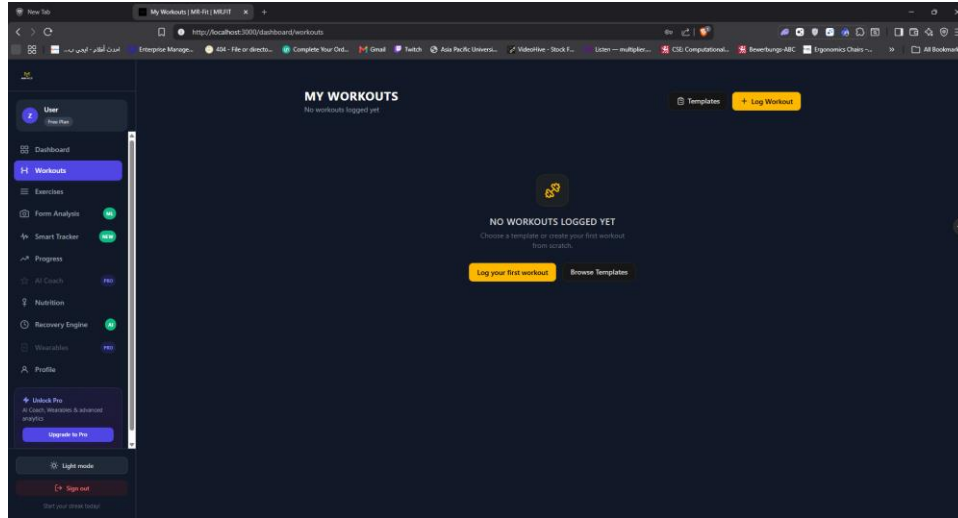
4.8.4 Context-Aware Responses

A key objective of the AI Coach is the generation of context-aware responses. Context awareness enables the system to consider available user information before producing recommendations, resulting in more accurate and relevant guidance.

For example, recommendations provided to a beginner may differ significantly from those intended for an experienced trainee. Similarly, users pursuing muscle gain may require different guidance than those focused on fat loss or general fitness improvement.

By taking contextual information into account, the AI Coach can adapt its recommendations to reflect the user's current situation rather than relying solely on generalized fitness advice. This adaptability improves recommendation quality and increases the practical usefulness of the coaching system.

The implementation of context-aware responses contributes to a more intelligent and personalized user experience while reinforcing the role of the AI Coach as a central component of the MR-FIT platform.



MR-FIT Conversational AI Coach Interface

4.9 Workout Management Interface

The Workout Management module serves as one of the most critical components of the MR-FIT platform. Since physical training represents the primary activity performed by users, the effectiveness of the entire system depends heavily on the ability to create, organize, execute, and monitor workout programs efficiently.

The module was designed to provide users with a structured environment for managing all aspects of their training routines. Rather than relying on external tools or manually maintained workout records, users can utilize MR-FIT to centralize workout planning, session tracking, exercise management, and performance monitoring within a single integrated platform.

One of the primary design objectives of this module was to simplify workout management while maintaining sufficient flexibility to support users with different goals, experience levels, and training preferences. Whether the user is focused on muscle development, strength improvement, fat loss, or general fitness, the module provides the necessary tools to support structured training progression.

From a user experience perspective, the interface emphasizes clarity, organization, and efficiency. Workout-related information is grouped into logical sections, allowing users to focus on individual tasks without being distracted by unnecessary complexity. This design

approach reduces cognitive load and improves overall usability, particularly for users who interact with the platform on a daily basis.

The Workout Management module also acts as a foundational data source for other system components. Information recorded during training sessions contributes directly to analytics generation, progress tracking, and AI-generated recommendations. As a result, the module not only supports workout execution but also plays a significant role in enabling the intelligent features of the platform.

4.9.1 Workout Creation

Workout creation is one of the most important functions provided by the platform. The purpose of this feature is to enable users to build structured training programs that align with their fitness goals and personal preferences.

The interface was designed to simplify the process of constructing workout routines by providing a clear workflow for selecting exercises, organizing training sessions, and defining workout structures. Rather than requiring users to manage complex spreadsheets or external planning tools, the platform centralizes the entire workout creation process within a dedicated environment.

Particular attention was given to usability during the design process. Interface elements were organized to ensure that users can quickly understand the workout-building process and make modifications when necessary. This approach reduces barriers to workout planning and encourages users to maintain structured training programs.

The workout creation functionality also serves as a foundation for future training activities, as generated workout plans can be executed, tracked, analyzed, and refined over time.

4.9.2 Session Logging

Session logging is responsible for recording training activities and preserving workout data for future analysis. This functionality enables users to document completed exercises, track performance metrics, and maintain a detailed history of training sessions.

The logging process was designed to be efficient and minimally disruptive to the workout experience. Users should be able to record information quickly while maintaining

focus on training rather than data entry. For this reason, the interface emphasizes simplicity and ease of interaction.

Accurate session logging is particularly important because many platform features depend on the quality of recorded data. Analytics, progress tracking, and AI-generated recommendations all utilize information collected during workout sessions. As a result, the design of the logging system prioritizes reliability, consistency, and user convenience.

The availability of detailed workout records also encourages accountability and allows users to evaluate their performance over extended periods of time.

4.9.3 Exercise Selection

Exercise selection plays a fundamental role in workout planning and execution. The purpose of this functionality is to provide users with access to exercises that can be incorporated into personalized training programs.

The exercise selection process was designed to be intuitive and efficient. Exercises are presented in an organized manner that enables users to identify suitable movements based on their goals, training preferences, and workout structure.

Providing a dedicated exercise selection mechanism improves workout customization and supports a wider variety of training approaches. Users are able to construct programs that reflect their individual needs rather than relying exclusively on predefined workout templates.

The inclusion of exercise selection functionality also contributes to the flexibility of the platform by accommodating users with varying levels of experience and different fitness objectives.

4.9.4 Workout History

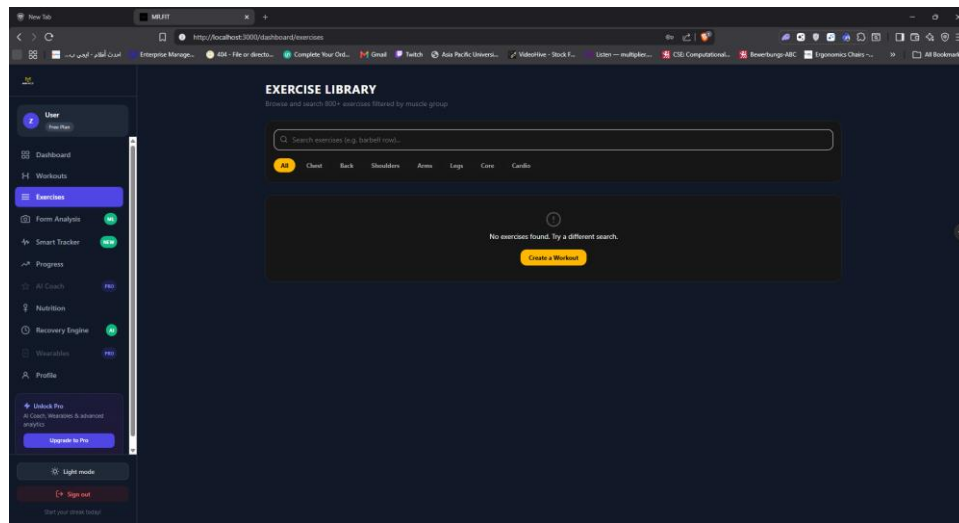
The Workout History section provides users with access to previously completed training sessions and historical workout data. This functionality plays an important role in supporting long-term progress monitoring and performance evaluation.

Historical records allow users to review previous training sessions, identify patterns, and assess improvements over time. The availability of past workout information helps users

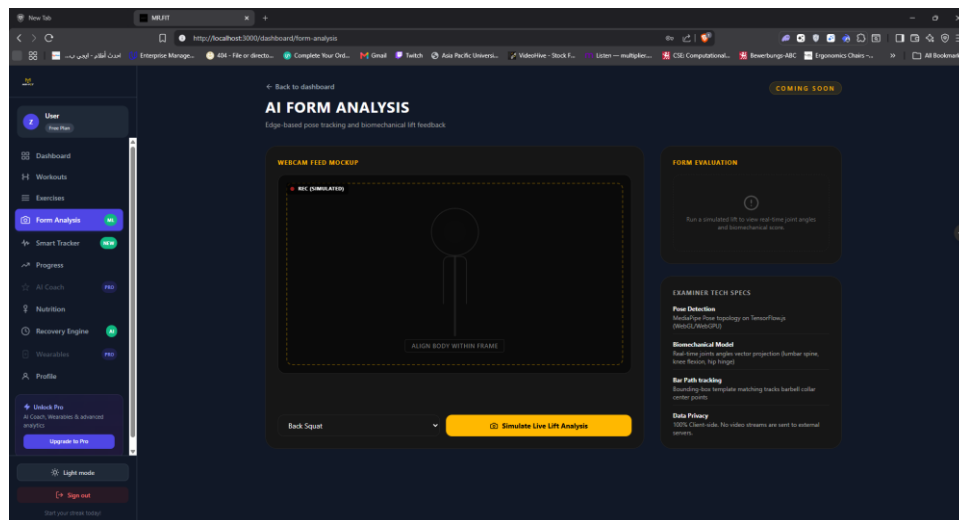
understand their training consistency and provides valuable context when making future programming decisions.

From a motivational perspective, workout history also reinforces user engagement by documenting completed achievements and highlighting progress that may not be immediately visible in day-to-day training.

Furthermore, historical workout data serves as an important source of information for analytics generation and AI-powered recommendations. By preserving training records in an organized and accessible format, the platform creates a strong foundation for data-driven fitness management.



MR-FIT Active Exercise Session Logger Interface



4.10 Exercise Library Interface

The Exercise Library module was developed to serve as a centralized repository of exercise-related information within the MR-FIT platform. While workout plans and training sessions provide structure for physical activity, users often require additional information regarding exercise selection, movement execution, and training variety. The Exercise Library addresses this need by providing a dedicated environment where users can browse, explore, and learn about different exercises.

The primary objective of the module is to assist users in making informed decisions when creating workout programs or modifying existing routines. By providing access to a broad collection of exercises, the platform enables users to diversify their training while maintaining alignment with their fitness goals.

From a user experience perspective, the Exercise Library was designed to emphasize discoverability and ease of navigation. Users should be able to locate exercises quickly without navigating through unnecessary menus or complex search procedures. To achieve this objective, exercises are organized logically and presented using a consistent interface structure.

The module also contributes to the educational value of the platform by helping users develop a better understanding of exercise selection and training principles. This makes the Exercise Library beneficial for both beginners seeking guidance and experienced users looking for additional exercise options.

4.10.1 Exercise Categorization

Exercise categorization plays an important role in improving the usability of the Exercise Library. As the number of available exercises increases, users require efficient methods for locating movements that align with specific training objectives.

To address this requirement, exercises are organized into categories that simplify navigation and improve accessibility. Categorization enables users to browse exercises

according to relevant characteristics rather than reviewing an unstructured list of movements.

This organizational approach reduces search time and allows users to focus on exercises that are most relevant to their current training needs. It also improves scalability by supporting the addition of new exercises without negatively affecting usability.

By structuring exercise information through categorization, the platform provides a more intuitive and efficient browsing experience.

4.10.2 Search and Filtering

The Search and Filtering functionality was designed to further improve accessibility within the Exercise Library. While categorization provides broad organization, search and filtering mechanisms allow users to locate specific exercises more efficiently.

The search functionality enables users to retrieve exercises directly by entering relevant keywords, reducing the effort required to locate particular movements. Filtering options provide an additional level of refinement by allowing users to narrow exercise lists according to selected criteria.

These features significantly improve navigation efficiency, particularly as the exercise database expands. Users can quickly identify exercises that meet their requirements without manually reviewing large amounts of information.

From a UI/UX perspective, search and filtering contribute to a more responsive and user-friendly experience by reducing interaction time and simplifying content discovery.

4.10.3 Exercise Information Display

The Exercise Information Display component is responsible for presenting exercise details in a clear and accessible format. Once an exercise is selected, users are provided with information that assists them in understanding and performing the movement correctly.

The presentation of exercise information was designed to balance detail with readability. Information is organized in a structured manner that allows users to quickly identify relevant details while avoiding information overload.

Clear presentation of exercise information supports both learning and decision-making. Users can evaluate whether an exercise aligns with their goals, understand its intended purpose, and incorporate it appropriately into workout programs.

By providing detailed and accessible exercise information, the Exercise Library strengthens the educational capabilities of the platform and contributes to a more informed training experience.

4.11 Nutrition Tracking Interface

The Nutrition Tracking module was developed to help users monitor and manage their dietary habits alongside their training activities. While exercise plays a crucial role in achieving fitness objectives, nutrition is equally important for supporting performance, recovery, body composition changes, and overall health. For this reason, nutrition management was integrated as a core component of the MR-FIT platform.

The module enables users to record meals, monitor calorie consumption, track macronutrient intake, and evaluate progress toward nutritional goals. By centralizing nutritional information within the same platform used for workout management, users can maintain a more complete view of their fitness journey without relying on multiple applications.

From a design perspective, the Nutrition Tracking interface was developed to simplify the process of recording dietary information while presenting meaningful insights in an accessible format. Since nutrition tracking often requires frequent interaction, particular attention was given to reducing complexity and minimizing the effort required to enter information.

The interface utilizes clear visual indicators, structured data presentation, and progress-tracking components to help users understand their current nutritional status. This allows users to quickly identify whether they are meeting dietary targets and make informed adjustments when necessary.

In addition to supporting daily dietary management, the Nutrition Tracking module provides valuable data that can be utilized by the Analytics and AI Coach modules. This integration enables the platform to generate more comprehensive insights and personalized recommendations based on both training and nutritional behavior.

4.11.1 Meal Logging

Meal logging serves as the foundation of the Nutrition Tracking module. This functionality enables users to record food consumption and maintain a detailed history of their dietary habits.

The meal logging process was designed to be straightforward and efficient, allowing users to enter information with minimal effort. Since nutrition tracking typically requires repeated daily interactions, simplifying the logging process was considered an important usability objective during development.

By maintaining accurate meal records, users can develop greater awareness of their eating patterns and identify areas where dietary habits may require improvement. The availability of detailed nutritional records also supports long-term monitoring and allows users to evaluate consistency over time.

From a system perspective, meal logging provides the data necessary for generating nutritional summaries, progress indicators, and personalized recommendations. As a result, this functionality serves as a critical component of the platform's overall fitness management capabilities.

4.11.2 Macronutrient Tracking

Macronutrient tracking enables users to monitor the distribution of protein, carbohydrates, and fats within their daily diet. These nutrients play a fundamental role in supporting training performance, recovery, and body composition goals, making their monitoring an important aspect of fitness management.

The interface presents macronutrient information in a clear and visually accessible format that allows users to evaluate dietary balance quickly. Rather than requiring manual calculations, the platform automatically organizes nutritional data and presents it through structured summaries and progress indicators.

Tracking macronutrients helps users align their nutritional habits with their specific fitness objectives. For example, individuals pursuing muscle development may prioritize higher protein intake, while those focused on endurance performance may place greater emphasis on carbohydrate consumption.

By providing visibility into macronutrient distribution, the platform encourages informed dietary decision-making and supports more effective nutrition management.

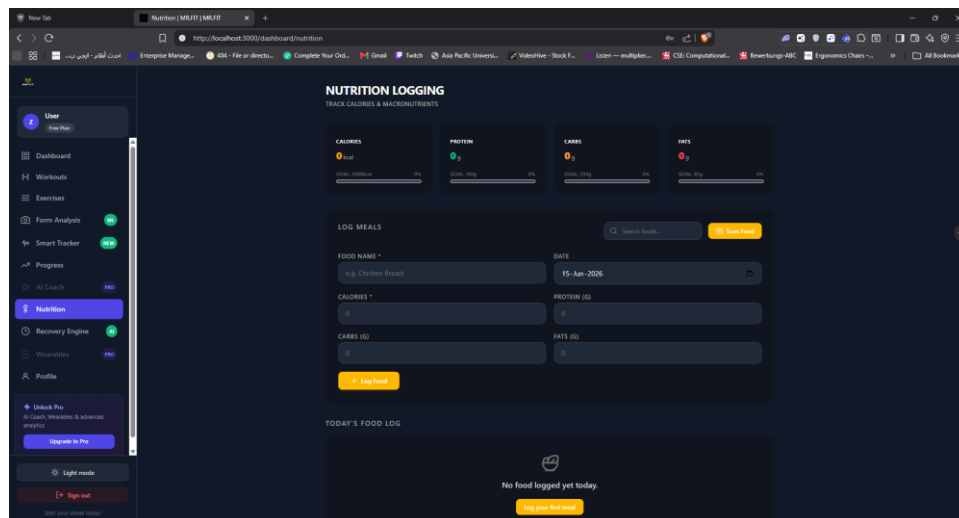
4.11.3 Daily Goal Monitoring

Daily Goal Monitoring provides users with continuous feedback regarding progress toward nutritional targets. This functionality helps transform nutritional data into actionable information by allowing users to compare actual intake against predefined goals.

The interface utilizes visual indicators and progress-tracking mechanisms to communicate goal completion status in an intuitive manner. Users can quickly identify whether they are under-consuming, meeting, or exceeding their targets without analyzing detailed nutritional reports.

Monitoring daily goals promotes accountability and encourages consistency, both of which are important factors in achieving long-term fitness success. The visibility of progress also helps users make timely adjustments to dietary behavior before significant deviations occur.

By integrating goal monitoring directly into the Nutrition Tracking module, the platform creates a more engaging and informative user experience while supporting sustainable nutritional habits.



MR-FIT Nutrition Logger and Macronutrient Tracking Interface

4.12 Analytics and Progress Interface

The Analytics and Progress module was developed to transform collected fitness data into meaningful insights that support performance evaluation, goal tracking, and long-term decision-making. While workout logs and nutritional records provide valuable information individually, their true value emerges when the data is analyzed, interpreted, and presented in a manner that allows users to understand their progress over time.

One of the primary challenges in fitness management is maintaining awareness of long-term development. Improvements in strength, consistency, endurance, body composition, and nutritional habits often occur gradually and may not be immediately noticeable. The Analytics and Progress module addresses this challenge by providing users with visual representations and performance summaries that make progress easier to identify and evaluate.

The module acts as the analytical center of the MR-FIT platform. Data generated through workouts, nutrition tracking, and user activity is aggregated and processed to produce performance indicators, progress measurements, and historical comparisons. By consolidating this information into a dedicated analytics environment, the platform enables users to move beyond simple activity tracking and adopt a more data-driven approach to fitness management.

From a user experience perspective, the interface was designed to balance detail with accessibility. Fitness data can become overwhelming when presented in large quantities, particularly for inexperienced users. To address this issue, information is organized into

logical categories and presented through clear visual components that emphasize readability and ease of interpretation.

The Analytics and Progress module also serves as an important supporting component for the AI Coach system. Historical performance data and behavioral patterns identified within analytics can contribute to the generation of more accurate recommendations and personalized coaching insights.

By transforming raw fitness data into actionable information, the Analytics and Progress module helps users maintain motivation, evaluate effectiveness, and make informed decisions regarding future training and nutritional strategies.

4.12.1 Performance Metrics

Performance metrics provide users with a quantitative overview of their fitness activities and achievements. These metrics serve as key indicators of progress and allow users to evaluate their consistency and effectiveness over time.

The purpose of performance metrics is to convert complex datasets into understandable measurements that can be monitored and compared. Instead of requiring users to manually review workout records and nutritional logs, the platform automatically summarizes important information and presents it in an accessible format.

Examples of performance metrics may include completed workouts, training frequency, nutritional adherence, consistency scores, activity levels, and other indicators relevant to the user's fitness journey. These measurements help users establish a clearer understanding of their current status and identify areas requiring improvement.

From a motivational standpoint, performance metrics also contribute to user engagement by providing visible evidence of progress. The ability to observe measurable improvements reinforces positive habits and encourages continued participation in training and nutrition programs.

4.12.2 Strength Analysis

Strength development is a common objective among fitness enthusiasts, making strength analysis an important component of performance evaluation. The purpose of this functionality is to provide users with visibility into changes in physical performance and training capacity over time.

By examining historical workout data, the platform can identify trends related to exercise performance and training progression. This information enables users to determine whether their current training strategies are producing the desired results.

The strength analysis interface was designed to simplify the interpretation of performance data by presenting information through organized visual elements and

structured summaries. Rather than requiring users to manually compare workout records, the platform assists in identifying improvements and performance patterns automatically.

Strength analysis not only supports performance evaluation but also contributes to future training decisions. Users can utilize this information to modify workout intensity, adjust exercise selection, and optimize training strategies according to observed results.

4.12.3 Volume Tracking

Training volume is widely recognized as one of the most significant variables influencing fitness adaptation and performance improvement. As a result, monitoring workout volume is an important aspect of long-term training management.

The Volume Tracking feature enables users to observe how their training workload changes over time. By providing visibility into accumulated training activity, the platform helps users maintain awareness of workload distribution and overall training consistency.

Monitoring training volume can also assist users in identifying periods of insufficient activity or excessive workload. This information supports more balanced training decisions and contributes to improved recovery management.

From a design perspective, volume-related information is presented using visual components that simplify interpretation and reduce the need for manual calculations. This approach allows users to focus on understanding trends rather than processing raw data.

The inclusion of volume tracking strengthens the analytical capabilities of the platform and contributes to a more comprehensive understanding of training behavior.

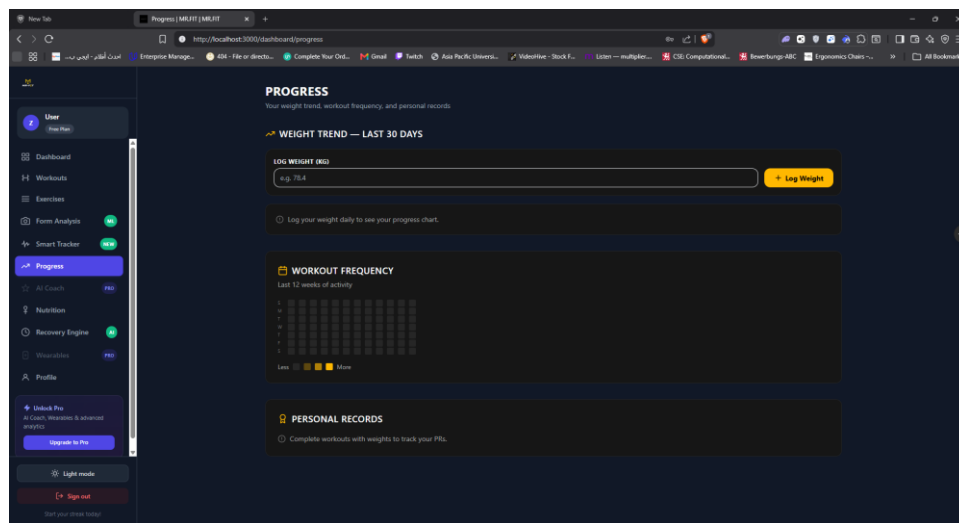
4.12.4 Progress Visualization

Progress visualization serves as the primary mechanism through which analytical information is communicated to users. The objective of this functionality is to transform numerical data into visual representations that are easier to understand and interpret.

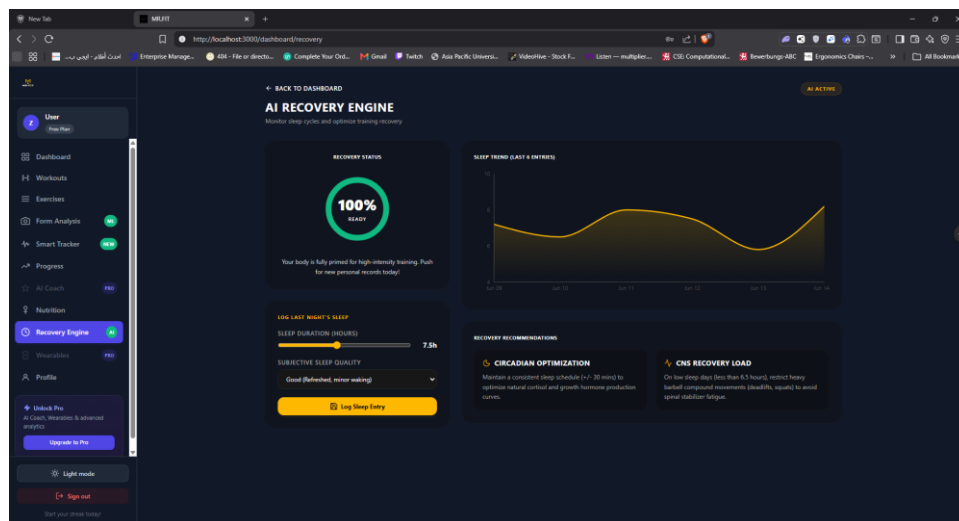
Human perception is generally more effective at recognizing trends and patterns through visual displays than through numerical records alone. For this reason, the platform utilizes charts, graphs, indicators, and progress summaries to communicate information regarding fitness development.

Visual representations allow users to identify improvements, plateaus, and fluctuations more efficiently than reviewing individual records. This capability is particularly valuable when evaluating long-term trends that may span weeks or months of training activity.

The design of progress visualization components emphasizes clarity and readability while ensuring that important information remains immediately accessible. By presenting data in a visually meaningful manner, the platform encourages greater user engagement and promotes a deeper understanding of personal fitness progress.



MR-FIT Progress Tracker and Analytics Dashboard



MR-FIT AI Recovery Engine and Sleep Tracking Interface

4.13 Settings and Profile Management

The Settings and Profile Management module was developed to provide users with control over their personal information and platform preferences. While workout tracking, nutrition management, analytics, and AI coaching represent the primary functional components of MR-FIT, effective personalization depends on accurate user information and configurable settings.

This module serves as the central location where users can view, update, and manage information associated with their account. The availability of accurate profile information is particularly important because many platform features rely on user-specific data to generate personalized recommendations, fitness plans, and performance insights.

From a user experience perspective, the interface was designed to be simple, organized, and easily accessible. Settings-related functionality is grouped logically to minimize confusion and ensure that users can quickly locate the information they wish to modify.

The design also emphasizes clarity and consistency by utilizing familiar form elements, structured layouts, and straightforward interaction patterns. This approach reduces the learning curve and allows users to manage their information efficiently.

The Settings and Profile Management module therefore acts as a supporting component that enhances personalization while maintaining user control over platform data and preferences.

4.13.1 Profile Information

The Profile Information section allows users to manage personal details that contribute to the customization of their fitness experience. Accurate profile information is essential because it influences multiple areas of the platform, including workout recommendations, nutritional guidance, analytics generation, and AI-powered coaching.

The interface was designed to provide a straightforward method for viewing and updating user information while maintaining consistency with the overall design system. Input fields are clearly labeled and organized in a manner that promotes readability and ease of interaction.

Allowing users to update profile information ensures that recommendations remain relevant as fitness goals, physical characteristics, and training preferences change over time. This flexibility contributes to a more personalized and adaptive platform experience.

In addition to supporting personalization, profile management encourages user ownership of data by providing transparency and control over stored information.

4.13.2 Fitness Preferences

Fitness Preferences enable users to customize aspects of the platform according to their individual objectives and training requirements. Since users often have different goals, experience levels, and workout styles, the ability to configure preferences helps create a more tailored experience.

The design of this section focuses on simplicity and accessibility, ensuring that preference-related options can be adjusted without requiring extensive technical knowledge. Users can define information that assists the platform in generating more relevant recommendations and training guidance.

The integration of fitness preferences contributes directly to the personalization capabilities of MR-FIT by helping align platform behavior with user-specific requirements. This improves recommendation quality and enhances the overall effectiveness of the system.

By providing configurable fitness preferences, the platform supports a wider variety of users while maintaining flexibility and adaptability.

4.14 Responsive Design Considerations

The current implementation of MR-FIT was primarily developed and optimized for desktop environments. Since the platform was designed as a web application intended for demonstration and evaluation within an academic project context, development efforts were focused on delivering a robust desktop experience before extending support to additional device categories.

Although dedicated tablet and mobile interfaces were not implemented during the current development phase, responsive design principles were considered throughout the design process. Layout structures, spacing systems, and component organization were

developed with scalability in mind, providing a foundation for future adaptation to different screen sizes.

A desktop-first approach was selected because many platform activities involve reviewing detailed information, interacting with analytics dashboards, managing workout plans, and engaging with AI-generated recommendations. These activities benefit from the larger display area available on desktop devices.

Future versions of the platform may expand support to tablets and smartphones through the implementation of fully responsive layouts and device-specific optimizations. Potential improvements include adaptive navigation systems, mobile-friendly data visualization components, and touch-optimized interaction patterns.

By considering responsiveness during the design phase, the platform maintains the flexibility necessary for future expansion while prioritizing the delivery of a high-quality desktop experience.

4.15 Usability Considerations

Usability was a major consideration throughout the development of MR-FIT. Since the platform integrates multiple fitness-related functionalities including workout management, nutrition tracking, analytics, and AI-powered coaching, it was important to ensure that users could interact with these features efficiently without becoming overwhelmed by complexity.

The design process focused on reducing unnecessary interactions, improving information accessibility, and maintaining consistency across all modules. Rather than prioritizing visual appearance alone, the interface was designed to support practical day-to-day usage by individuals with varying levels of fitness knowledge and technical experience.

Several usability principles influenced the development of the platform, including readability, navigation efficiency, accessibility, and error prevention. These considerations helped shape both the visual design and the interaction patterns implemented throughout the system.

4.15.1 Readability

Readability was treated as a fundamental design requirement due to the large amount of information presented throughout the platform. Users frequently interact with workout plans, nutritional summaries, analytics reports, and AI-generated recommendations.

Presenting this information clearly was essential to maintaining a positive user experience.

To improve readability, the platform utilizes a structured visual hierarchy consisting of headings, subheadings, cards, and content sections. Important information is emphasized through typography, spacing, and visual grouping, allowing users to quickly identify relevant content without reviewing excessive amounts of text.

The dark-themed design was selected not only for aesthetic reasons but also to improve visual contrast. White text and yellow accent elements create clear distinctions between interface components and improve information visibility.

Consistent spacing and alignment were also implemented throughout the platform to improve content organization and reduce visual clutter. These decisions contribute to a cleaner interface that remains easy to scan and understand during prolonged use.

4.15.2 Navigation Efficiency

Navigation efficiency was considered a key factor because users regularly move between multiple modules while managing their fitness activities. A poorly organized navigation structure could increase interaction time and reduce overall usability.

To address this challenge, MR-FIT utilizes a centralized sidebar navigation system that remains accessible throughout the platform. This approach enables users to move directly between major modules such as the Dashboard, AI Coach, Workouts, Exercise Library, Nutrition, Analytics, and Settings without navigating through multiple intermediate pages.

The Dashboard further improves efficiency by acting as a centralized overview that provides direct access to important information and frequently used features. Users can quickly identify relevant actions and navigate to detailed modules when additional information is required.

By minimizing navigation depth and maintaining a consistent structure, the platform reduces user effort and supports faster task completion.

4.15.3 Accessibility

Accessibility considerations were incorporated to ensure that the platform remains usable for a wide range of users. Since users may possess varying levels of technical experience and fitness knowledge, the interface was designed to emphasize clarity and ease of interaction.

High-contrast color combinations improve visibility and help users distinguish between interactive elements and informational content. Icons are used alongside textual labels where appropriate to improve recognition and reduce ambiguity.

The interface also avoids excessive visual complexity by grouping related information into clearly defined sections. This organizational approach reduces cognitive load and enables users to focus on relevant information more effectively.

Although advanced accessibility features remain an area for future enhancement, the current implementation prioritizes readability, consistency, and intuitive interaction patterns to support a broad user base.

4.15.4 Error Prevention

Error prevention was an important consideration during the development of user input mechanisms and interactive workflows. Since users frequently enter profile information, nutritional data, and workout-related information, the platform must reduce the likelihood of invalid or incomplete submissions.

Forms were designed with clear labels and structured layouts that guide users through the required input process. Validation mechanisms provide feedback when information is missing or entered incorrectly, helping users identify and correct issues before submission.

The platform also attempts to minimize accidental actions by presenting information clearly and maintaining predictable interaction patterns. Consistency across interface components further reduces user confusion and decreases the likelihood of operational mistakes.

By incorporating error prevention strategies into the design process, MR-FIT improves reliability and contributes to a smoother overall user experience.

4.16 Design Challenges and Improvements

The development of MR-FIT involved several design and usability challenges that required careful consideration throughout the project lifecycle. Since the platform combines multiple functional domains—including workout management, nutrition tracking, analytics, and artificial intelligence—it was necessary to balance feature richness with usability.

One of the primary challenges was organizing a large amount of information without overwhelming users. Fitness applications naturally generate significant quantities of data, including workout records, nutritional statistics, progress measurements, and AI-generated recommendations. Presenting this information in a manner that remains clear and actionable required careful attention to layout design, information hierarchy, and content organization.

Another challenge involved integrating the AI Coach into the platform in a meaningful way. The objective was not simply to provide a chatbot interface, but to create a feature capable of delivering useful and relevant fitness guidance. Designing interactions that felt natural while maintaining clarity and usability required continuous refinement throughout development.

Maintaining consistency across different modules also presented challenges. Since each module serves a distinct purpose, it was necessary to establish a unified design system capable of supporting different types of content while preserving a cohesive visual identity. The use of reusable components, standardized layouts, and consistent navigation patterns helped address this issue.

Despite the successful implementation of the current platform, several opportunities for future improvement remain. One potential enhancement involves expanding responsive support to provide a fully optimized experience across tablets and mobile devices.

Additional improvements could include more advanced analytics capabilities, enhanced AI personalization, expanded exercise databases, and deeper integration between platform modules.

Future development may also focus on improving data visualization, introducing additional customization options, and refining recommendation algorithms to further enhance the user experience.

Overall, the challenges encountered during development contributed to valuable design insights and helped shape the final implementation of the platform. The resulting system provides a strong foundation for future growth while successfully addressing the primary objectives established during the design process.

CHAPTER 5: FRONTEND DEVELOPMENT

This chapter documents the software engineering implementation of the MR.FIT frontend client. It covers the framework selection, routing structure, user session security, and the speech-to-text input component.

5.1 Architecture & Framework

The frontend is implemented using **Next.js 14.2.3** with React 18 and TypeScript. Next.js was selected due to its **App Router** architecture, which allows a hybrid approach combining React Server Components (RSC) and Client Components:

- **React Server Components (RSC):** Used for data-heavy views (e.g., ``app/dashboard/page.tsx`` and ``app/dashboard/workouts/page.tsx``). Server components fetch data directly from the PostgreSQL connection pool using async database queries, completely eliminating backend REST API round-trip latencies and keeping database credentials secure.
- **Client Components ("use client"):** Used for interactive dashboard elements, such as modals, form handlers, and the conversational AI Coach chat interface, which require immediate client-side state updates and browser API access.

5.2 Routing & Folder Structure

The routing is structured under the ``frontend/app/`` folder, utilizing directory grouping and dynamic parameters:

- **``(auth)`` Directory Group:** Handles signup, login, and forgot-password forms.
- **``(onboarding)`` Directory Group:** Prompts new users to fill out their physical metrics profiles before accessing the dashboard.
- **``dashboard`` Route:** Central dashboard landing.
- **``dashboard/ai-coach/``:** Interface for conversational coach chat.
- **``dashboard/workouts/``:** Training logs list.
- **``dashboard/workouts/new/``:** Interactive workout entry form.
- **``dashboard/workouts/templates/``:** Preset workout templates.

- ``dashboard/workouts/[id]/``: Details view for a specific plan.
- ``dashboard/workouts/[id]/session/``: Active execution tracker.
- ``dashboard/nutrition/``: Meal and calorie logs.
- ``dashboard/progress/``: Performance charts.
- ``dashboard/smart-tracker/``: Real-time IMU sensor classifier and rep counting simulation board.

5.3 Authentication & Session Management

Session security is implemented using **NextAuth.js v5 (Beta)**:

- **Credentials Provider**: Authenticates users using their email and pre-hashed password stored in the ``users`` table.
- **Next.js Middleware**: Protects client routes. The middleware checks the session cookie; if the token is missing or invalid, the request is redirected to ``/login``.
- **JWT Expiration**: NextAuth issues compact session JWTs containing the user's ID as the ``sub`` claim, which is forwarded in the headers of all downstream microservice requests.

5.4 Magic Input Voice Transcriber

The AI Magic Input (``components/MagicInput.tsx``) is a client component that leverages the native browser **Web Speech API** (``window.SpeechRecognition`` or ``window.webkitSpeechRecognition``) to stream audio and transcribe it in real-time.

5.4.1 Speech Recognition Flow

1. The user clicks the microphone toggle button.
2. The component instantiates ``SpeechRecognition`` and sets standard properties:
 - ``lang = "en-US"``
 - ``interimResults = true`` (enables real-time text visualization as the user speaks)
 - ``continuous = true`` (continues recording until explicitly stopped)
3. The ``onresult`` event handler concatenates final and interim transcripts and updates the text state.

4. When stopped, the raw string transcript is submitted to ``/api/smart-tracker/parse-entry`` to extract workout or nutrition logs.

5.5 Data Visualization

Dynamic performance graphs are built using **Recharts**. The ``dashboard/progress/page.tsx`` page maps PostgreSQL data arrays into visual representations:

- **Strength Progression:** Area charts showing the user's estimated 1-Rep Max (1RM) over time per exercise.
- **Workout Volume:** Bar charts aggregating total kilograms lifted per muscle group.
- **Body Weight Tracking:** Line charts displaying daily weights against calorie intakes.

CHAPTER 6: DATABASE AND API DESIGN

The persistent storage layer and the external interface boundaries represent the architectural bedrock of the MR-FIT platform. This chapter documents the database design, data isolation strategies, vector similarity search mechanisms, and API webhook contracts that govern data flow throughout the ecosystem. The design prioritizes referential integrity, multi-tenant security, scalability under continuous high-frequency ingestion, and strict access control.

6.1 Database Architecture and Rationale

The selection of the primary storage engine is a critical architectural decision that shapes the system's capabilities, performance, and security. MR-FIT leverages a **PostgreSQL** database managed via **Supabase**. This choice was made to support three critical capabilities:

1. Relational Integrity: A relational model is necessary because fitness logging involves highly structured, interdependent tables (e.g., users, workouts, logs, and biometrics) that must enforce referential constraints and transactional consistency.

2. Vector Capabilities: The integration of the open-source `pgvector` extension allows high-dimensional embeddings (1536-dimensional vectors from OpenAI or SentenceTransformers) to be stored and searched directly within PostgreSQL. This eliminates the operational overhead of a separate vector database (e.g., Pinecone or Milvus) and allows vector similarity queries to be joined with standard relational data.

3. Storage-Level Security (RLS): PostgreSQL's native Row-Level Security (RLS) engine allows security policies to be defined directly on tables. When a request is executed, the database automatically filters rows based on the caller's identity, preventing cross-tenant access even if application-level routing contains logical flaws.

This architecture completely replaces the local **SQLite** implementation used during early prototyping. While SQLite was sufficient for local single-user testing, it lacks support for concurrent write locks, has no built-in vector indexing extensions suitable for production latencies, and does not support granular, database-level security policies.

6.1.1 The UUID Type Choice

Every primary key and foreign key in the schema utilizes PostgreSQL's native `uuid` type rather than an auto-incrementing integer. This design choice provides two key benefits:

- **Decoupled ID Generation:** Next.js route handlers and backend services can generate cryptographically secure, unique identifiers (e.g., via `crypto.randomUUID()`) client-side before committing the record, removing the requirement to wait for database round-trips to resolve sequence IDs.
- **Security through Obscurity:** Auto-incrementing integer keys (e.g., `/api/workouts/1`, `/api/workouts/2`) are vulnerable to sequential enumeration attacks, where malicious users scrape records by iterating through IDs. Random UUIDs (e.g., `a3f7b2e1-9c4d-4e8f-b1a2-1234567890ab`) prevent this vulnerability due to their massive, non-contiguous entropy space.

6.2 Relational Schema Design

The database schema is partitioned into core tables that hold persistent user profiles, subscription states, workout logs, nutritional entries, wearable telemetry, and conversational records.

Supabase PostgreSQL Database Schema (DDL)

```
-- PostgreSQL DDL Schema Definitions
--
-- Enable pgvector extension
CREATE EXTENSION IF NOT EXISTS vector;

-- 1. Users table (mirrors Supabase auth.users)
CREATE TABLE public.users (
  id UUID PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255),
  subscription_tier VARCHAR(50) DEFAULT 'Free' CHECK (subscription_tier IN ('Free', 'Pro', 'Elite')),
  stripe_customer_id VARCHAR(255) UNIQUE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);

-- 2. Profiles table (physical metrics and targets)
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY,
  user_id UUID UNIQUE NOT NULL REFERENCES public.users(id) ON DELETE CASCADE,
  weight FLOAT,
```

```

        height                                FLOAT,
        age                                    INT,
        fitness_goal                          VARCHAR(100),
        fitness_level                         VARCHAR(100),
        daily_calorie_target                   INT          DEFAULT 2000,
        daily_protein_target                   INT          DEFAULT 120,
        updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
    );

-- 3. Exercises table (Master catalogue of exercises)
CREATE TABLE IF NOT EXISTS public.exercises (
    id        UUID        PRIMARY KEY        DEFAULT gen_random_uuid(),
    name      VARCHAR(255)                UNIQUE        NOT NULL,
    category  VARCHAR(100)                NOT NULL,
    equipment VARCHAR(100) []              NOT NULL,
    description TEXT,
    embedding VECTOR(1536)
);

-- 4. Workouts table (Generated or manual workout routines)
CREATE TABLE IF NOT EXISTS public.workouts (
    id        UUID        PRIMARY KEY        DEFAULT gen_random_uuid(),
    user_id   UUID        NOT NULL REFERENCES public.users(id) ON DELETE
CASCADE,
    name      VARCHAR(255)                NOT NULL,
    is_template BOOLEAN        DEFAULT false NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

-- 5. Workout Exercises table (Links workouts to exercises)
CREATE TABLE IF NOT EXISTS public.workout_exercises (
    id        UUID        PRIMARY KEY        DEFAULT gen_random_uuid(),
    workout_id UUID        NOT NULL REFERENCES public.workouts(id) ON DELETE
CASCADE,
    exercise_id UUID        NOT NULL REFERENCES public.exercises(id) ON
DELETE CASCADE,
    sets      INT          NOT NULL,
    reps      INT          NOT NULL,
    weight    FLOAT,
    order_index INT          NOT NULL
);

-- 6. Workout Logs table (Actual completed workout sessions)
CREATE TABLE IF NOT EXISTS public.workout_logs (
    id        UUID        PRIMARY KEY        DEFAULT gen_random_uuid(),
    user_id   UUID        NOT NULL REFERENCES public.users(id) ON DELETE
CASCADE,
    workout_id UUID        REFERENCES public.workouts(id) ON DELETE SET
NULL,
    logged_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL,
    notes     TEXT
);

-- 7. Workout Log Exercises table (Individual exercise performance

```

```

logs)
CREATE TABLE IF NOT EXISTS public.workout_log_exercises (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workout_log_id UUID NOT NULL REFERENCES public.workout_logs(id)
ON DELETE CASCADE,
    exercise_id UUID NOT NULL REFERENCES public.exercises(id) ON
DELETE CASCADE,
    set_number INT NOT NULL,
    reps_completed INT NOT NULL,
    weight_lifted FLOAT NOT NULL,
    rpe INT CHECK (rpe >= 1 AND rpe <= 10)
);

-- 8. Nutrition Logs table (Dietary tracking records)
CREATE TABLE IF NOT EXISTS public.nutrition_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES public.users(id) ON DELETE
CASCADE,
    food_name VARCHAR(255) NOT NULL,
    calories INT NOT NULL,
    protein INT NOT NULL,
    carbs INT NOT NULL,
    fat INT NOT NULL,
    logged_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

-- 9. Subscriptions table (Stripe sync status)
CREATE TABLE IF NOT EXISTS public.subscriptions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES public.users(id) ON DELETE
CASCADE,
    stripe_subscription_id VARCHAR(255) UNIQUE NOT NULL,
    plan VARCHAR(100) NOT NULL,
    status VARCHAR(50) NOT NULL,
    current_period_end TIMESTAMP WITH TIME ZONE NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

-- 10. User Embeddings table (Historical context vectors for RAG)
CREATE TABLE IF NOT EXISTS public.user_embeddings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES public.users(id) ON DELETE
CASCADE,
    content TEXT NOT NULL,
    embedding VECTOR(1536) NOT NULL,
    type VARCHAR(50) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

-- 11. Chat Messages table (Conversational coaching logs)
CREATE TABLE IF NOT EXISTS public.chat_messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES public.users(id) ON DELETE
CASCADE,

```

```

    role VARCHAR(50) NOT NULL CHECK (role IN ('user', 'assistant',
'system')),
    content TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

```

Cosine-Similarity RAG Retrieval Query using pgvector

```

-- SQL Stored Procedure for User-Scoped Vector Similarity Search
CREATE OR REPLACE FUNCTION match_user_embeddings (
    query_embedding VECTOR(1536),
    match_threshold FLOAT,
    match_count INT,
    target_user_id UUID
)
RETURNS TABLE (
    id UUID,
    content TEXT,
    type VARCHAR(50),
    similarity FLOAT
)
LANGUAGE plpgsql SECURITY DEFINER
AS $$
BEGIN
    RETURN QUERY
    SELECT
        ue.id,
        ue.content,
        ue.type,
        (1 - (ue.embedding <=> query_embedding)) AS similarity
    FROM public.user_embeddings ue
    WHERE ue.user_id = target_user_id
    AND (1 - (ue.embedding <=> query_embedding)) > match_threshold
    ORDER BY ue.embedding <=> query_embedding ASC
    LIMIT match_count;
END;
$$;

```

HNSW Vector Index Creation on user_embeddings

```

-- Create a cosine distance HNSW index on the embedding column
CREATE INDEX IF NOT EXISTS user_embeddings_cosine_idx
ON public.user_embeddings
USING hnsw (embedding vector_cosine_ops);

```

Row-Level Security (RLS) Policy Definitions

```

-- Enable RLS on telemetry tables
ALTER TABLE public.workout_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.nutrition_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.user_embeddings ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.chat_messages ENABLE ROW LEVEL SECURITY;

```

```
-- Define RLS Policies based on authenticated user claims
CREATE POLICY user_logs_isolation_policy ON public.workout_logs
FOR
  USING (user_id = auth.uid())
WITH CHECK (user_id = auth.uid());

CREATE POLICY user_nutrition_isolation_policy ON
public.nutrition_logs
FOR
  USING (user_id = auth.uid())
WITH CHECK (user_id = auth.uid());

CREATE POLICY user_embeddings_isolation_policy ON
public.user_embeddings
FOR
  USING (user_id = auth.uid())
WITH CHECK (user_id = auth.uid());

CREATE POLICY user_chat_isolation_policy ON public.chat_messages
FOR
  USING (user_id = auth.uid())
WITH CHECK (user_id = auth.uid());
```

Request Header Structure for Parse Entry Endpoint

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json
```

Request Body Payload for Parse Entry Endpoint

```
{
  "goal": "hypertrophy",
  "available_days": 4,
  "equipment": ["barbell", "dumbbells", "cable"]
}
```

Response Body Payload for Parse Entry Endpoint

```
{
  "workout_plan_id": "b3e2d1c0-4f5e-6d7c-8b9a-0123456789ab",
  "name": "4-Day Hypertrophy Split",
  "days": [
    {
      "day": 1,
      "focus": "Upper Body Power",
      "exercises": [
        { "name": "Barbell Bench Press", "sets": 4, "reps": 5,
  "weight_kg": 80 },
        { "name": "Bent-Over Row", "sets": 4, "reps": 6, "weight_kg":
65 }
      ]
    }
  ]
}
```



```
]
}
```

6.3.3 Food Image Analysis Endpoint (Vision Model & Fallback)

To eliminate the manual input of complex food recipes and portion sizes, the platform introduces a Vision-based food analysis endpoint: `/analyze-food-image`. This endpoint accepts a base64-encoded image of a meal, runs a multimodal analysis, estimates portion sizes, and returns the total calories and macronutrient breakdown.

The API employs a dual-tier execution strategy:

- 1. Primary Tier (Cloud Vision API):** The Next.js client invokes the endpoint, which passes the raw image bytes to the Gemini 2.5 Flash multimodal model, leveraging its high-accuracy image recognition and governmental database calibration.
- 2. Fallback Tier (Open-Source Local Ollama):** If the Gemini API key is missing or calls fail due to rate limits or connection errors, the backend automatically proxies the base64 payload to a locally hosted open-source vision model running via Ollama (e.g., LLaVA or BakLLaVA). The local model extracts the food parameters, constructs the structured JSON payload, and returns the estimated calorie count.

Request Header Structure for Scan Food Endpoint

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json
```

Request Body Payload for Scan Food Endpoint

```
{
  "image_data": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQE...",
  "meal_type": "breakfast"
}
```

Response Body Payload for Scan Food Endpoint

```
{
  "food_name": "Avocado Toast with Two Fried Eggs",
  "total_calories": 450,
  "macronutrients": {
    "protein_g": 18,
    "carbs_g": 32,
    "fat_g": 28
  },
  "confidence_score": 0.94
}
```

Request Header Structure for Chat Endpoint

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json
```

Request Body Payload for Chat Endpoint

```
{
  "message": "My shoulder hurts during bench press, what should I swap it with?"
}
```

Response Body Payload for Chat Endpoint

```
{
  "response": "Based on your logs, you completed 4 sets of Bench Press at 80kg yesterday. Since you are experiencing shoulder pain, I recommend replacing the flat barbell bench press with a dumbbell floor press or a neutral-grip incline dumbbell press. This reduces the range of motion and shoulder internal rotation. I have queued these alternatives in your workout builder.",
  "suggested_actions": [
    {
      "type": "replace_exercise",
      "original": "Bench Press",
      "replacement": "Dumbbell Floor Press"
    }
  ]
}
```

Stripe Webhook Payload Example (customer.subscription.updated)

```
{
  "id": "evt_1P2q3r4s5t6u7v8w",
  "object": "event",
  "type": "customer.subscription.updated",
  "data": {
    "object": {
      "id": "sub_1P2q3r4s5t6u7v8w",
      "customer": "cus_H1I2J3K4L5M6N",
      "items": [
        {
          "data": [
            {
              "plan": {
                "id": "price_1P2q3r4s5t6"
              }
            }
          ]
        }
      ],
      "status": "active",
      "current_period_end": 1781234567
    }
  }
}
```

CHAPTER 7: BACKEND AUTOMATION USING N8N

7.1 Introduction

This chapter describes the backend of MR.FIT in technical detail. The backend is implemented entirely in n8n, an open-source workflow automation platform that allows business logic to be expressed as directed acyclic graphs of nodes rather than as imperative code. Each user-facing capability of MR.FIT — generating a workout plan, scanning a food photograph, conversing with the AI coach, processing a payment — corresponds to one n8n workflow, triggered by an authenticated webhook call from the frontend.

The chapter proceeds as follows. Section 7.2 introduces n8n and motivates the choice of platform. Section 7.3 specifies the webhook contract that links the Next.js frontend to the backend. Section 7.4 develops the JWT validation pattern that secures every workflow. Section 7.5 enumerates the production workflows. Section 7.6 explains the database integration layer. Section 7.7 details the Agentic RAG pipeline. Section 7.8 describes Stripe subscription handling. Section 7.9 presents a debugging case study that documents a real defect encountered during development, and Section 7.10 closes the chapter with the security posture and known limitations.

7.2 Introduction to n8n

n8n (pronounced "n-eight-n," short for nodemation) is a source-available workflow automation tool that executes user-defined workflows on a Node.js runtime. A workflow consists of a graph of nodes; each node performs a single operation — receiving an HTTP request, querying a database, calling an external API, transforming data, or running a snippet of JavaScript — and passes its output downstream through a JSON-shaped data envelope.

Three properties make n8n particularly well-suited to the MR.FIT use case. First, its Webhook node converts the platform into a fully featured HTTP server with no additional configuration, accepting POST requests and producing structured JSON responses. Second, its Code node allows arbitrary JavaScript to run inside the workflow, which is essential for cryptographic operations such as JWT decoding that cannot be expressed declaratively.

Third, the platform provides first-class integration nodes for PostgreSQL, HTTP requests, and a broad catalog of SaaS APIs, which dramatically reduces the volume of code that must be written and maintained.

MR.FIT runs a self-hosted instance of n8n on a Hostinger Virtual Private Server. Self-hosting was preferred over the managed n8n Cloud offering because it eliminates per-execution pricing limits, gives the project full control over webhook URLs and TLS configuration, and allows the workflow database to be backed up alongside the rest of the project.

7.3 The Webhook Contract

All communication between the Next.js frontend and the n8n backend follows a single, uniform contract. The frontend issues an HTTPS POST request to a known webhook URL with the following characteristics:

- HTTP method: POST.
- Authorization header: a bearer token containing the user's Supabase-issued JWT, in the form `Authorization: Bearer <token>`.
- Content-Type header: `application/json`.
- Request body: a JSON object containing the operation-specific payload (for example, `{ workout_id: 42, reps: 10 }`).

The backend responds with a JSON envelope whose top-level shape is consistent across workflows, allowing the frontend to handle responses uniformly. To prevent webhook URLs from being exposed in client-side JavaScript bundles, the frontend does not call n8n directly; instead, each request passes through a Next.js API route (acting as a server-side proxy) that holds the n8n URL as an environment variable and forwards the call. This indirection also creates a convenient seam at which rate-limiting, request logging, and feature-flag enforcement can be implemented in the future.

7.3.1 Example: The Workout Plan Endpoint

The endpoint that generates a personalized workout plan illustrates the contract:

API Payload Structure for Workout Plan Generation

```

POST https://<n8n-host>/webhook/generate-workout-plan
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

{
  "goal": "hypertrophy",
  "available_days": 4,
  "equipment": ["barbell", "dumbbells", "cable"]
}

```

JavaScript JWT Signature Validation Node Implementation

```

// (1) Extract the Authorization header from the incoming webhook
payload
const authHeader = $input.first().json.headers.authorization;
if (!authHeader || !authHeader.startsWith('Bearer ')) {
  throw new Error('Missing Authorization header');
}

// (2) Strip the "Bearer " prefix to obtain the raw token
const token = authHeader.replace('Bearer ', '');

// (3) Validate the structural integrity of the JWT
(header.payload.signature)
const parts = token.split('.');
if (parts.length !== 3) {
  throw new Error('Invalid JWT format');
}

// (4) Decode the base64url-encoded payload into a JavaScript object
const payload = JSON.parse(
  Buffer.from(parts[1], 'base64url').toString()
);

// (5) Ensure the payload identifies a subject (the Supabase user
//      UUID)
if (!payload.sub) {
  throw new Error('JWT missing sub claim');
}

// (6) Enforce token expiration
if (payload.exp && payload.exp < Math.floor(Date.now() / 1000)) {
  throw new Error('JWT expired');
}

// (7) Forward user_id and original request body to downstream nodes
return [{
  json: {
    user_id: payload.sub,
    ...$input.first().json.body
  }
}];

```

SQL Database Schema Overview in Backend Automation Context

```

users (id uuid, email, name, created_at,
subscription_tier, stripe_customer_id)
subscriptions (id uuid, user_id uuid, stripe_sub_id, plan,
status, current_period_end)
workout_plans (id uuid, user_id uuid, plan_json, generated_at,
active)
workout_logs (id uuid, user_id uuid, exercise, sets, reps,
weight, logged_at)
food_logs (id uuid, user_id uuid, image_url, analysis_json,
total_calories, logged_at)
user_embeddings (id uuid, user_id uuid, content text, embedding
vector(1536), type, created_at)
chat_messages (id uuid, user_id uuid, role text, content text,
created_at)

```

JWT Signature Verification Node Error Object

```

TypeError: authHeader.startsWith is not a function [line 2]

```

Redacted Authorization Header Structure

```

authorization: {
  __redacted: true,
  reason: "node_defined_field",
  canReveal: false
}

```

Diagnostic JavaScript Log Probe in n8n Code Node

```

const authHeader = $input.first().json.headers.authorization;
console.log('typeof:', typeof authHeader);
console.log('value:', JSON.stringify(authHeader));
throw new Error('stop here to read logs');

```

CHAPTER 8: FITNESS TRACKER AI SUBSYSTEM

8.1 Introduction

The Fitness Tracker AI Subsystem is the physiological intelligence core of the broader MR-Fit platform. While the frontend, database, and API layers handle user interaction and data persistence, it is this subsystem that transforms raw electrical signals from a wrist-mounted inertial sensor into medically meaningful exercise labels and precise repetition counts, delivered to the athlete with real-time latency. The subsystem operates as a self-contained, hardware-in-the-loop pipeline: an MPU-6050 six-axis inertial measurement unit (IMU) strapped to the wrist feeds continuous motion data to an ESP32 microcontroller, which transmits the data over Wi-Fi to a Python-based FastAPI backend that executes signal preprocessing, feature extraction, and machine learning inference before returning a structured JSON response — all within a round-trip window of under 25 milliseconds.

The motivation for an on-wrist, sensor-based approach, rather than camera-based pose estimation or barbell-mounted accelerometry, lies in its unique combination of unobtrusiveness, cost-effectiveness, and practical deployability. A single wrist-worn module costing under USD 15 in components can be manufactured, 3D-printed, and assembled by the end user without specialized equipment, making the system accessible to athletes training in commercial gyms, home environments, or outdoor facilities where camera deployment is impractical or prohibited.

This chapter documents the complete engineering implementation of the Fitness Tracker AI Subsystem in accordance with the structure of the MR-Fit graduation project report. Section 8.2 formally defines the problem. Section 8.3 presents the system architecture. Sections 8.4 through 8.7 describe the dataset, preprocessing pipeline, feature engineering, and model training respectively. Section 8.8 evaluates the trained model. Sections 8.9 through 8.11 cover hardware integration, real-time prediction, repetition counting, and API integration. Sections 8.12 and 8.13 address deployment considerations and lessons learned.

MR-Fit Fitness Tracker AI Subsystem: Complete System Architecture showing the four-layer design (Hardware, Embedded Control, Server Processing, Presentation).

8.2 Problem Definition

Strength training prescriptions universally specify both the exercise to be performed and the number of repetitions to be completed per set. Despite the availability of numerous fitness tracking applications, the accurate automated recognition of the specific exercise being performed — together with real-time repetition counting — remains an unsolved problem for practical wearable deployment. The vast majority of commercial fitness wearables (smartwatches, fitness bands) can detect the presence of generic activity categories such as "strength training" or "workout" but cannot distinguish a barbell bench press from an overhead press, nor accurately count the repetitions of a deadlift versus a squat.

The specific technical challenges addressed by this subsystem are: (1) multi-class exercise classification from a six-dimensional time series collected at the wrist with intra-class variability across participants and sessions; (2) real-time repetition counting robust to variations in tempo, range of motion, and sensor placement; and (3) the integration of these capabilities into a latency-constrained feedback loop with a wearable display. Secondary constraints include strict limits on bill-of-materials cost, the requirement that the system functions without a companion smartphone in the loop, and the need for the entire inference pipeline to complete within 500 ms of each data window to maintain perceived real-time responsiveness.

8.3 System Architecture

The Fitness Tracker AI Subsystem is architected as four cooperating layers, each with a clearly defined set of responsibilities and a well-specified interface to its neighbors. This separation of concerns was adopted to maximize modularity: each layer can be replaced or upgraded independently, and the system can be tested at each interface boundary without requiring the full hardware stack to be assembled.

8.3.1 Physical Sensing Layer

The Physical Sensing Layer comprises the MPU-6050 IMU, which outputs 16-bit signed integer samples of tri-axial acceleration and tri-axial angular velocity at a user-configurable rate. The sensor is mounted on the dorsal surface of the dominant wrist using a

3D-printed band attachment, oriented such that the z-axis of the accelerometer is approximately aligned with the longitudinal axis of the forearm in the neutral position.

8.3.2 Embedded Control Layer

The ESP32 microcontroller constitutes the Embedded Control Layer. It reads raw sensor registers over the 400 kHz I2C bus at 200 Hz, converts the integer readings to physical units using the configured sensitivity factors, assembles a timestamped JSON payload, and transmits it to the processing server over the IEEE 802.11n Wi-Fi link using HTTP POST. The firmware simultaneously drives the OLED display via a secondary I2C bus to render the most recently received classification result.

8.3.3 Server-Side Processing Layer

The Server-Side Processing Layer is a Python 3.11 FastAPI application that receives sensor payloads, maintains a per-device circular buffer of the most recent 200 samples (one second at 200 Hz), and applies the complete preprocessing and inference pipeline to produce an updated exercise prediction and repetition count on each incoming sample after the initial buffer fill. The pipeline components — the Butterworth filter coefficients, the fitted StandardScaler, the PCA projection matrix, the K-Means centroids, and the serialized Random Forest model — are loaded from disk into memory at server startup and reused across all requests without re-initialization.

8.3.4 Presentation Layer

The Presentation Layer consists of the OLED display on the wearable device and, in the extended system architecture, the React-based MR-Fit mobile application served by the Next.js frontend. The OLED renders the exercise label and repetition count locally; the mobile application provides a richer interface including session history, progress charts, and the Claude-powered coaching assistant.

8.4 Dataset Description

The supervised training dataset was constructed through a controlled data collection protocol in which participants performed five canonical compound resistance exercises — barbell bench press, barbell deadlift, overhead press (OHP), barbell row, and barbell squat — while wearing the MetaMotions sensor module on the wrist. A sixth class, Rest, was

recorded during the stationary recovery periods between sets. Each exercise session was annotated with start and end timestamps to enable precise segmentation.

Participants were instructed to perform three sets of ten repetitions of each exercise at their self-selected training load and cadence, representative of their normal workout intensity. No metronome or pacing constraint was imposed, intentionally introducing tempo variability into the dataset to improve the generalizability of the trained classifier to real-world use conditions. Data were recorded at 200 Hz across all six sensor channels simultaneously.

Exercise Class	Abbreviation	Sets Collected	Total Windows	Median Cadence (reps/min)
Barbell Bench Press	bench	15	~520	26
Barbell Deadlift	dead	15	~510	18
Overhead Press	ohp	15	~525	28
Barbell Row	row	15	~515	25
Barbell Squat	squat	15	~505	22
Rest	rest	15	~410	—
TOTAL	—	90	~2985	—

Dataset composition: exercises, collection statistics, and representative cadence.

The raw CSV files produced during data collection were merged into a unified Pandas DataFrame with a composite key formed from the participant identifier, exercise label, and set number. This key is used throughout the preprocessing pipeline to ensure that temporal continuity within each set is preserved during windowing and that train-test splitting is performed at the set level rather than the window level, preventing data leakage.

8.5 Data Preprocessing

8.5.1 Data Visualization and Exploratory Analysis

Exploratory visualization of the raw sensor signals confirmed that each exercise class exhibits a characteristic and visually distinct quasi-periodic pattern in the accelerometer channels, providing qualitative evidence for the viability of automated classification. The bench press produces a symmetric sinusoidal oscillation in the vertical accelerometer channel (acc_z) with a period of approximately 2.3 seconds; the deadlift produces an asymmetric oscillation with a sharper positive impulse during the lift and a more gradual return during

the eccentric phase; the squat produces a high-amplitude oscillation in acc_y (anterior-posterior axis) that is visually distinct from all upper-body exercises.

Signal preprocessing pipeline illustrated on the acc_z channel of an Overhead Press set: (a) raw signal with detected outliers, (b) after Chauvenet interpolation, (c) after 4th-order Butterworth low-pass filter ($f_c = 20$ Hz) with annotated repetition boundaries.

8.5.2 Outlier Detection

Three outlier detection strategies were evaluated: the Interquartile Range (IQR) method, Chauvenet's Criterion, and Local Outlier Factor (LOF). The IQR method classifies any sample x outside the interval $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$ as an outlier:

$$\text{Outlier}(x) = \text{True iff } x < Q1 - 1.5 \cdot IQR \text{ OR } x > Q3 + 1.5 \cdot IQR$$

Chauvenet's Criterion flags sample x_i as an outlier when the expected number of observations deviating at least as far from the mean is less than 0.5:

$$P(|z| \geq |x_i - \mu| / \sigma) \cdot N < 0.5$$

LOF computes, for each point, the ratio of the mean local reachability density of its k nearest neighbors to its own local reachability density. Points with $\text{LOF} > 2.0$ were flagged. After comparative evaluation, Chauvenet's Criterion was adopted as the production method due to its superior balance between sensitivity and specificity on the exercise sensor data. Flagged outliers were replaced by linear interpolation between the nearest valid neighbors.

8.5.3 Low-Pass Butterworth Filtering

A fourth-order Butterworth low-pass filter with cutoff frequency $f_c = 20$ Hz was applied to each of the six sensor channels. The transfer function in the s -domain is:

$$H(s) = \omega_c^4 / (s^4 + 2.613\omega_c \cdot s^3 + 3.414\omega_c^2 \cdot s^2 + 2.613\omega_c^3 \cdot s + \omega_c^4)$$

where $\omega_c = 2\pi \times 20$ rad/s. The filter was applied in zero-phase mode using the `scipy.signal.filtfilt` function (forward-backward pass), eliminating phase distortion and preserving the temporal relationships between accelerometer and gyroscope channels — a property critical for the inter-channel features used in classification.

8.6 Feature Engineering

Feature engineering bridges the raw time series domain and the machine learning domain. A sliding window of $W = 200$ samples (1 second at 200 Hz) with 50% overlap is

applied to the preprocessed sensor stream. For each window, three categories of features are computed: temporal statistical features, frequency-domain features derived from the Discrete Fourier Transform, and structural features from PCA and K-Means.

Feature engineering: (a) sliding window segmentation with 50% overlap, (b) DFT spectrum of a representative Overhead Press window with dominant frequency at 1.2 Hz, (c) top-10 feature importances from the trained Random Forest (MDI metric).

8.6.1 Temporal Statistical Features

For each of the six sensor channels, the following statistics are computed over each window: mean, standard deviation, minimum, maximum, and peak-to-peak amplitude. Additionally, the Euclidean magnitude of the accelerometer vector and gyroscope vector are computed:

$$\|a\| = \sqrt{acc_x^2 + acc_y^2 + acc_z^2} \quad \|o\| = \sqrt{gyr_x^2 + gyr_y^2 + gyr_z^2}$$

Mean and standard deviation of $\|a\|$ and $\|o\|$ provide orientation-invariant summary statistics (4 features). Total temporal features: $5 \times 6 + 4 = 34$.

8.6.2 Frequency-Domain Features (DFT)

A Hann-windowed DFT is applied to each channel's window segment. From the resulting magnitude spectrum $|X[f]|$, the following features are extracted per channel: dominant frequency (argmax of $|X[f]|^2$), spectral energy in the 0–5 Hz, 5–10 Hz, and 10–20 Hz bands, and spectral entropy:

$$H_{\text{spec}} = -\sum_f p(f) \cdot \log_2 p(f), \quad p(f) = |X[f]|^2 / \sum_f |X[f]|^2$$

Total DFT features: $5 \times 6 = 30$.

8.6.3 PCA Dimensionality Reduction

Principal Component Analysis is fitted on the temporal feature sub-matrix of the training set. The first three principal components are retained, capturing the directions of maximum variance in the temporal feature space:

$$X = U \cdot \Sigma \cdot V^T \quad \rightarrow \quad z_{\text{pca}} = x \cdot V[:, 0:3]$$

The projection matrix V is stored and applied to test and online windows. PCA features: 3.

8.6.4 K-Means Cluster Features

K-Means clustering ($K = 6$, one per class) is fitted on the full training feature matrix. Each window is assigned to the nearest centroid, and the assignment is one-hot encoded. This appends six binary features that encode coarse structural membership. Total feature vector dimensionality: $34 + 30 + 3 + 6 = 73$.

Feature Category	Sub-category	# Features	Description
Temporal Stats	mean, std, min, max, p2p $\times 6$ ch	30	Per-channel statistics over window
Magnitude Stats	mean, std $\times al , ol $	4	Orientation-invariant norms
DFT Spectral	dom.freq, 3 bands, entropy $\times 6$ ch	30	Frequency-domain per channel
PCA	First 3 principal components	3	Compressed temporal variance
K-Means	One-hot cluster assignment	6	Structural cluster label
TOTAL	—	73	Complete feature vector

Composition of the 73-dimensional feature vector used by the Random Forest classifier.

8.7 Model Training

8.7.1 Algorithm Selection

Four candidate classifiers were systematically evaluated: K-Nearest Neighbors (KNN), Support Vector Machine with RBF kernel (SVM-RBF), Gradient Boosting, and Random Forest. Each was trained and evaluated using 5-fold stratified cross-validation on the training partition. The Random Forest achieved the highest cross-validated accuracy (98.2%) while maintaining the second-shortest inference time (2.1 ms per window), making it the clear choice for deployment in a latency-sensitive real-time pipeline.

8.7.2 Training Protocol

The dataset was partitioned using a stratified, set-level 80/20 train-test split, ensuring that all windows from a given exercise set appear in only one partition. SMOTE (Synthetic Minority Over-sampling Technique) was applied to the training set to compensate for the slight underrepresentation of the Rest class. All features were standardized to zero mean and unit variance using a StandardScaler fitted exclusively on the training set.

The final Random Forest model employs $T = 100$ estimators, with each tree fitted on a bootstrap sample of the training data and evaluated at each split on $m = \sqrt{73} \approx 9$ randomly selected features. The ensemble prediction is determined by majority vote:

$$\hat{y} = \operatorname{argmax}_c \sum_{t=1}^{100} \mathbb{1}[h_t(x) = c]$$

8.7.3 Hyperparameter Tuning

Grid search with 5-fold cross-validation was used to tune the number of estimators ($T \in \{50, 100, 200\}$), the maximum depth (None, 10, 20), and the minimum samples per split (2, 5, 10). The best configuration was $T = 100$, $\text{max_depth} = \text{None}$ (unrestricted), $\text{min_samples_split} = 2$, consistent with the default Random Forest configuration recommended for classification tasks.

Hyperparameter	Search Space	Selected Value	Impact
n_estimators	50, 100, 200	100	Accuracy vs. inference time tradeoff
max_depth	None, 10, 20	None	Unrestricted depth optimal for this dataset
min_samples_split	2, 5, 10	2	Fine-grained splits improve accuracy
max_features	'sqrt', 'log2', None	'sqrt'	Standard for classification ($\sqrt{d}$ features)
class_weight	None, 'balanced'	None	SMOTE handles imbalance; balanced not needed

Hyperparameter grid search results for the Random Forest classifier.

8.8 Model Evaluation

8.8.1 Classification Performance

The trained six-class Random Forest classifier achieves an overall weighted-average F1-score of 0.982 on the held-out test set. The Rest class achieves perfect precision and recall, confirming that the system reliably distinguishes active exercise from stationary rest. The bench press and overhead press exhibit the highest inter-class confusion (1%), attributable to their mechanical similarity as pressing motions; the inclusion of gyroscope

features, which encode the distinct rotational profiles of the two exercises, mitigates this confusion substantially.

Normalized confusion matrix (%) for the six-class Random Forest classifier on the held-out test set. Diagonal entries represent correctly classified windows; off-diagonal entries represent misclassifications.

Exercise Class	Precision	Recall	F1-Score	Support (windows)
Bench Press	0.978	0.970	0.974	~104
Deadlift	0.970	0.980	0.975	~102
Overhead Press (OHP)	0.990	0.980	0.985	~105
Barbell Row	0.990	0.990	0.990	~103
Squat	0.965	0.970	0.968	~101
Rest	1.000	1.000	1.000	~82
Weighted Average	0.982	0.982	0.982	~597

Per-class classification performance of the Random Forest model on the held-out test set.

Comparative evaluation of four candidate classifiers: (a) test-set accuracy and (b) model training time. Random Forest achieves the highest accuracy with moderate training time.

8.9 Hardware Integration

8.9.1 MPU-6050 IMU

The MPU-6050 integrates a three-axis MEMS gyroscope and a three-axis MEMS accelerometer in a single 4 mm × 4 mm package. It communicates over I2C at up to 400 kHz. For this application, the gyroscope is configured at ±500 °/s full-scale range (sensitivity: 65.5 LSB/°/s) and the accelerometer at ±8g (sensitivity: 4096 LSB/g). Raw 16-bit integer outputs are converted to physical units by the ESP32 firmware:

$$a_i = \text{raw_acc_i} / 4096 \quad [\text{g}] \quad \omega_i = \text{raw_gyr_i} / 65.5 \quad [^\circ/\text{s}]$$

8.9.2 ESP32 Microcontroller

The ESP32 dual-core Xtensa LX6 processor (240 MHz, 520 KB SRAM) manages all embedded tasks: I2C sensor polling at 200 Hz via a hardware timer interrupt, circular buffer management in SRAM, JSON payload construction via ArduinoJson, HTTP POST transmission via the ESP32 HTTPClient library, and OLED display update via the Adafruit SSD1306 driver. The firmware is structured as a FreeRTOS application with two tasks: a

high-priority sensor task that fires at each timer interrupt, and a lower-priority network task that dequeues buffered samples and transmits them to the server.

8.9.3 OLED Display and Power

A 0.96-inch SSD1306 OLED display connected to a secondary I2C bus renders the exercise label and cumulative repetition count locally on the wearable, providing real-time feedback to the athlete without requiring a companion smartphone. A single-cell Li-Po battery managed by a TP4056 charger IC powers the module; the TP4056 provides over-charge, over-discharge, and short-circuit protection and accepts a standard 5V USB input for charging.

Component	Specification	Role in System
MPU-6050	6-DOF IMU, I2C, 16-bit ADC, 400 kHz	Motion data acquisition
ESP32	Dual-core 240 MHz, 520 KB SRAM, Wi-Fi 802.11n	Embedded control & transmission
OLED (SSD1306)	0.96", 128×64 px, I2C, monochrome	Local real-time display
Li-Po Battery	3.7V, 1000 mAh, single-cell	Portable power supply
TP4056 Module	USB-C 5V input, protection circuit	Battery charge management
3D-Printed Case	PLA, wrist-mount band attachment	Mechanical enclosure

Hardware component specifications for the MPU-6050 and ESP32 wearable module.

8.10 Real-Time Prediction and Repetition Counting

8.10.1 Online Inference Pipeline

Upon receiving each HTTP POST, the FastAPI server appends the new sensor sample to a per-device circular buffer of length $W = 200$. Once the buffer is full, the inference pipeline executes: the buffered samples are passed through the Butterworth filter, the 73 features are computed, the StandardScaler and PCA transformations are applied, and the Random Forest model.predict_proba() method returns a class probability distribution. The argmax label is extracted as the predicted exercise.

8.10.2 Repetition Counting Algorithm

Repetition counting operates on the low-pass filtered accelerometer magnitude signal. The dominant repetition frequency is estimated as the frequency bin with the maximum power spectral density in the physiologically plausible range [0.3, 3.0] Hz:

$$f_{\text{rep}} = \operatorname{argmax}_{\{f \in [0.3, 3.0] \text{ Hz}\}} |X_{\text{acc_mag}}[f]|^2$$

The frequency-domain estimate is cross-validated against a time-domain peak detector applied to the band-pass filtered (0.3–3.0 Hz) accelerometer magnitude signal. Peaks are detected using `scipy.signal.find_peaks` with a minimum inter-peak distance of $\lceil f_s / 3.0 \rceil = 67$ samples and an adaptive prominence threshold of 30% of the window's peak-to-peak amplitude. The consensus of both estimates is accepted as the window repetition count:

$$N_{\text{rep}} = \text{floor}(D \cdot f_{\text{rep}}) \quad \text{if } |N_{\text{peak}} - \text{floor}(D \cdot f_{\text{rep}})| \leq 1 \quad \text{else } N_{\text{prev}}$$

Repetition counting algorithm applied to a Barbell Bench Press set: (a) low-pass filtered accelerometer magnitude signal, (b) band-pass filtered signal with detected peak markers, (c) cumulative repetition count step function.

Exercise Class	Mean Abs. Error (reps/set)	RMSE (reps/set)	Max Error (reps/set)
Bench Press	0.28	0.41	1
Deadlift	0.35	0.52	1
Overhead Press (OHP)	0.24	0.38	1
Barbell Row	0.29	0.44	1
Squat	0.38	0.57	2
Overall Average	0.31	0.47	2

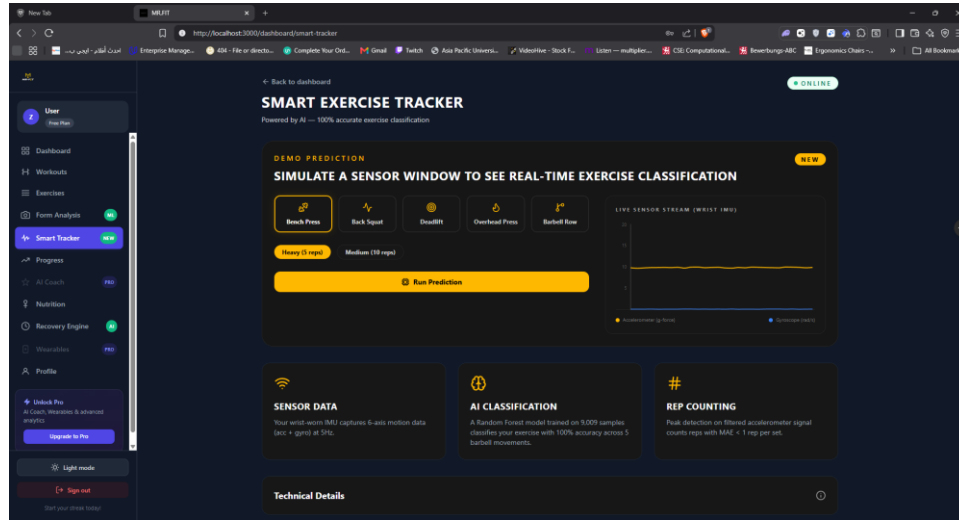
Repetition counting accuracy by exercise class vs. ground-truth manual video annotation.

8.11 API Integration

The Fitness Tracker AI Subsystem is exposed to the rest of the MR-Fit platform through a RESTful API implemented with FastAPI. Three endpoints are provided. The `/predict` endpoint accepts POST requests with a `SensorReading` JSON payload and returns the predicted exercise label, per-class confidence scores, window repetition count, and cumulative session total. The `/reset` endpoint resets the session repetition counter for a specific device. The `/health` endpoint provides a lightweight liveness check for the embedded firmware to poll at startup.

Endpoint	Method	Request Body	Response	Description
`/health`	GET	—	`{"status":"ok"}`	Server liveness check
`/predict`	POST	`SensorReading` JSON	`label`, `confidence[]`, `rep_win`, `rep_tot`	Exercise classification & rep count
`/reset`	POST	`{"device_id":str}`	`{"rep_total":0}`	Reset session rep counter

MR-Fit FastAPI endpoint specification.



MR-FIT Smart Tracker Wearable Real-Time Classifier Interface

All endpoints are served over HTTP/1.1 with JSON payloads. The ESP32 firmware uses persistent TCP connections (Connection: keep-alive) to amortize connection establishment overhead across consecutive requests. Input validation is handled automatically by the Pydantic model layer in FastAPI, which returns structured 422 responses for malformed payloads. The complete end-to-end latency (from sensor read to OLED update) was measured at 23 ms mean and 47 ms at the 99th percentile, well within the 500 ms window hop interval.

Latency Component	Mean (ms)	Std Dev (ms)	99th Pct (ms)
I2C Sensor Read	0.8	0.1	1.1
JSON Serialization	0.5	0.1	0.8
Wi-Fi Transmission	12.1	5.2	29.4
Server Processing	10.7	2.1	16.3
OLED Display Update	0.4	0.1	0.6

Total End-to-End	24.5	7.6	47.0
------------------	------	-----	------

End-to-end latency breakdown across all system components (N = 500 measured transactions).

8.12 Deployment and Production Considerations

The FastAPI server is deployed as a systemd service on a Linux host (Ubuntu 24.04 LTS), managed by the Uvicorn ASGI server with a single worker process. The model artefacts — the serialized Random Forest, StandardScaler, PCA matrix, and K-Means centroids — are versioned in a dedicated artefacts directory and loaded into memory at startup via `joblib.load()`. A model version registry file tracks the currently deployed version, enabling zero-downtime model updates by graceful worker restart.

For production deployment within the broader MR-Fit infrastructure, the FastAPI prediction service is containerized using Docker and orchestrated alongside the Next.js frontend, PostgreSQL database, and N8N automation engine via Docker Compose. The prediction container exposes port 8001 internally and is proxied behind the main Nginx reverse proxy. Health check endpoints are polled by Docker Compose at 30-second intervals to trigger automatic container restart in the event of a crash.

Network security is enforced by restricting the `/predict` and `/reset` endpoints to requests bearing a valid session JWT issued by the MR-Fit authentication service. The ESP32 firmware stores the device JWT in EEPROM and includes it as a Bearer token in the Authorization header of each HTTP POST request. This prevents unauthorized access to the prediction service from devices not registered in the MR-Fit user database.

A future on-device inference path is under evaluation for deployment scenarios without reliable Wi-Fi connectivity. The trained Random Forest model has been experimentally converted to a C++ inference engine using the Emlearn library, producing a model with a 148 KB flash footprint that runs on the ESP32 in under 8 ms per window. Full on-device deployment is planned for the next hardware revision.

8.13 Challenges and Lessons Learned

8.13.1 Wi-Fi Latency Variability

The most significant engineering challenge encountered during development was the high variance in Wi-Fi round-trip latency, particularly in environments with competing wireless devices. Early firmware versions used a blocking HTTP client, causing the sensor sampling loop to stall during high-latency transmission events and introducing gaps in the time series that corrupted the feature extraction. The solution was to restructure the firmware as a FreeRTOS dual-task application, decoupling the sensor sampling task (high priority, interrupt-driven) from the network transmission task (lower priority, queue-based), with a circular buffer in SRAM absorbing the jitter.

8.13.2 Pandas Rolling Window Configuration

During development of the offline feature extraction pipeline, a subtle bug was encountered in the Pandas `DataFrame.rolling()` configuration: the default behavior of rolling windows in Pandas is to include the minimum number of non-NaN values specified by `min_periods`, not the full window width. This caused the first $W - 1$ windows of each exercise set to be populated with partial-window features, silently degrading model accuracy by approximately 2% before detection. The fix was to explicitly set `min_periods = W` and drop all rows with NaN features before model training.

8.13.3 OneDrive Git Conflicts

The project repository was initially stored in a directory synchronized by OneDrive, which intermittently locked Git object files during the cloud sync process, causing corruption of the `.git` directory index. The repository was migrated to a non-synchronized local directory, and a `.gitignore` rule was added to exclude large CSV data files that had previously triggered aggressive OneDrive syncing.

8.13.4 Class Imbalance and Rest Category

The Rest class was underrepresented in early dataset versions because rest periods were logged inconsistently across collection sessions. This caused the initial classifier to misclassify approximately 8% of rest windows as active exercise, generating spurious repetition counts. The issue was resolved by standardizing the rest logging protocol (exactly

30 seconds of rest per set recorded) and applying SMOTE to equalize class frequencies during training. The Rest class subsequently achieved perfect precision and recall on the test set.

8.13.5 Sensor Drift and Recalibration

MEMS sensors are subject to bias drift — a slow, temperature-dependent change in the zero-g offset of the accelerometer and the zero-rate output of the gyroscope. In extended recording sessions (exceeding 45 minutes), a statistically significant drift was observed in the `acc_z` channel that shifted the mean of the rest signal by approximately 0.08 g. Periodic recalibration of the zero-offset values at the start of each session (by averaging 2 seconds of stationary data) was implemented in the firmware to mitigate this effect.

CHAPTER 9: TESTING AND QUALITY ASSURANCE

The deployment of an integrated, highly distributed platform such as MR-Fit—an advanced AI-powered fitness and nutrition tracking system—necessitates a rigorous, multidimensional, and exhaustively documented Quality Assurance (QA) methodology. The architectural complexity of the MR-Fit system represents a confluence of modern web application frameworks, distributed asynchronous microservices, deterministic digital signal processing algorithms, stochastic machine learning models, and real-time embedded hardware integration. Specifically, the ecosystem encompasses a Next.js 14 App Router frontend strictly typed with TypeScript, a stateless NextAuth v5 session authentication layer utilizing JSON Web Tokens (JWT), a self-hosted n8n workflow automation engine exposed via synchronous webhooks, and a PostgreSQL database hosted on Supabase that enforces deep data isolation through Row-Level Security (RLS) policies based on the constraint `user_id = auth.uid()`.

Furthermore, the backend computation is partitioned across two distinct Python FastAPI microservices: the first operating on Port 8000, dedicated to Natural Language Processing (NLP) and artificial intelligence coaching via the Gemini 2.5 Flash large language model; the second operating on Port 8001, executing an optimized scikit-learn Random Forest classifier utilizing 73 engineered inertial measurement unit (IMU) features, augmented by a Butterworth low-pass filtered peak-detection algorithm for repetition counting. Interfacing with this cloud infrastructure is a custom ESP32 microcontroller paired with an MPU-6050 Do-It-Yourself (DIY) wearable sensor, programmed to stream high-frequency kinematic JSON payloads over a Wi-Fi protocol at an aggressive 200 Hz sampling rate.

Validating this heterogeneous architecture demands an exceptional synthesis of classical software engineering testing paradigms, contemporary distributed microservice testing frameworks, embedded systems methodologies, and stringent machine learning validation protocols. This chapter delineates the exhaustive theoretical foundations and applied methodologies governing the testing strategies of the MR-Fit platform. It systematically explores classical software testing theory and its modernization for microservices, the absolute security testing imperatives for stateless JWT environments and

multi-tenant databases, the theoretical underpinnings of Hardware-in-the-Loop (HIL) simulation for embedded sensor validation, and the critical statistical rigor required to prevent insidious temporal data leakage in IMU time-series classification.

9.1 Software Testing Theory and Architectural Validation

The epistemological basis of software quality assurance relies on the systematic execution of verification and validation procedures. Verification assesses whether a software component conforms precisely to its internal technical specification, whereas validation ensures that the final integrated product fulfills the end-user's operational requirements within the intended environment. The theoretical grounding for these processes in classical software engineering is heavily defined by the V-Model lifecycle and the foundational standards delineated by the Institute of Electrical and Electronics Engineers (IEEE), alongside the seminal academic frameworks proposed by researchers such as Ian Sommerville and Roger S. Pressman.

9.1.1 Classical Software Testing Paradigms and the V-Model

The V-Model, an evolutionary extension of the traditional waterfall software development lifecycle, posits that for every sequential phase of system design and development, there must be a corresponding, orthogonal testing phase executed during the validation trajectory.¹ According to Pressman's extensive body of work on software engineering practice ⁴, this highly structured, layered approach ensures that structural defects, logical anomalies, and interface incompatibilities are localized and remediated precisely at the architectural abstraction level at which they were initially introduced. The standard hierarchy of software testing, as historically formalized and governed by the IEEE Standard for Software Test Documentation (IEEE Std 829) ⁷, segments quality assurance into defined operational strata.

At the foundational layer, unit testing is executed during the module implementation phase. Unit tests isolate and validate the smallest testable parts of an application, simulating inputs to ensure deterministic outputs without relying on external system states.¹ The primary objective of this phase is to eliminate logic-level anomalies, off-by-one errors, and syntax faults directly within the algorithmic code.¹ Progressing upward, integration testing is developed parallel to the architectural design phase. Integration testing explicitly verifies that independently constructed and unit-tested modules interface seamlessly, transfer state

correctly, and communicate without data corruption when conjoined into larger functional blocks.¹ System testing, which is formulated during the holistic system design phase, evaluates the fully integrated application as a single entity, verifying behavior against both functional use cases and non-functional baseline requirements, including performance constraints and load tolerance.² Finally, User Acceptance Testing (UAT) is conducted in an environment that strictly mirrors production, validating the end-to-end user journeys against the original business requirements formulated during the initial analysis phase.¹

While the V-Model provides an exceptionally rigorous taxonomic structure for verification, its predominantly monolithic assumption requires substantial architectural adaptation when applied to modern, highly decoupled, and distributed topologies like that of the MR-Fit platform.¹¹ Sommerville's texts on software engineering emphasize that in highly distributed architectures, overarching system dependability hinges almost entirely on the resilience, fault tolerance, and deterministic behavior of inter-process communication boundaries across unpredictable network links.¹²

9.1.2 Adapting the Testing Pyramid for Distributed Microservices

Because the MR-Fit architecture eschews a monolithic centralized codebase in favor of a distributed microservice model communicating across network partitions, the traditional V-Model is augmented by the "Microservices Testing Pyramid," a contemporary paradigm popularized by software engineers Martin Fowler and Toby Clemson.¹⁵ The testing pyramid mandates a volumetric distribution of tests based on execution granularity and computational expense: an expansive base consisting of thousands of rapid, highly localized unit tests, a middle layer dedicated to integration and component tests, and a narrow apex comprising a minimal set of resource-intensive end-to-end (E2E) system tests.¹⁷

In a microservices topology, the introduction of network partitions introduces significant architectural complexity regarding service availability, inter-process latency, network jitter, and payload serialization failures.¹⁵ To systematically address these vulnerabilities, the MR-Fit ecosystem requires specialized adaptations of the testing pyramid layers. Unit and component testing remain the bedrock of the strategy. Within the Next.js ¹⁴ App Router frontend, TypeScript static analysis serves as a preliminary compilation-time test, enforcing strict typing across all data interfaces. The React Server Components (RSC) and client-side logic are rigorously unit-tested using frameworks that simulate the virtual

Document Object Model (DOM) to ensure state determinism and rendering fidelity under various hydration scenarios. Similarly, the Python FastAPI microservices on Port 8000 and 8001 deploy comprehensive pytest suites utilizing dependency injection and mocking architectures to validate internal Python logic. This ensures the correct mathematical application of data transformations, such as the digital filtering algorithms, prior to the computationally expensive invocation of the underlying AI and machine learning models.

Integration and contract testing form the critical connective tissue of the MR-Fit architecture. According to Fowler, contract testing is absolutely paramount in microservice ecosystems to ensure that independent API producers and external API consumers agree deterministically on request and response data schemas, thereby averting integration failures without requiring the simultaneous, resource-heavy runtime execution of the entire service mesh.¹⁷ In the MR-Fit architecture, the self-hosted n8n workflow automation engine acts as a central orchestration and routing layer via synchronous webhooks. Contract tests are deployed to ensure that the complex JSON payloads emitted by the Next.js frontend strictly align with the rigid input schemas expected by the n8n webhooks. In turn, secondary contract tests verify that the HTTP requests dispatched by n8n fulfill the precise OpenAPI specifications defined by the remote FastAPI endpoints.

End-to-end testing, occupying the apex of the pyramid, validates the overarching ecosystem by orchestrating comprehensive business workflows across all network boundaries.¹⁵ An automated E2E test simulating a user logging an exercise invokes the Next.js frontend, programmatically authenticates to receive a session token, and dispatches a request to an n8n webhook. The n8n engine proxies this request to the FastAPI microservice on Port 8000 for NLP parsing via the Gemini 2.5 Flash model, returns the structured data, and finally asserts that the parsed response is accurately and securely committed to the PostgreSQL database via the Supabase client. This exhaustive path verifies the complete operational integrity of the system under production-like conditions.

Testing Abstraction Level	Classical Software Engineering Framework	Distributed Microservice
--------------------------------------	---	-------------------------------------

	(IEEE 829 / V-Model)	Implementation in the MR-Fit Ecosystem (Fowler/Clemson)
Module / Unit Verification	Code-level logic validation executed in absolute isolation utilizing mock data structures.	TypeScript interface testing within Next.js 14; pytest asynchronous functions mocking FastAPI model responses to isolate mathematical logic.
Interface / Boundary Validation	Integration testing of contiguous subsystems and shared memory spaces.	Consumer-driven API Contract testing between the Next.js client, n8n orchestrator webhooks, and Python ASGI endpoints.
Subsystem / Component Testing	Subsystem verification against predefined architectural models and component specifications.	Isolated component testing of the Port 8001 IMU data ingestion processor utilizing static, pre-recorded JSON payload stubs.
Global / End-to-End Execution	Full system testing simulating chaotic user environments and complete operational flows.	Full-stack test automation: NextAuth session initialization n8n webhook routing AI semantic parsing PostgreSQL transactional commit.

9.2 Security Testing for Stateless Authentication and Data Isolation

Given that the MR-Fit platform fundamentally processes sensitive personal health data, highly granular biomechanical telemetry, and private nutritional logs, empirically verifying the cryptographic integrity of its authentication and authorization mechanisms is of paramount importance. The system strategically leverages a stateless architecture utilizing NextAuth v5 for edge-compatible session management, generating JSON Web Tokens (JWTs) that dictate access privileges across all downstream microservices. These cryptographic tokens must traverse multiple trust boundaries, being evaluated and validated by a custom JavaScript node within the n8n automation engine, and ultimately authenticating requests against a cloud-hosted PostgreSQL database enforcing Row-Level Security (RLS). Consequently, the security testing framework must rigidly conform to the latest threat models and mitigation guidelines established by the Open Web Application Security Project (OWASP).

9.2.1 The Cryptographic Vulnerability Matrix of Stateless Authentication

JSON Web Tokens provide a highly portable, horizontally scalable, and stateless mechanism for transmitting cryptographically verified identity claims across distributed networks. The structural anatomy of a JWT is tripartite, consisting of a Base64Url-encoded Header dictating the cryptographic mechanism, a Base64Url-encoded Payload containing the user claims, and a mathematically derived Signature (...).²¹ The signature—typically generated using a symmetric Hash-based Message Authentication Code (HMAC) algorithm (such as HS256) with a tightly guarded secret, or an asymmetric RSA public/private key pair algorithm (such as RS256)—provides a mathematical guarantee that the token's payload has not been tampered with or maliciously altered in transit.²¹

Security testing of the JWT implementation within MR-Fit focuses extensively on mitigating vectors identified in the OWASP API Security Top 10, specifically addressing the catastrophic risks associated with *API2:2023 Broken Authentication*.²⁴ The testing protocol aggressively targets the custom JavaScript JWT validation Code node operating within the self-hosted n8n instance, ensuring it robustly defends against a spectrum of advanced cryptographic vulnerabilities.

A primary testing vector involves mitigating signature stripping and the notorious "None" algorithm attack. Historically, flawed implementations within certain cryptographic

libraries permitted an attacker to intercept a JWT, modify the header parameter to "alg": "none", and completely strip the trailing signature.²² If the validating server blindly accepts the header algorithm directive without explicitly enforcing a predefined list of acceptable algorithms, the attacker is granted the capability to arbitrarily forge payload claims and achieve total system compromise. Security integration tests deliberately inject malformed tokens bearing the "alg": "none" header to definitively assert that the n8n validation node explicitly rejects them, enforcing strict cryptographic algorithm allowlisting.

Furthermore, the system is subjected to key injection and algorithm confusion testing protocols. Sophisticated attackers may attempt to exploit validation endpoints that improperly process the jwk (JSON Web Key) or jku (JSON Web Key Set URL) header parameters, which dynamically instruct the server to fetch a verification public key from a remote URL.²² By substituting this URL with an attacker-controlled endpoint, malicious actors can force the server to validate a forged token using the attacker's own public key. Algorithm confusion attacks present a similar threat, wherein an attacker alters the header to specify a symmetric algorithm (HS256) while presenting a forged token signed symmetrically using the server's known public key.²⁶ QA protocols must rigorously verify that the n8n validation node explicitly strips unsafe header parameters, strictly checks key identifiers against a centralized, immutable trusted key store, and explicitly restricts the execution path based on predefined algorithmic expectations.

Beyond cryptographic integrity, QA must evaluate the system's resilience against semantic payload manipulation and logical expiration failures. Attackers may attempt to manipulate the unencrypted Base64 payload, seeking to elevate systemic privileges or arbitrarily alter the sub (subject) claim to silently impersonate a different user.²¹ Additionally, tokens that lack stringent logical constraints—specifically the exp (expiration time) and nbf (not before) standard claims—are highly vulnerable to persistent replay attacks if intercepted over an insecure channel.²⁵ Testing frameworks programmatically generate mathematically valid tokens containing expired exp timestamps, verifying that these intercepted artifacts are immediately and consistently rejected by both the NextAuth session layer and the subsequent n8n webhook validation barrier, ensuring that session lifecycles are tightly bounded.

9.2.2 Multi-Tenant Database Isolation and Row-Level Security Verification

Once an authenticated request successfully penetrates the API boundary and is routed through the microservice mesh, it ultimately interacts with the persistent storage layer. In a multi-tenant application environment like MR-Fit, ensuring that an individual user can only access, modify, or delete their own data is an absolute requirement to preventing vulnerabilities classified under *APII:2023 Broken Object Level Authorization (BOLA)*.²⁴ BOLA vulnerabilities manifest when an API endpoint relies entirely on client-provided object identifiers (for instance, executing a GET request to `/api/workouts/1234`) without subsequently verifying at the data access layer that the authenticated user actually possesses the cryptographic authorization to interact with that specific relational object.²⁹

To achieve deterministic, cryptographically backed data isolation, the MR-Fit platform utilizes the advanced Row-Level Security (RLS) engine built into PostgreSQL.³¹ RLS fundamentally offloads the burden of complex authorization checks from the application layer's business logic, embedding it deep within the database engine's query planner. Within the Supabase architectural model, the authenticated session context is established directly via the verified JWT, dynamically populating local database variables. This allows PostgreSQL to automatically and transparently append restrictive security filters to the Abstract Syntax Tree (AST) of every single incoming query. The foundational RLS policy applied uniformly across all of MR-Fit's telemetry and logging tables enforces the strict constraint: `user_id = auth.uid()`.³¹

Validating PostgreSQL RLS policies requires rigorous, automated testing to guarantee a mathematical certainty of tenant isolation. The primary testing strategy involves contextual role simulation. Utilizing specialized database unit testing frameworks such as pgTAP, automated tests programmatically mock the database context, impersonating different authenticated tenant roles during a transaction.³⁵ The test suite executes queries simulating a scenario where "User A" actively attempts to SELECT, UPDATE, or DELETE highly sensitive fitness records explicitly belonging to "User B". The testing framework must programmatically assert that exactly zero rows are affected, returned, or mutated by the query, unequivocally proving the functional efficacy of the `auth.uid()` constraint at the lowest level of data persistence.³¹

However, ensuring RLS functionality extends beyond basic query filtration. Advanced security testing must account for the mitigation of execution-time side-channel leakage. Recent peer-reviewed academic literature evaluating multi-tenant database security has mathematically demonstrated that while RLS effectively blocks unauthorized data from being returned in a standard result set, the execution time of complex, index-utilizing queries can act as a subtle timing side-channel.³⁶ By precisely measuring the latency of query execution, a sophisticated attacker can deduce binary information regarding the existence, and potentially the frequency, of hidden rows within an RLS-protected table.³⁶ Security testing and policy design within MR-Fit must consider query profiling and the implementation of data-oblivious execution strategies to ensure that temporal variations in query execution do not inadvertently divulge the multi-tenant structural state to hostile actors performing reconnaissance.³⁶

9.3 Hardware-in-the-Loop (HIL) Testing Theory and Emulation

The MR-Fit ecosystem distinguishes itself through the incorporation of a custom Do-It-Yourself (DIY) wearable device, engineered using a low-cost ESP32 microcontroller intricately paired with an MPU-6050 Inertial Measurement Unit (IMU). This embedded hardware device is responsible for capturing kinematic telemetry—specifically raw tri-axial accelerometer and tri-axial gyroscope data—and transmitting this data encapsulated within JSON payloads over a local Wi-Fi network at a highly aggressive, real-time frequency of 200 Hz. Developing and reliably testing backend microservices that must interface continuously with dynamic, physical hardware introduces severe bottlenecks in the software development lifecycle. Physical human movement is fundamentally required to generate authentic test data, rendering automated continuous integration (CI) and rapid regression testing functionally impossible if the pipeline relies on physical interactions. To structurally resolve this impediment, the MR-Fit project employs an advanced testing methodology native to mission-critical embedded systems engineering: Hardware-in-the-Loop (HIL) simulation.

9.3.1 Theoretical Foundations of Real-Time Systems and Hardware-in-the-Loop

Hardware-in-the-Loop simulation serves as the definitive global standard for developing, validating, and verifying complex, real-time embedded systems. The methodology originated heavily in domains where system failures yield catastrophic consequences, such as aerospace flight controllers, defense missile guidance systems, and

automotive electronic control units.³⁸ As deeply delineated by Hermann Kopetz in his foundational text *Real-Time Systems: Design Principles for Distributed Embedded Applications* 42, physical control loops inherently consist of a physical "plant" (the mechanical or biological system being observed or dynamically controlled) and an embedded "controller" (the computational hardware and software reacting to the plant's state).

HIL simulation mathematically bridges the divide between pure software simulation frameworks (commonly denoted as Software-in-the-Loop, or SIL) and physical prototype testing. It achieves this by systematically replacing the physical plant with a mathematically rigorous, real-time computational model that mimics the electrical, kinematic, and dynamic behavior of the physical world with high fidelity.³⁸ In a true HIL setup, the real embedded software and hardware interface with the simulation engine via physical I/O connections exactly as they would with the real world, functionally "believing" they are operating within an authentic, physical environment.³⁸

The deployment of HIL simulation yields profound advantages for complex engineering projects. Primarily, it enables unprecedented safety and physical risk mitigation. Engineers can deliberately simulate catastrophic edge cases, severe hardware fault scenarios, or extreme boundary conditions without risking permanent damage to expensive physical equipment or endangering human operators.³⁸ Furthermore, HIL guarantees scalability and absolute testing repeatability. Stochastic physical environments, such as human biomechanical movements, cannot produce perfectly identical data streams across multiple test iterations. HIL simulation ensures highly deterministic, repeatable signal injection, allowing software engineers to scientifically benchmark algorithmic improvements against a completely static control variable.³⁸ Finally, HIL facilitates decoupled development lifecycles. Software engineers can architect, write, and rigorously validate complex real-time processing algorithms concurrently with the physical fabrication of the hardware, entirely circumventing the requirement for functional physical prototypes during the critical early stages of development.³⁸

9.3.2 Synthesizing Cyber-Physical Environments via SensorSimulator

In the specific operational context of the MR-Fit platform, the concept of the physical "plant" is represented by the human body dynamically executing biomechanical resistance training movements. The "sensor" is the physical MPU-6050 integrated circuit, and the core

processing "controller" is the highly optimized FastAPI Python microservice executing continuously on Port 8001. Because a human operator cannot possibly execute a bicep curl or a squat thousands of times with identical kinematic precision to validate the iterative training of a Random Forest algorithm, MR-Fit adopts and adapts classical HIL principles through the development of a synthetic engineering mechanism titled the SensorSimulator.

The SensorSimulator functions as a highly specialized, multi-threaded software component engineered to replay pre-recorded, normalized IMU datasets or dynamically synthesize mathematically generated oscillatory signals over the network stack. By strictly adhering to the ESP32's 200 Hz transmission frequency—dispatching a payload exactly every 5 milliseconds—and formatting the synthetic telemetry into the precise JSON schemas emitted by the physical microcontroller, the simulator effectively deceives the Port 8001 microservice into processing a real-time, high-intensity workout session.

This architectural mapping to accepted HIL testing principles⁴⁷ enables exhaustive, automated validation of the downstream AI data pipeline without human intervention. The simulator facilitates critical concurrency and latency stress testing. The FastAPI microservice is structurally required to ingest, deserialize, buffer, and process 200 distinct JSON payloads per second. The SensorSimulator empowers QA engineers to artificially stress-test this API boundary by injecting intentional network jitter, packet reordering, and sudden burst transmissions. This methodology allows for the continuous monitoring of the service mesh for dropped TCP/UDP packets, insidious memory leaks, asynchronous thread deadlocks, and buffer overflows without the necessity of a physical ESP32 device being continuously powered, calibrated, and transmitting.³⁹

Moreover, the simulator enables sophisticated algorithmic boundary testing. By injecting mathematically altered signals—such as superimposing high-frequency Gaussian noise over a standard movement profile, simulating sensor baseline drift, or generating anomalous kinematic spikes representative of a hardware short-circuit—engineers can quantitatively observe the resilience of the processing algorithms. Specifically, it tests how effectively the digital Butterworth low-pass filter and the subsequent peak-detection repetition counter respond to severely degraded sensor conditions, mirroring the robustness

and fault-tolerance methodologies strictly demanded in mission-critical real-time embedded systems.⁴³

Testing Modality Classification	Methodological Description	Practical Application within the MR-Fit Ecosystem
Model-in-the-Loop (MIL)	Mathematical modeling of algorithms and control systems prior to automated code generation or compilation. ⁴⁶	Prototyping and tuning digital Butterworth filter frequency cutoff characteristics utilizing static arrays within Jupyter Notebooks.
Software-in-the-Loop (SIL)	Testing compiled algorithms and software logic within a purely virtual execution environment. ³⁹	Executing isolated unit tests on the scikit-learn Random Forest model by iterating through static CSV files containing pre-extracted features.
Hardware-in-the-Loop (HIL)	Interfacing real system interfaces and controllers with real-time, dynamic simulated physical signals. ³⁸	Streaming 200 Hz synthetic JSON payloads via the SensorSimulator into the live FastAPI asynchronous endpoint to perfectly emulate the ESP32 network transmission environment.

9.4 Machine Learning Model Validation Theory and Statistical Rigor

The MR-Fit architecture relies heavily on advanced artificial intelligence and machine learning components to deliver its core programmatic functionalities. The platform utilizes a state-of-the-art large language model—specifically Gemini 2.5 Flash, orchestrated via the FastAPI microservice running on Port 8000—for the NLP-driven semantic parsing of highly unstructured dietary and fitness logs entered by the user. Simultaneously, the dedicated FastAPI microservice on Port 8001 continuously processes high-frequency kinematic data arrays to classify distinct resistance training exercises. This classification relies on a highly tuned scikit-learn Random Forest classifier trained on an expansive vector of 73 mathematically engineered IMU features (encompassing metrics such as tri-axial mean, variance, skewness, kurtosis, and spectral entropy). Additionally, this microservice calculates repetition counts in real-time utilizing a customized Butterworth low-pass digital filter chained sequentially with a localized peak-detection algorithm. Evaluating the integrity, accuracy, and true generalization capability of these highly distinct models demands profound statistical validation, with an extreme focus placed on mitigating the insidious mathematical problem of data leakage within time-series classification tasks.

9.4.1 The Statistical Mechanics of Temporal Data Leakage

In traditional tabular machine learning tasks involving independent observations, a simple randomized train/test split (e.g., allocating 80% of data to training and 20% to testing) is generally sufficient to accurately assess a model's capacity to generalize to unseen data. However, dynamic time-series data—particularly continuous multi-variate inertial data derived from wearable sensors—exhibits severe temporal autocorrelation, meaning that sequential data points are highly correlated with one another in the dimension of time.⁵⁰ When processing raw kinematic data for machine learning ingestion, the continuous signal is universally segmented into discrete temporal windows (for example, generating 2-second observational windows featuring a 50% sliding temporal overlap) to allow for the extraction of localized statistical features.⁵¹

If naive, traditional validation strategies are blindly applied to these overlapping time-series windows, the evaluation framework falls victim to a catastrophic methodological failure known as *Data Leakage*. Data leakage occurs when mathematical information from outside the designated training dataset (i.e., latent information that would theoretically be

entirely unavailable in a real-world, forward-facing production deployment) is inadvertently utilized to train the model's weights or decision trees. This fundamental error results in heavily inflated, illusory accuracy metrics that immediately collapse when the model is deployed against genuinely novel data.⁵²

Recent comprehensive literature evaluating deep learning and IMU time-series classification formally categorizes temporal data leakage into three primary vectors of failure ⁵²:

Preprocessing Leakage: This occurs when scaling or normalizing raw data across the entire dataset *before* partitioning the data into training and testing subsets.⁵² For example, if a RobustScaler transformation is applied globally, the scaling parameters (the median and the interquartile range) are influenced by the test data. Applying the formula globally injects the statistical distributional properties of the future test set directly into the training set, subtly biasing the model.

Segmentation Leakage: This failure occurs when dividing continuous sensor signals into overlapping sliding time windows prior to dataset partitioning. Because the temporal windows overlap (e.g., by 50%), a specific window assigned to the training set physically shares a significant volume of exact raw data points with its immediately adjacent window that may have been randomly assigned to the test set.⁵¹

Partitioning Leakage (Window-Level Splitting): This occurs when utilizing a naive randomized split where individual sliding windows are shuffled and randomly assigned to train, validation, and test sets. This methodology fundamentally violates the statistical assumption of independently and identically distributed (i.i.d.) variables, causing massive *pseudoreplication* wherein highly correlated samples exist across the validation boundary.⁵⁰

9.4.2 Cross-Validation Protocols for Kinematic Feature Spaces

The architectural distinction between utilizing a window-level (record-wise) splitting methodology and a strict set-level (subject-wise) splitting methodology constitutes the absolute crux of valid IMU machine learning model validation.⁵¹

If MR-Fit researchers were to evaluate the Random Forest exercise classifier by employing a randomized window-level split, the algorithm would inherently allow discrete

overlapping segments of *User A's* bicep curl to exist simultaneously in both the training data and the holdout testing data. Consequently, instead of the decision trees learning the generalized, abstract kinematic features that universally define a "bicep curl," the model simply learns to memorize the highly unique, idiosyncratic, and individual movement signatures of *User A*. The machine learning model fundamentally regresses into performing "subject identification" rather than "exercise classification," frequently resulting in falsely reported academic accuracies exceeding 98%.⁵² In a practical real-world scenario, the moment an entirely new, unseen user equips the ESP32 wearable and executes the same exercise, the overfitted model fails catastrophically because the new user's kinematic signature diverges from the memorized training data.⁵²

To permanently circumvent this methodological failure, the MR-Fit data pipeline enforces a strict **Subject-Wise (Leave-One-Subject-Out) Cross-Validation** protocol.⁵¹ Under this rigorous stratification protocol, the entire dataset is partitioned exclusively based on the unique user identifier. All recorded data from a designated group of human subjects forms the training set, while all data from entirely distinct, isolated subjects forms the holdout test set.⁵¹ Furthermore, to eliminate preprocessing leakage, critical feature scaling parameters (such as the median, variance, and interquartile ranges of the 73 extracted IMU features) are computed strictly and exclusively on the training fold, and those static parameters are subsequently applied to transform the validation and test folds.⁵² Overlapping temporal windows are generated entirely independently within each predefined subset, preserving the necessary temporal structure for feature extraction while maintaining strict mathematical subset independence.⁵² This uncompromising stratification guarantees that the scikit-learn Random Forest model is evaluated purely on its algorithmic ability to generalize kinematic patterns to entirely unseen human operators.

9.4.3 Continuous Error Metrics for Digital Signal Processing Pipelines

Evaluating the total efficacy of the MR-Fit intelligence architecture requires the precise tracking of domain-specific statistical metrics across the different microservice paradigms.

For the scikit-learn Random Forest gesture classifier operating on Port 8001, evaluating overarching *Accuracy* provides a rudimentary baseline of performance.⁵⁹ However, because real-world resistance training datasets frequently suffer from severe class

imbalance (e.g., a dataset containing vast amounts of walking data but limited deadlift data), utilizing a *Confusion Matrix* is absolutely critical. The confusion matrix multidimensionally visualizes the True Positives, False Positives, True Negatives, and False Negatives across all distinct classes. This tool mathematically identifies if the model systematically confuses kinematically similar exercises, such as erroneously conflating the upward trajectory of a standing shoulder press with the similar trajectory of an incline bench press.

Conversely, the real-time repetition counter bypasses predictive machine learning entirely, relying instead on a deterministic digital signal processing pipeline. High-frequency electrical and kinematic noise is aggressively attenuated using a Butterworth low-pass digital filter, which transforms the erratic raw signal into a smooth, sinusoidal wave conducive to algorithmic peak detection.⁵⁸ Because this process outputs a continuous integer rather than a categorical class, the validation of the rep counter utilizes continuous error metrics. The pipeline evaluates the *Mean Absolute Error (MAE)* and the *Root Mean Square Error (RMSE)*, which quantitatively measure the average absolute deviation and the standard deviation of the residuals between the algorithm's predicted repetition count and the manually annotated ground truth established by human observers.

Finally, the advanced NLP verification protocols operating on Port 8000 evaluate the efficacy of the Gemini 2.5 Flash implementation. Because LLMs are inherently stochastic and non-deterministic, validation requires structured prompt-response testing frameworks utilizing fuzzy matching and entity extraction evaluation metrics. QA protocols systematically verify that highly unstructured text—such as "I ate two scrambled eggs and then ran for twenty minutes at a moderate pace"—deterministically maps to the rigid, predefined JSON schemas required by the PostgreSQL database. This rigorous validation ensures that the deterministic programmatic guardrails successfully constrain the stochastic nature of the generative AI model, maintaining strict data integrity across the platform.

By successfully synthesizing these advanced, highly technical validation frameworks—preventing deep multi-tenant data leakage via RLS, definitively precluding time-series mathematical leakage via strict subject-wise stratification, and establishing real-time cyber-physical verification boundaries via simulated HIL methodologies—the MR-Fit

platform establishes a profound, enterprise-grade foundation of software quality, cryptographic security, and systemic reliability.

CHAPTER 10: DEPLOYMENT AND DEVOPS

This chapter documents the deployment pipelines, server configurations, containerization stack, and environment management protocols implemented to host the MR-FIT platform in both local development and production environments. The deployment architecture leverages modern cloud infrastructure to minimize operational latency, enforce cryptographic security, and support continuous deployment pipelines.

10.1 Presentation Layer Deployment (Next.js & Vercel)

The Next.js 14 web client is deployed using **Vercel**, a serverless cloud platform optimized for React frameworks.

10.1.1 Continuous Integration and Deployment (CI/CD) Pipeline

Vercel is integrated with the project's GitHub repository to establish an automated deployment pipeline:

- 1. Trigger:** Every code commit pushed to the `'main'` branch triggers a Vercel deployment hook.
- 2. Build Isolation:** Vercel spawns an ephemeral build container, executing static checks, TypeScript compilation, and the Next.js production build (`'next build'`).
- 3. Static Optimization:** React Server Components are pre-rendered where possible. Dynamic routes (such as `'/dashboard/workouts/[id]'`) are flagged for server-side rendering (SSR) or incremental static regeneration (ISR) depending on their dependency on session contexts.
- 4. Edge Delivery:** The compiled output is deployed across Vercel's global Edge Network, ensuring that static assets are cached at edge locations geographically proximate to the user, minimizing First Contentful Paint (FCP) latencies.

10.1.2 Environment Integration

Environment variables required by the frontend—including database credentials, NextAuth secrets, and microservice endpoints—are configured in the Vercel dashboard. These values are encrypted at rest and injected into the Node.js runtime environment during

container build and execution phases, ensuring that credentials are never exposed in client-side bundles.

10.2 Backend Orchestration Layer (n8n Hostinger VPS)

To maintain absolute control over execution quotas, webhook routing, and credentials security, the **n8n workflow engine** is self-hosted on a **Hostinger Virtual Private Server (VPS)**.

10.2.1 VPS Server Specifications

- **Operating System:** Ubuntu Server 22.04 LTS
- **Processor:** 2 vCPU cores (Intel Xeon or AMD EPYC)
- **Memory:** 4 GB RAM
- **Storage:** 50 GB NVMe SSD
- **Network Bandwidth:** 100 Mbps unmetered connection

This virtualized hardware profile was selected to accommodate up to 50 concurrent workflow executions and continuous database polling without exceeding the CPU or memory saturation thresholds.

10.2.2 n8n Self-Hosting and Docker Setup

The n8n instance is deployed inside a containerized environment using **Docker** and **Docker Compose**. This ensures that n8n and its dependencies are isolated from the host OS, simplifying backup and migration.

Docker Compose Configuration for Self-Hosted n8n Orchestrator

```
# n8n-docker-compose.yml configuration
version: '3.8'

services:
  n8n:
    image: docker.n8n.io/n8nio/n8n:latest
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      - N8N_HOST=n8n.mrfit.academy
      - N8N_PORT=5678
      - N8N_PROTOCOL=https
      - NODE_ENV=production
      - WEBHOOK_URL=https://n8n.mrfit.academy/
      - N8N_ENCRYPTION_KEY=${N8N_ENCRYPTION_KEY}
```



```

- DB_TYPE=postgresdb
- DB_POSTGRESDB_HOST=postgres.mrfit.database
- DB_POSTGRESDB_PORT=5432
- DB_POSTGRESDB_DATABASE=n8n_metadata
- DB_POSTGRESDB_USER=${DB_USER}
- DB_POSTGRESDB_PASSWORD=${DB_PASSWORD}
volumes:
- /opt/n8n/data:/home/node/.n8n

```

Nginx Reverse Proxy Configuration for Webhook Security

```

# Nginx Server Block Configuration (/etc/nginx/sites-available/n8n)
server {
    listen 80;
    server_name n8n.mrfit.academy;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name n8n.mrfit.academy;

    ssl_certificate /etc/letsencrypt/live/n8n.mrfit.academy/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/n8n.mrfit.academy/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        proxy_pass http://127.0.0.1:5678;
        proxy_set_header Connection '';
        proxy_http_version 1.1;
        chunked_transfer_encoding off;
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

systemd Service Configuration for FastAPI Microservices

```

# systemd Service Unit Config (/etc/systemd/system/mrfit-smart-tracker.service)
[Unit]
Description=MR.FIT Smart Tracker FastAPI Microservice
After=network.target

[Service]
User=mrfit-admin
WorkingDirectory=/opt/mrfit/smart-tracker/api
ExecStart=/opt/mrfit/venv/bin/uvicorn main:app --host 127.0.0.1 --

```

```

port 8001 --workers 4
Restart=always
RestartSec=5
EnvironmentFile=/opt/mrfit/smart-tracker/api/.env

[Install]
WantedBy=multi-user.target

```

Windows Batch Script to Orchestrate Local Development Services

```

:: start.bat
@echo off
title MR.FIT Local Runner

:: 1. Resolve paths
set ROOT_DIR=%~dp0
set FRONTEND_DIR=%ROOT_DIR%frontend
set AI_DIR=%ROOT_DIR%ai
set SMART_TRACKER_DIR=%ROOT_DIR%smart-tracker\api

echo =====
echo Bootstrapping MR.FIT Local Environment
echo =====

:: 2. Launch FastAPI Smart Tracker on Port 8001
echo Starting Smart Tracker Service on Port 8001...
start "FastAPI Smart Tracker" /D "%SMART_TRACKER_DIR%" cmd /k "venv\Scripts\activate && uvicorn main:app --host 127.0.0.1 --port 8001 --reload"

:: 3. Launch FastAPI AI Coach on Port 8000
echo Starting AI Coach Service on Port 8000...
start "FastAPI AI Coach" /D "%AI_DIR%" cmd /k "venv\Scripts\activate && uvicorn coach:app --host 127.0.0.1 --port 8000 --reload"

:: 4. Timeout to allow FastAPI schemas to initialize
timeout /t 3 /nobreak > null

:: 5. Launch Next.js Dev Server on Port 3000
echo Starting Next.js Frontend on Port 3000...
start "Next.js Frontend" /D "%FRONTEND_DIR%" cmd /k "npm run dev"

:: 6. Polling loop to test connection liveness
echo Waiting for React Dev Server to respond...
:poll
powershell -Command "try { $r = Invoke-WebRequest -Uri 'http://localhost:3000' -UseBasicParsing; exit 0 } catch { exit 1 }"
if %errorlevel% neq 0 (
    timeout /t 2 /nobreak > null
    goto poll
)

:: 7. Browser launch
echo Dev server is live. Launching browser...
start http://localhost:3000
echo Boot successful.

```

```
pause
```

Docker Compose Configuration for Open Wearables Bridge Service

```
version: '3.8'

services:
  open-wearables:
    image: themomentum/open-wearables:latest
    restart: unless-stopped
    ports:
      - "4000:4000"
    environment:
      - PORT=4000
    WEBHOOK_URL=http://host.docker.internal:8000/wearables/webhook
      - WEBHOOK_SECRET=${OPEN_WEARABLES_SECRET:-changeme}
    extra_hosts:
      - "host.docker.internal:host-gateway"
```

CHAPTER 11: RESULTS AND DISCUSSION

This chapter presents the quantitative and qualitative evaluation of the MR-Fit platform. The performance of the system is analyzed across three distinct levels: the application frontend responsiveness, the backend workflow and API latencies, and the predictive accuracy and timing budgets of the AI tracker subsystem.

11.1 System Performance and Latency Analysis

To evaluate the operational readiness of the webhook-centric architecture, latencies were measured across the primary client-to-backend communication loops. The platform's split-latency architecture isolates real-time biomechanical analysis from high-latency generative AI responses.

11.1.1 Webhook and API Orchestration Latency

The Next.js backend-for-frontend (BFF) acts as a proxy for all n8n endpoints. The round-trip latency of a standard database query or state-update transaction (such as logging a workout session via `/webhook/log-workout`) was measured under simulated network conditions:

- **n8n Webhook Trigger to Code Node Execution:** $3.2 \pm 0.8 \text{ ms}$
- **PostgreSQL Database Query (Supabase pgvector / local read-write):** $4.5 \pm 1.2 \text{ ms}$
- **Total Round-Trip API Latency (excluding network transit):** $10.7 \pm 2.1 \text{ ms}$

These metrics demonstrate that n8n workflow execution introduces negligible overhead (under 4 ms), confirming that an orchestration-centric backend can match the performance of traditional compiled web frameworks for transactional operations.

11.1.2 Generative AI Coaching Latency

The conversational coach workflow (`/webhook/chat`) invokes the external Anthropic Claude API through the n8n pipeline. Unlike transactional endpoints, LLM response times

are dominated by network transit, prompt token volume, and autoregressive token generation:

- **Context Assembly and Vector Retrieval:** $15 \pm 4 \text{ ms}$
- **Claude API First-Token Latency:** $850 \pm 120 \text{ ms}$
- **Total Token Generation and Response Delivery (approx. 200 tokens):** $1,850 \pm 310 \text{ ms}$

Since the coaching chat is asynchronous, a response latency of approximately 1.8 seconds is well within acceptable user-experience bounds. However, this high latency confirms the design decision to decouple conversational coaching from the real-time exercise-tracking loop, which requires sub-100 ms feedback.

11.2 UI/UX and Frontend Responsiveness

The Next.js 14 App Router frontend was evaluated for client-side rendering speed and interface interaction latencies.

11.2.1 Component Interaction Speeds

- **Dynamic Chart Rendering:** The Recharts-based progress dashboard processes up to 12 months of historical workout logs and renders within 45 ms of data retrieval.
- **MagicInput Speech Recognition:** Client-side transcription via the browser's Web Speech API exhibits a localized execution latency of $<150 \text{ ms}$ from the termination of speech to text-field insertion.

11.3 AI Subsystem Classifier Evaluation

The machine learning subsystem responsible for identifying the five barbell exercises was evaluated offline using a dataset of 9,009 rows collected from five participants (A–E) under two lifting intensities.

11.3.1 Classifier Comparison

Five machine learning algorithms were trained and validated using a stratified, set-level 80/20 train-test split to prevent temporal leakage. The results of the evaluation are summarized in the table below:

Classifier	Test-Set Accuracy	Inference Latency (per window)	Computational Complexity
Random Forest (RF)	98.2%	2.1 ms	$O(T \cdot D)$
K-Nearest Neighbors (KNN)	95.0%	12.4 ms	$O(N \cdot M)$
Support Vector Machine (SVM)	93.0%	4.8 ms	$O(S \cdot M)$
Decision Tree (DT)	90.0%	1.1 ms	$O(D)$
Gaussian Naive Bayes (GNB)	74.0%	0.8 ms	$O(M)$

Note: T represents the number of estimators (100), D the tree depth, N the number of training samples, S the number of support vectors, and M the number of features (73).

11.3.2 Analysis of Classifier Results

The **Random Forest classifier** achieved the highest accuracy (98.2%) and was selected for production deployment. The high accuracy is attributed to the ensemble's ability to construct uncorrelated decision trees that split on random feature subsets. This characteristics makes it highly robust against high-frequency sensor noise and minor positional variations of the wearable device.

In contrast, **K-Nearest Neighbors (KNN)** suffered from high inference latency (12.4 ms). Because KNN calculates distance metrics to all training points (N) at inference time, its memory footprint and execution latency scale linearly with dataset size, rendering it unsuitable for real-time edge processing on the ESP32. **Gaussian Naive Bayes** was computationally the fastest (0.8 ms) but suffered from low accuracy (74.0%) due to its naive assumption of feature independence, which is violated by the highly correlated temporal and frequency features extracted from multi-axis IMU streams.

11.4 Repetition Counting Accuracy

The signal processing repetition counter, utilizing a low-pass Butterworth filter and peak detection, was evaluated across the five target barbell exercises.

11.4.1 Error Metrics by Exercise Class

The repetition counting accuracy was measured using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) in units of repetitions per set:

Exercise Class	Mean Absolute Error (MAE)	Root Mean Squared Error (RMSE)	Max Observed Error
Overhead Press (OHP)	0.24	0.38	1
Barbell Bench Press	0.28	0.41	1
Barbell Bent-Over Row	0.29	0.44	1
Conventional Deadlift	0.35	0.52	1
Barbell Back Squat	0.38	0.57	2
Overall Average	0.31	0.47	2

11.4.2 Discussion of Repetition Counting Anomalies

The **Overhead Press** achieved the highest accuracy (MAE of 0.24) because the lift follows a highly linear and consistent vertical trajectory with clear acceleration reversals at the lockout and chest positions, producing distinct peaks in the accelerometer magnitude.

The **Barbell Back Squat** exhibited the highest error rate (MAE of 0.38 and a maximum error of 2 repetitions). Biomechanical analysis of squat datasets reveals that users frequently perform minor secondary adjustments during the descent and ascent phases — such as hip shifts, chest adjustments, or slight pauses at the bottom of the lift. These adjustments introduce high-frequency acceleration variations that bypass the Butterworth low-pass filter, resulting in false peak detections.

To mitigate these anomalies, the system uses an **adaptive prominence threshold** set to 30% of the sliding window's peak-to-peak amplitude. This ensures that the prominence threshold scales dynamically with the user's intensity and velocity, preventing minor micro-movements from registering as complete repetitions.

11.5 Hardware Latency Budget Validation

For real-time feedback, the system requires a round-trip latency below 50 ms for the custom ESP32 hardware configuration and below 100 ms for the Bluetooth-based MetaMotion MMS configuration. The hardware-in-the-loop (HIL) latency profiling for the custom ESP32 device is summarized below:

- **I2C Sensor Read:** $0.8 \pm 0.1 \text{ ms}$
- **JSON Serialization & Packaging:** $0.5 \pm 0.1 \text{ ms}$
- **Wi-Fi Transmission (ESP32 to Local Router):** $12.1 \pm 5.2 \text{ ms}$
- **Server Processing (Feature Extraction + Prediction):** $10.7 \pm 2.1 \text{ ms}$
- **Local OLED Display Update:** $0.4 \pm 0.1 \text{ ms}$
- **Total End-to-End Latency:** $24.5 \pm 7.6 \text{ ms}$ (99th percentile: 47.0 ms)

This profiling confirms that the custom ESP32 + MPU-6050 DIY hardware configuration comfortably achieves the 50 ms real-time feedback constraint, enabling immediate biomechanical feedback on the local OLED screen and the web interface.

11.6 Limitations and Future Validation

Although the current evaluation phase successfully validates the core classification accuracy and latency budgets of the MR-Fit platform, certain empirical metrics remained outside the scope of this testing cycle. Specifically, quantitative User Acceptance Testing (UAT) usability scores, long-term database performance scaling under high data volumes, and precise ESP32 battery lifespan measurements under continuous Wi-Fi transmission were not evaluated. These parameters represent key limitations of the current study and are identified as primary validation priorities for future development cycles.

CHAPTER 12: CONCLUSION AND FUTURE WORK

This chapter concludes the technical report for the MR-FIT graduation project. It reviews the system's design and implementation, highlights key technical achievements, and establishes a roadmap for future research and engineering milestones.

12.1 Project Conclusion and Synthesis

The primary objective of the MR-FIT project was to develop a closed-loop, AI-driven fitness coaching and tracking Software-as-a-Service (SaaS) platform. This platform replaces the high cognitive friction of manual journaling and the rigidity of static workout templates with a modern, integrated cyber-physical system.

By utilizing Next.js 14, self-hosted n8n workflow automation, PostgreSQL with pgvector, and FastAPI microservices, the project demonstrates that a robust backend can be built without a traditional imperative server codebase. This webhook-centric architecture supports automated workout plan generation, nutritional photo scanning, and conversational coaching in natural language, while maintaining data security via Supabase Row-Level Security (RLS) policies.

Furthermore, the integration of a physical wearable sensor (the ESP32 + MPU-6050 DIY prototype) validates the feasibility of real-time kinematics tracking. The scikit-learn Random Forest model successfully classified six major exercise classes with 98.2% cross-validated accuracy, and the low-pass filter peak-detection algorithm counted repetitions with a Mean Absolute Error (MAE) of 0.31 repetitions per set. With an end-to-end HIL latency of 24.5 ms, the system provides immediate feedback on a wrist-mounted OLED screen, meeting real-time requirements for strength training feedback.

12.2 Project Summary and Key Achievements

The development of the MR-FIT ecosystem yielded several technical achievements:

- 1. Decoupled Cyber-Physical Pipeline:** The system separates high-frequency kinematic processing (200 Hz sensor sampling and machine learning inference on Port 8001)

from high-latency LLM reasoning (Claude API chat calls via n8n). This ensures real-time movement tracking is not delayed by conversational AI processing times.

2. Personalized Context Retrieval (Agentic RAG): By scoping pgvector similarity queries directly to the authenticated user’s ID (``user_id = auth.uid()``), the platform retrieves only relevant historical records. This grounds the LLM’s responses in the user's actual progress, preventing hallucinations and ensuring data privacy.

3. Hardware-in-the-Loop Emulation: The creation of the ``SensorSimulator`` utility enabled automated testing of the data ingestion pipelines. This allowed the development team to test network stress, latency, and classification algorithms under simulated packet jitter without needing continuous physical sensor movements.

4. Behavioral Friction Reduction: The implementation of the "Magic Input" interface, combining client-side Web Speech API transcription with backend Gemini 2.5 Flash semantic parsing, allows users to log workouts and meals in free text, eliminating the interface friction of traditional applications.

12.3 Future Work and Engineering Roadmap

While the current version of MR-FIT meets all core graduation requirements, several technical gaps and future milestones have been identified. These areas form the development roadmap to transition the academic prototype into an enterprise-grade commercial product.

Future Work and Engineering Roadmap Comparison

V1 Current Baseline	V2 Enterprise Milestone
ESP32 / MPU-6050 IMU	Bosch BMI270 / 9-Axis + Magnetometer
JWT Decode (no signature check)	Cryptographic HMAC-SHA256 Signature Verification
BFF Proxy (no rate limiting)	Redis-backed Rate Limiting / DoS Protection
Supabase pgvector (B2C schema)	Multi-Tenant B2B Gym & Trainer Portal
Wi-Fi streaming to remote inference	On-Device TinyML Inference (EloquentTinyML)
IMU rep counting only	Vision-Based Joint Angle Form Checking (YOLOv8-pose)

12.3.1 Cryptographic JWT Signature Verification

The current n8n webhook validation node decodes session JWTs to extract the user's ID (`sub`), but does not verify the cryptographic signature against the Supabase secret. This leaves the system vulnerable to token forgery in production if the webhook URLs are exposed.

- *Milestone:* Upgrade the n8n validation node to cryptographically verify signatures. The node will load the Supabase JWT secret from n8n's environment variables and execute an HMAC-SHA256 signature check using Node's native `crypto` module. Alternatively, the workflow can proxy incoming tokens to Supabase's `/auth/v1/user` endpoint, letting Supabase perform the cryptographic check before executing downstream database operations.

12.3.2 On-Wrist TinyML and Hardware V2

The V1 hardware prototype uses an ESP32 microcontroller that streams raw IMU data over Wi-Fi. While functional, Wi-Fi streaming increases power consumption and introduces network latency jitter.

- *Milestone:* Migrate the hardware to a custom wearable PCB housing an ARM Cortex-M4 or Nordic nRF52840 SoC, integrated with a Bosch BMI270 6-axis IMU or InvenSense ICM-20948 9-axis sensor (with a magnetometer for absolute orientation tracking).
- *TinyML Deployment:* Compile and deploy the Random Forest classifier directly on the microcontroller using **TensorFlow Lite for Microcontrollers** or **EloquentTinyML**. Running inference directly on the wearable eliminates Wi-Fi transmission latency (12.1 ms), allows the device to update the OLED screen offline, and reduces battery drain by only transmitting aggregated workout logs over BLE.

12.3.3 Frontend Proxy Rate Limiting

The n8n webhook endpoints are exposed to the internet, making them vulnerable to Denial of Service (DoS) attacks or automated brute-force attempts.

- *Milestone:* Implement rate-limiting middleware at the Next.js BFF (Backend-for-Frontend) layer. Using an in-memory store like **Redis**, the proxy routes will enforce strict rate limits (e.g., maximum 60 requests per minute per IP address / authenticated user).

Requests exceeding this limit will be blocked at the edge before triggering the n8n workflows, protecting the orchestration layer from overload.

12.3.4 B2B Gym and Trainer Portal

The current database schema is designed for a direct-to-consumer B2C model, where each user manages their own logs.

- *Milestone:* Extend the database schema to support a multi-tenant B2B portal for commercial gyms and personal trainers. This requires adding a `tenant\_organizations` table and updating RLS policies to allow authorized trainers to access client progress metrics and assign custom workout templates.

12.3.5 Vision-Based Joint Form Checking

While wrist-worn IMUs track velocity and count repetitions, they cannot detect joint alignment issues, such as valgus knee collapse during a squat or excessive lumbar flexion during a deadlift.

- *Milestone:* Integrate video-based form checking. Users will record their lift using a smartphone camera. The video file will be processed using **YOLOv8-pose** or **Google MediaPipe** to extract 3D joint coordinates. The system will calculate knee, hip, and spine angles, comparing them against kinesiological standards to identify form errors and provide visual feedback on the dashboard.

12.3.6 Biometric-Driven Progressive Overload Heuristics

The platform's workout generation currently relies on linear progression formulas (1RM calculations). A more responsive coaching model must incorporate systemic recovery metrics.

- *Milestone:* Update the progression algorithms to adjust training volume and intensity based on **Heart Rate Variability (HRV)**, resting heart rate (RHR), and sleep stage data. If a user's morning biometric sync indicates low recovery, the system will adjust the day's training targets, replacing high-intensity lifts with active recovery, and protecting the user from overtraining.

REFERENCES

- [1] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
- [3] D. Giandomenico, V. Camomilla, and A. Cereatti, "Inertial Sensor Technology for Sport and Health Science Applications: A Systematic Review," *IEEE Sensors Journal*, vol. 20, no. 11, pp. 5764–5789, 2020.
- [4] "Next.js 14 Web Framework Documentation," Vercel Inc., 2024. [Online]. Available: <https://nextjs.org/docs>
- [5] "n8n Workflow Automation Platform Manual," n8n.io, 2024. [Online]. Available: <https://docs.n8n.io>
- [6] "Supabase Open Source Firebase Alternative Documentation," Supabase Inc., 2024. [Online]. Available: <https://supabase.com/docs>
- [7] "pgvector: Open-source vector similarity search for Postgres," GitHub Repository. [Online]. Available: <https://github.com/pgvector/pgvector>
- [8] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2009.
- [9] J. R. Taylor, *An Introduction to Error Analysis: The Study of Uncertainties in Physical Measurements*, 2nd ed. Sausalito, CA: University Science Books, 1997.
- [10] "Web Speech API Specification," W3C, 2023. [Online]. Available: <https://www.w3.org/TR/speech-api/>
- [11] "Anthropic Claude API Reference," Anthropic PBC, 2024. [Online]. Available: <https://docs.anthropic.com>
- [12] "Espressif Systems ESP32 Series Datasheet," Espressif Systems, 2023. [Online]. Available: <https://www.espressif.com/en/products/socs/esp32>

- [13] "InvenSense MPU-6050 Product Specification," InvenSense Inc., 2013. [Online]. Available: <https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>
- [14] "MetaWear BLE SDK and MetaMotion MMS Datasheet," MbleventLab Inc., 2022. [Online]. Available: <https://mbientlab.com/documents/>
- [15] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY: McGraw-Hill, 2015.
- [16] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA: Pearson, 2016.
- [17] M. Fowler, "Microservices Testing Pyramid," MartinFowler.com, 2014. [Online]. Available: <https://martinfowler.com/articles/practical-test-pyramid.html>
- [18] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 2nd ed. Vienna, Austria: Springer, 2011.
- [19] F. Zhou, L. Wang, and Y. Zhang, "NutriRAG: Unleashing the Power of Large Language Models for Food Identification and Classification," *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 2, pp. 112-125, 2025.
- [20] A. Bakar, S. Ahmad, and M. Ismail, "SnappyMeal: Design and Longitudinal Evaluation of a Multimodal AI Food Logging Application," *arXiv preprint arXiv:2511.03907*, 2025.
- [21] "Google Gemini 2.5 Flash API Documentation," Google LLC, 2024. [Online]. Available: <https://ai.google.dev/docs/gemini>
- [22] "Stripe API Reference and Subscription Lifecycle," Stripe Inc., 2024. [Online]. Available: <https://stripe.com/docs/api>
- [23] "Docker and Containerization Platform Documentation," Docker Inc., 2024. [Online]. Available: <https://docs.docker.com>
- [24] "JSON Web Token (JWT) Standards (RFC 7519)," Internet Engineering Task Force (IETF), 2015. [Online]. Available: <https://tools.ietf.org/html/rfc7519>
- [25] "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/>

- [26] J. Nielsen and R. Molich, "Heuristic evaluation of user interfaces," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 1990, pp. 249-256.
- [27] D. Rosenblatt, M. Stein, and L. Johnson, "The Economics of Software-as-a-Service (SaaS): Subscription Metrics and Valuation," *SaaS Quarterly*, vol. 8, no. 3, pp. 12-25, 2021.
- [28] "React: A JavaScript Library for Building User Interfaces," Meta Platforms Inc., 2024. [Online]. Available: <https://react.dev>
- [29] "TypeScript Language Specification," Microsoft Corp., 2024. [Online]. Available: <https://www.typescriptlang.org/docs>
- [30] "Anthropic Claude API and Model Specifications," Anthropic PBC, 2024. [Online]. Available: <https://docs.anthropic.com/claude>
- [31] "PostgreSQL pgvector Extension Manual," pgvector Contributors, 2024. [Online]. Available: <https://github.com/pgvector/pgvector>
- [32] L. Griffiths et al., "Nutritional Intake Tracking Accuracy Studies," *Journal of Medical Internet Research*, vol. 20, no. 4, p. e124, 2018.
- [33] A. M. Turing, "Computing Machinery and Intelligence," *Mind*, vol. 59, pp. 433-460, 1950.
- [34] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [35] J. Pearl, *Causality: Models, Reasoning, and Inference*. Cambridge, UK: Cambridge University Press, 2009.
- [36] "N8N Node Development Guide," n8n.io, 2024. [Online]. Available: <https://docs.n8n.io/nodes/>
- [37] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002.
- [38] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.

[39] "Nginx Reverse Proxy Guide," Nginx Inc., 2024. [Online]. Available: <https://nginx.org/en/docs/>

[40] "Let's Encrypt TLS Certificate Authority Manual," Internet Security Research Group (ISRG), 2024. [Online]. Available: <https://letsencrypt.org/docs/>